



ECOEATS

Agile Software Project (CM2020-01) Report

Team 64

Cuong (220517197)
Jacqueline (220570468)
Ng Nicole (220570398)
Thien Tran (220517186)
Xiang En (220516466)

The Report:

An AGILE Process and Progress Report
covering the team's endeavours in the
creation of the ECOEATS mobile application.



TABLE OF CONTENTS

TABLE OF CONTENTS	2
BACKGROUND	5
Introduction	5
Aims And Objectives	6
Scopes And Limitations	6
Scope	6
Limitations	7
Deviation From Proposal	7
PLANNING AND RESEARCH	8
Literature Review	8
Sustainability and Food Waste Reduction	8
Mobile Applications for Community Engagement	8
User Management and Security	8
Market Research	9
Project Management Planning	10
Timeline Remarks - Gantt Chart	10
Work Breakdown Structure	12
Approach Taken to Development	17
Research Done - Trello (Kanban Board)	18
Team Composition	19
Work Allocation	19
PROTOTYPING AND ITERATION	21
Iteration of Prototypes	22
Mid-Fidelity Prototype	23
First Iteration	23
Second Iteration	23
High-Fidelity Prototype	26
First Iteration	26
Second Iteration	29
Third Iteration	31
Factors To Consider	36
User testing	36
Phase one: Basic functionality and layout	36
Phase two: Specific features and overall satisfaction	40
DESIGN	45
Front-end design	45
Back-end design	47
SYSTEM DEVELOPMENT	50
Product's core components	50
Share Page	50
Explore Page	50
Deals Page	51

User Page	51
Approach taken with Test-Driven Development	52
Experience highlights	53
React Native Setup	53
Learning Curve with React Native	53
Frontend Development Challenges	53
Backend Development Challenges	55
Code challenges	56
Frontend Product Concept Issues:	56
Backend Product Concept Issues	57
ANALYSIS	58
Code Solutions to Aims and Objectives	58
User Management and Authentication	58
Social Interaction and Community Engagement	58
Redeem Slider	58
Methodologies and Their Impact	59
Agile Development	59
User-Centred Design (UCD)	59
EVALUATION	60
Development Processes	60
Agile Methodology	60
Collaboration and Communication	60
Technical Challenges	60
Testing and Iteration	61
Team and Methodological Reflection	61
Final Product Evaluation	61
User Interface (UI) and Experience (UX)	61
Backend and Database Management	61
Security and Data Handling	62
Future Work and Improvements	62
Scalability and Cloud Integration	62
User Feedback and Personalization	62
Design Cohesion	62
Extended Security Measures	62
CONCLUSION / SUMMARY	63
APPENDIX	64
Work history	64
Cuong	64
Nicole	66
Xiang En	67
Thien Tran	68
Jacqueline	69
GitHub page	69
Project references	69

Libraries installed	69
Website References	70
GitHub Desktop (Version Control)	71
Design Details	72
Development Process Details	86
Required modules	86
React Native	86
SQLite	86
Axios	87
Imgur API	87
Application Structure	88
Code Details	89
Resources Used	89
Front-end Resources	89
Back-end Resources	91

BACKGROUND

Introduction

Food waste is a growing global concern, contributing to environmental degradation, economic loss, and food insecurity. With increasing awareness around sustainability and resource management, many communities and organisations are seeking innovative ways to reduce waste and encourage more efficient food distribution. The EcoEats project aims to address this issue by developing a mobile platform where users can share surplus food items, reducing the amount of food waste at a local level.

The application allows users to post, discover, and claim deals on surplus food, fostering community engagement and promoting sustainability. The EcoEats app is designed for individuals and businesses to manage surplus food efficiently, offering a seamless experience. EcoEats emphasises both **eco-friendliness** and **economic benefits**—by providing a space for people to share food that might otherwise go to waste, the platform helps minimise environmental impact while also offering cost-effective solutions for consumers.

After much consideration we decided to implement a mobile application instead of a website as it would be more convenient and easy to use on the go for our users, allowing users to interact with the platform anytime, anywhere. The development of this application involved extensive research, prototyping, and user testing to create a user-friendly interface that meets the needs of both givers and receivers within the community. This report details the planning, design, and implementation of the EcoEats platform by effectively dividing the key tasks among us, as well as the challenges faced and how we overcame them, with a focus on its potential to drive both ecological and economic benefits.

Aims And Objectives

The aim of the EcoEats project is to create a mobile platform that not only reduces food waste but also promotes environmentally sustainable practices while providing affordable food options for users. By encouraging users to share surplus food and items, we can foster a more sustainable food culture where excess food and items can be shared instead of discarded.

The key objectives of the EcoEats project are to design and build a mobile application with intuitive navigation that allows users to easily post and discover surplus food deals. We also aim to create a secure and efficient system where users can share food items with others, including features like posting food details, and images. Another objective is to promote community-based food sharing by incorporating features such as a follower system, comments, and likes, which encourage user interaction and engagement within the app. We also aim to implement robust authentication methods, such as password hashing, ensuring that user data is securely stored and managed within the platform. Another objective is to contribute to the reduction of food waste by providing a platform that enables surplus food redistribution, aligning with sustainable practices and helping users reduce their environmental footprint. We want to integrate a reliable and scalable backend infrastructure that ensures smooth data handling, and secure storage of user and food deal information and to iterate on the platform based on user testing, ensuring that the final product meets the needs and expectations of its target audience, and continuously improves user experience.

Scopes And Limitations

Scope

Our project aims to provide a comprehensive mobile platform designed to reduce food waste by enabling users to share surplus food and access affordable food deals within their communities. The primary focus of the application is to encourage eco-friendly and economically friendly practices, ensuring that users can participate in food sharing while minimising environmental impact. The platform is designed with several key features, including secure registration and login processes, allowing users to create and manage profiles, post food deals, and engage with other users and a dedicated space where users can post surplus food or items with details and images, such as expiring dates, offering them for free or at reduced prices. We also implemented features that allow users to discover, post and search for cost-effective food and activity recommendations, and share with other users their experiences and thoughts by using comments, reply comments and comment likes and businesses can list food items offering discounts or deals such as 'buy 1 get 1 free' for surplus items that may be expiring soon. Users can redeem the deals with the redemption slider that can only be used once. We also implemented a community-based engagement system that includes following, commenting, and liking posts to enhance interaction between users and the mobile application is designed to work seamlessly across Android platforms, providing users with a consistent and convenient experience.

Limitations

While the EcoEats platform has successfully implemented its core features, there are certain limitations that arose during development due to resource and time constraints. Initially intended to help users find food deals based on their location, this feature was not implemented due to the complexity of its integration and the need to focus on the core functionalities. The app also currently lacks a Chatbox Using RESTful API feature, this was envisioned to allow direct communication between users within the app. However, it was omitted from this version as we decided to prioritise basic functionalities and ensure they were robust before adding additional features.

Deviation From Proposal

The original project proposal included plans to integrate geolocation services and a chatbox using a RESTful API. However, as the project progressed, we realised that due to limited resources and time constraints, we needed to focus on core functionalities instead.

Initially, we planned to implement EcoEats as a website, but after considering user needs and the platform's purpose, we decided that a mobile app would be a better fit. Although our team had no prior experience developing a mobile app, we believed it would provide more convenience for users.

Mobile apps also tend to have higher engagement rates than websites. Features like push notifications can remind users of new deals or relevant posts, increasing user retention and interaction. This continuous engagement is crucial for a community-driven platform like EcoEats. Additionally, a mobile app offers access to native device features, such as the camera, enabling users to easily capture and upload images of the food they are sharing. The potential for GPS integration allows for location-based food sharing, further improving user convenience.

In terms of performance, mobile apps generally provide a smoother and more responsive experience than websites. Since EcoEats relies on dynamic content like posting food items and redeeming deals, a mobile app ensures efficient interaction with minimal load times and fewer navigation issues. Furthermore, mobile apps can offer offline access to certain features, making them especially useful in areas with poor internet connectivity, whereas websites require a constant connection.

Finally, in today's fast-paced world, most users rely on their smartphones for daily tasks. A mobile app seamlessly integrates into users' routines.

PLANNING AND RESEARCH

Literature Review

The literature review examines relevant research and technologies that informed the design and development of the EcoEats platform.

Sustainability and Food Waste Reduction

Food waste is a significant environmental issue, with studies showing that up to 20% of food produced is wasted locally (SG101, 2022) in the last decade. Community-based platforms like TreatSure have emerged as a promising solution to reduce food waste by encouraging food sharing at a local level (SG101, 2022). Research on digital food-sharing platforms, such as Treatsure and JustDabao, highlights the positive impact of facilitating food redistribution among communities. This informed EcoEats' core objective of providing a mobile platform for surplus food sharing to promote sustainability.

Mobile Applications for Community Engagement

Mobile applications offer a more responsive and accessible solution compared to websites (Michael, 2024). Research demonstrates that mobile apps increase user engagement through features like push notifications and real-time interactions (MoldStud, 2024). In order to capitalise on this, EcoEats was designed as a mobile app.

User Management and Security

User security and privacy regarding personal data are critical concerns in mobile applications (Rehman & Khazin, 2021). Literature on authentication methods emphasises the importance of secure login systems and password hashing to protect user information (Iqbal, 2023). EcoEats incorporated these practices in user security to ensure the safety of its users.

Market Research

We implemented two design concepts inspired from our market research. Specifically, we incorporated “Eatigo’s Streamlined Navigation” by dividing the application's core functions into distinct, easily accessible pages. This approach simplifies user interaction by reducing clutter and allowing users to navigate intuitively through the app.

Additionally, we drew inspiration from “Just Dabao’s UI Layout References”, adapting their user-friendly interface design to fit our application’s needs and maintain a clean and organised layout.

However, we did encounter limitations due to time constraints. This led to the postponement of some features from our initial proposal, including 'Two-Factor Authentication', 'Themed Deals', and an 'Advanced Tags System.' Although these functionalities were not implemented in the current version, the application was developed with robust and scalability in mind. This ensures that, should we decide to push the application to market, these advanced features can be integrated seamlessly.

In fact, we have already laid the groundwork for these features. For instance, the foundation for the "Advanced Tags System", as illustrated in Figure 2.1. Each "Share" post includes a 'tags' column where the original poster can input keywords related to their post. These keywords are then concatenated into a single string, with each keyword separated by a comma. For example, a post might have the tag string "toy,cat,furry,cotton."

This design allows the tag string to be easily split into individual words using the `.split(',')` function. By doing so, we can efficiently filter posts based on specific tags. For example, a search for the word "toy" would return all posts whose tag arrays include that keyword. This groundwork ensures that it can be fully integrated in future iterations with minimal additional development effort.

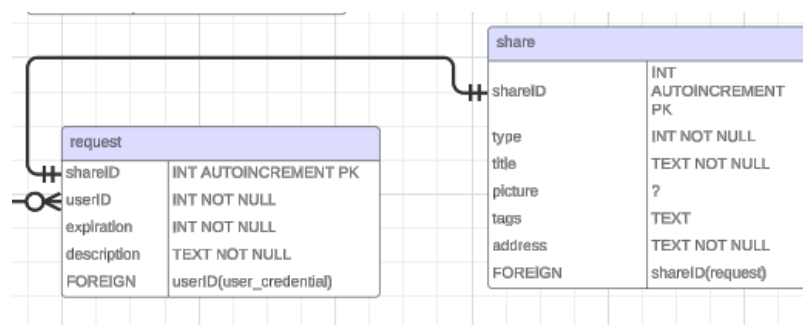


Figure 2.1

Project Management Planning

Timeline Remarks - Gantt Chart

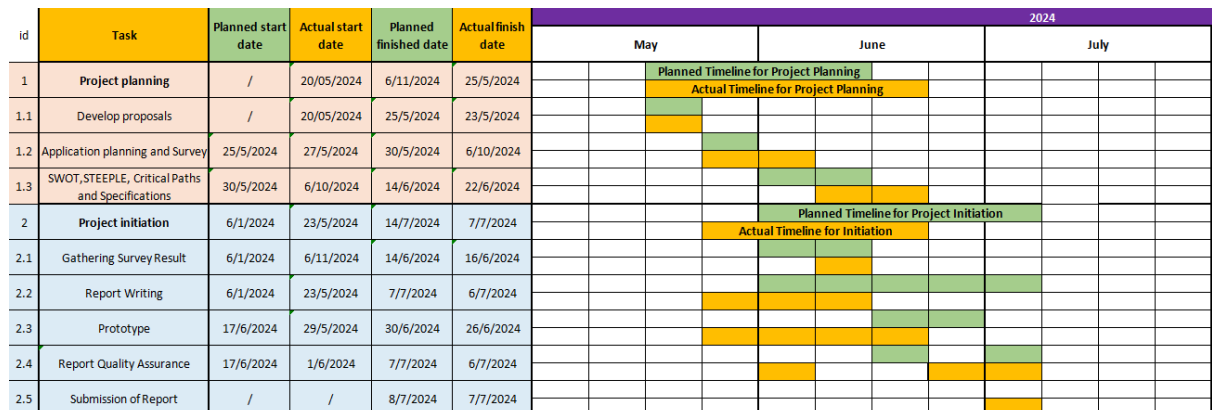


Figure 2.2: Gantt Chart for Mid-semester report

After our previous report submission, we had a discussion together and realised that developing a React Native mobile application might be better than a website for our case. The benefits we found are:

Enhanced User Experience:

React Native Mobile apps would be smoother and provide a more responsive user interface than the many libraries we can use to aid with our design.

Access to Native Device Features:

We could access and integrate with native device functionalities like push notifications, cameras, and GPS.

Performance Optimization:

React Native apps can be developed to run on both iOS and Android platforms. Thus, saves development time if we want to push our application to the App Store.

Better User Engagement:

Mobile app's push notifications encourage higher user retention, which can be used to remind users of important ongoing deals and may motivate them to start using the application again.

To ensure a smooth transition to develop a React Native mobile application, we took the following steps to ensure we are adequately prepared:

Extended Development Cycle for Learning:

The development timeline was extended by one additional week (starting from the 14th of July). This period is dedicated to familiarise ourselves with React Native's architecture and its integration with GitHub.

Requirement Analysis and Library Selection:

Broke down the project requirements into specific design elements and backend functionalities. For each requirement, a relevant library or tool is selected for the section to run. This ensures that the necessary resources are readily available when we are coding the application.

Creation of a Foundational Base App:

Having practical experience with React Native is still the most important thing. Therefore, we tasked ourselves with a hands-on introduction by developing a simple app that serves as a template for the project.

id	Task	Planned start date	Actual start date	Planned finished date	Actual finish date	2024											
						July				August				September			
3	Application Coding Initiation	14/7/2024	22/7/2024	8/9/2024	1/9/2009					Planned Timeline for Application Coding Initiation							
3.1	Page Section Assignment	14/7/2024	22/7/2024	18/7/2024	24/7/2024												
3.2	Code Writing	21/7/2024	22/7/2024	25/8/2024	27/8/2024					Code Writing							
3.3	Code Review	30/7/2024	1/8/2024	25/8/2024	30/8/2024					Code Review							
3.4	Application Quality Assurance	30/7/2024	17/8/2024	25/8/2024	30/8/2024												
4	Project monitoring and control	14/8/2024	9/8/2024	1/9/2024	30/8/2024						Planned Monitoring						
5	project submission	/	/	8/9/2024	1/9/2009											Submit	

Figure 2.3: Updated Gantt chart

Work Breakdown Structure

Main Task	Start Date	Finish Date	Details	Remarks	Completed by
EcoEats design finalisation	10/7/2024	13/7/2024	Confirmation of EcoEat's design. The final design is similar to the one found in the proposal report.	The designs of popups or request screens are not finished. Only the main pages are done.	All members
Familiarising with React Native and Setting up its development environment.	14/7/2024	7/8/2024	The team to look through relevant React-native documents to set up a mobile development environment correctly and to code an application with React Native.	We had trouble setting up the environment. The differences in devices and programs' versions resulted in Android Studio compiling the ".gradle" differently. Hence, lead to one copy of an application can't be run on other computers.	All members
Creates the SQLite 'ecoeats.db' for the application.	24/7/2024	24/7/2024	To create a database for the application. Since the EcoEats app is a demo, a SQLite database is sufficient		Cuong
Research the necessary libraries to function	21/7/2024	24/7/2024	To find the libraries that will allow the app to perform as desired. Such as the image-picker library.		All members
Develop the foundation application template for EcoEats.	24/7/2024	1/8/2024	The template base app should include the implementation of the page navigation system, bottom navigation bar, and data retrieval from the first iteration of SQLite implementation.	The SQLite DB is not fully copied into the emulated device's storage. Thus, we are unable to insert a new row.	All members

Develop the “Sharing” page	2/8/2024	22/8/2024	To build the back-end for the core functionalities and style the front-end of the “Sharing” page.		Cuong
Fix the “Posts in the Share page are not displayed in two columns” issue	4/8/2024	5/8/2024	Often to display contents in two or more columns, it is recommended to use FlatList. However, I do not want to use that as it will affect the flexibility of the application.	This was solved by setting a limit in width of each post and allowing the content to be wrapped. This will make displaying the content to be more flexible as using a device with longer display width will allow more posts to be displayed accordingly.	Cuong
Research on “Modal” and “Picker” to create popups and dropdown boxes respectively	10/8/2024	10/8/2024	Popups and dropdown boxes are crucial for the user experience in using the application.		Cuong
Test the popups and dropdown boxes of the “Request” page	10/8/2024	10/8/2024	Popups and dropdown boxes are to be used so the user can request a meet-up with the ‘giver’ at a desired date and time.		Cuong
Develop the “Request” page	10/8/2024	12/8/2024	To build the back-end for the core functionalities and style the front-end of the “Request” page.		Cuong
Develop the “Deals” page	11/8/2024	28/8/2024	To build the back-end for the core functionalities and style the front-end of the “Deals” page.		Nicole
Develop the “Explore” page	12/8/2024	27/7/2024	To build the back-end for the core functionalities and style the front-end of the “Explore” page.		Nicole, Winnie

Develop the "User profile" page	13/8/2024	26/8/2024	To build the back-end for the core functionalities and style the front-end of the "Request" page.	Xiang En styled most of the frontend of the "Login" page	Cuong, Xiang En
Test the Imgur API	15/8/2024	15/8/2024	To simulate an online image storage through Imgur.	By modifying some lines, EcoEats can be made to utilise known image hosting services for business.	Cuong
Fix the "Users with no posts will produce error when loading their profile" and the "Users with no post won't have their post shown on their profile"	17/8/2024	17/8/2024	"Users with no post will produce error when loading their profile". A user with no post will result in having a null being used as an input to the query to retrieve all of their posts. Resulting in errors in data retrieval. "Users with no post won't have their post shown on their profile". A user with no post made would have a null value in their 'share_Posts' column. You can't concat a null value to a string.	"Users with no post will produce error when loading their profile" was solved by adding a conditional to return an empty array if the user has made zero post (which is null in the datatable). "Users with no post won't have their post shown on their profile" was solved by adding a conditional that if the user has made zero posts, it would replace the null with the id of the post rather than concatenating the id.	Cuong
Redesigns the current DB system.	17/8/2024	18/8/2024	To allow the insertion of a new row into a data table	The new database, instead of using a copy created from the project android's assets folder, is copied and stored locally in the emulated device's storage. This will allow insertion of new data rows.	Cuong

Modify all of the currently existing pages to utilise the new image-storing system.	18/8/2024	18/8/2024	To update the completed pages that's display images to use "source={{uri: link}}" instead		Cuong
Develop the back-ends of the authentication system and apply it to the rest of the pages	23/8/2024	23/8/2024	To set up a system that checks if the user is logged in. This will aid in performing actions such as redirecting the user to the "Login" page when they attempt to access their profile and displaying actions only available to logged-in users.	This was implemented, for demonstration purposes, using a global variable that is initially null. After logging in, the global variable will now contain the logged user's ID. Thus, helps with displaying the appropriate contents for certain pages.	Cuong
Style the front-end of the "Login" page.	23/8/2024	23/8/2024	Designed the 'Login' page, created the interface with username and password fields, and tested functionality using a default login to ensure everything worked correctly. Additionally, implemented and tested a logout button, allowing users to securely log out and return to the login page.	This was to test run and ensure the Login page had a properly functioning basic setup for logging in and logging out	Xiang En
Develop the back-end of the "Login" page	23/8/2024	24/8/2024	To code the actions and appropriate data retrieval for credential checking and appropriate page redirection	Nicole implemented password hashing	Cuong, Nicole
Develop the "Register" page	24/8/2024	24/8/2024	To build the back-end for the core functionalities and style the front-end of the "Register" page.	Nicole implemented password hashing	Cuong, Nicole

Stylise the whole application	25/8/2024	1/9/2024	To have a consistent theme throughout the application and ensure the UI is not confusing for new users.	This was done to match with the purpose of our application. Thus, the inclusion of colour relating to the idea of “nature”.	Cuong, Nicole, Winnie
Fix the “Not displaying appropriate profile” bug	25/8/2024	25/8/2024	If a user is to view another person’s user page, pressing the “User” tab at the bottom navigation bar will not redirect the user to their own user page. Rather, it will stay as the previous profile visited.	The bottom bar navigation’s ‘initialParams{}’ is only used the first time the screen is mounted, navigating to the screen later doesn’t reset or update the parameter. This was solved by using the ‘tabBarButton{}’ to overwrite its default functionality to ensure pressing the ‘User’ tab would always redirect the user to the logged user’s profile.	Cuong
Merging of GitHub project’s branches and finalising the application for submission	27/8/2024	28/8/2024	To merge all branches and ensure the application demo works as intended without issues.	Essentially, standardise all functionalities across the pages.	Cuong, Nicole

Approach Taken to Development

In addition to basic communication channels like Telegram, which we used for quick and easy communication, and Discord, which served as our central repository for project-related documents and the platform for hosting our weekly meetings, we also utilised other tools to streamline our development process. Other platforms we used are GitHub and Trello, to ensure our applications are being built properly and we are on track to finish the application before the deadline respectively.

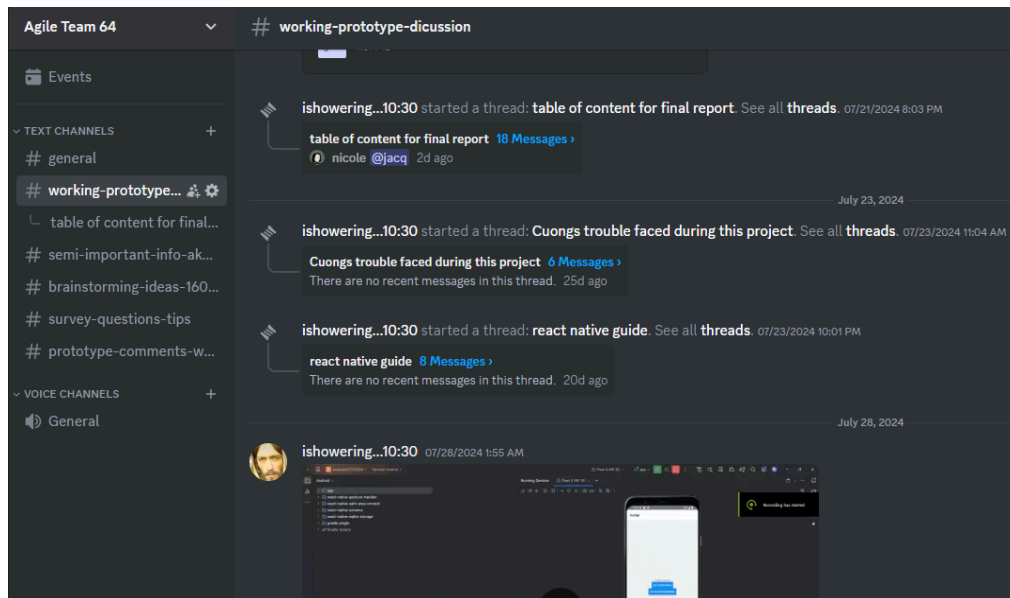


Figure 2.4: Our Discord Server

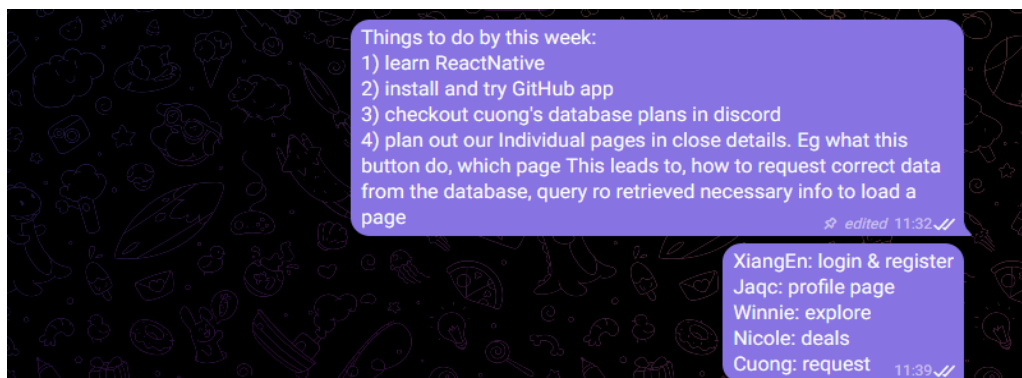


Figure 2.5: A snippet of our communication in Telegram team chat

Research Done - Trello (Kanban Board)

Trello is used for task management, offering a visual interface that helps the team organise and prioritise tasks. It allows for easy tracking of what needs to be completed and what is in progress. Each weekend of our development period, the available members would meet up to discuss our progress and update these tasks accordingly.

The Trello interface serves as a Kanban Board to display our tasks visually, showing their status and progression. A Kanban Board is most effective where priorities frequently shift, or where continuous delivery is crucial. Considering our team's dynamic and how some members have their own life's priorities, using a Kanban Board ensures the EcoEats mobile application is being continuously developed.

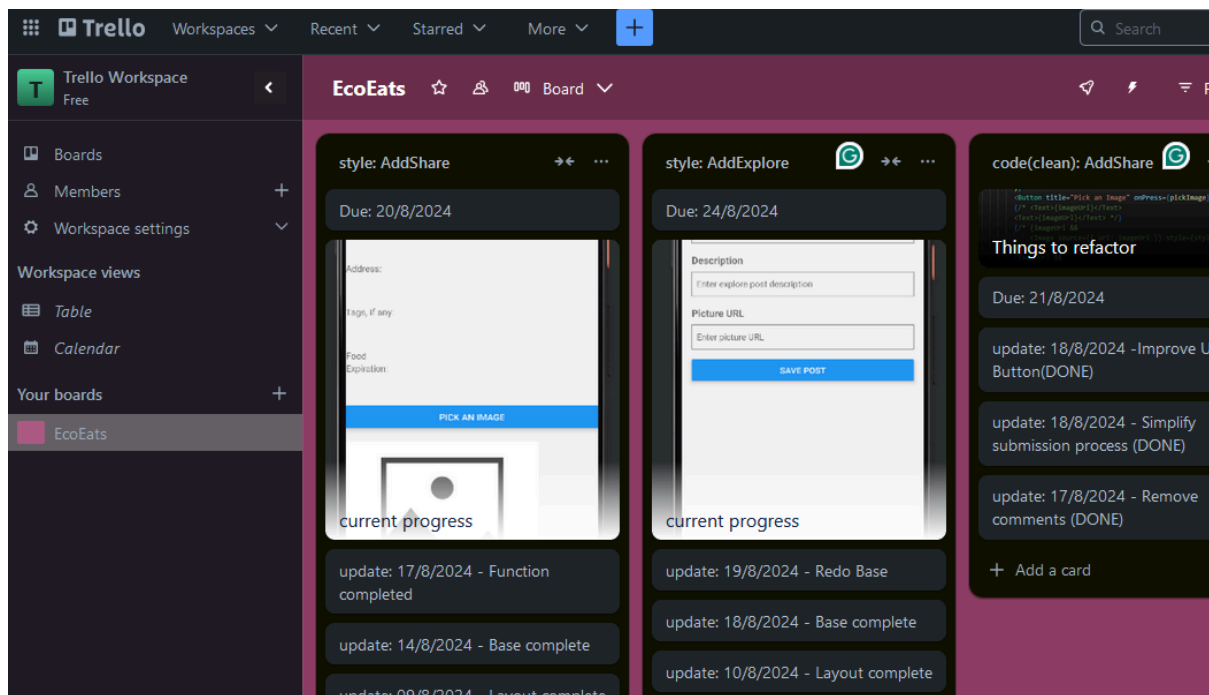


Figure 2.6: A snippet of our task cycle on Trello as of 19/8/2024

Furthermore, Trello is particularly beneficial when team members do not have access to a PC, as it can be accessed from mobile devices due to its mobile friendly interface. Overall, with this project management structure, we were able to ensure our application works and meets our goals effectively.

Team Composition

Effective team composition plays a pivotal role in the success of agile software development projects. Our team was structured with a balanced mix of skill sets, ensuring that the project progressed smoothly across all technical and user-facing aspects.

Our team followed the principles of cross-functionality, where members had overlapping skill sets but specialised in different areas. For example, team members specialising in frontend development worked closely with those focused on the backend to integrate complex features such as the Explore and Deals pages, while ensuring data consistency across the SQLite database.

The collaborative nature of our agile process allowed us to identify and resolve issues. Regular team meetings and communication channels facilitated transparency, ensuring that each member was aligned with the overall project goals.

The dynamic between team members also allowed for iterative feedback, particularly in refining features like the Explore and Deals pages. As new functionalities were developed, they were tested and evaluated by team members with different areas of expertise, resulting in a well-rounded application.

Work Allocation

Effective work allocation was essential in ensuring that our project progressed smoothly, particularly given the varied expertise and experience levels within the team. We adopted a task-based approach to work allocation, aligning tasks with individual strengths while maintaining flexibility to adapt to new challenges.

Work allocation followed the principles of the agile methodology, focusing on breaking down the project into manageable sprints with clear deliverables. Tasks were assigned based on expertise in either frontend or backend development.

For instance, the implementation of the Deals and Explore pages was assigned to team members with a strong background in frontend development. These members were responsible for creating a visually cohesive user interface, ensuring that the pages were user-friendly and aligned with the overall design theme of the application.

Backend tasks, particularly related to the SQLite database, were assigned to members with expertise in database management and query optimization. This included setting up the database structure, managing data for the Explore and Deals pages, and ensuring that the redeemed deals and user profile updates were accurately reflected in real-time.

Additionally, responsibilities for user authentication, password hashing, and security features were allocated to team members with experience in data security. They ensured that user data was stored securely, focusing on the development of authentication protocols and secure password storage mechanisms.

Team members also took on cross-functional roles, contributing to tasks outside their primary expertise. For example, those working on frontend development assisted in testing and troubleshooting backend functions. Likewise, backend developers provided input into frontend tasks, ensuring that the design and functionality worked seamlessly with the underlying data structure.

Regular meetings and sprint reviews ensured that work allocation remained flexible and adaptable. As new challenges emerged, tasks were reallocated to ensure that high-priority features were addressed. For example, when challenges arose with image upload functionality, team members were reassigned to focus on resolving issues with image handling across different devices, while others worked on ensuring smooth integration with the backend.

PROTOTYPING AND ITERATION

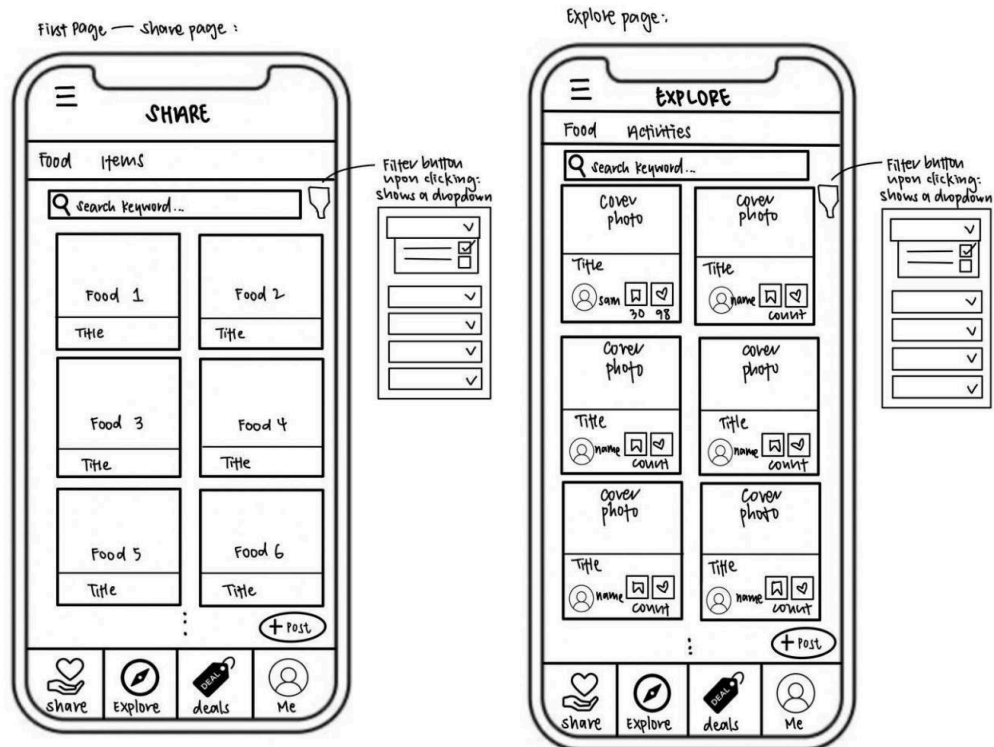


Figure 3.1 and 3.2: proposal prototype “Share” page and “Explore” page

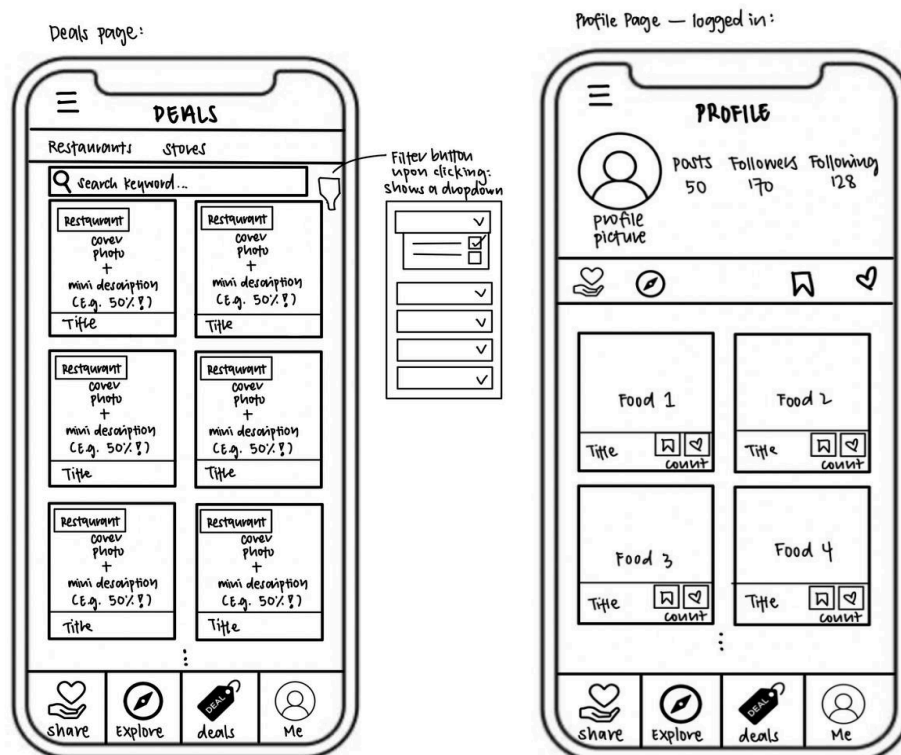


Figure 3.3 and 3.4: proposal prototype “Deals” page and “User” page

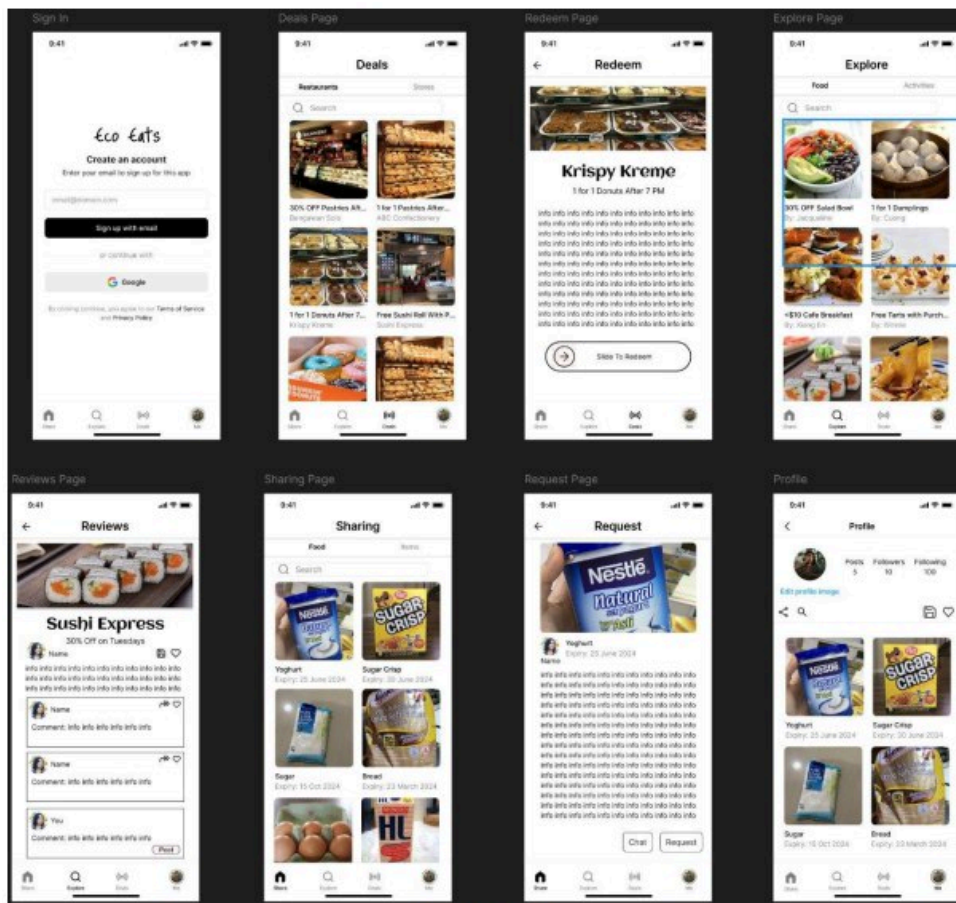


Figure 3.5: Figma of our prototype for our proposal

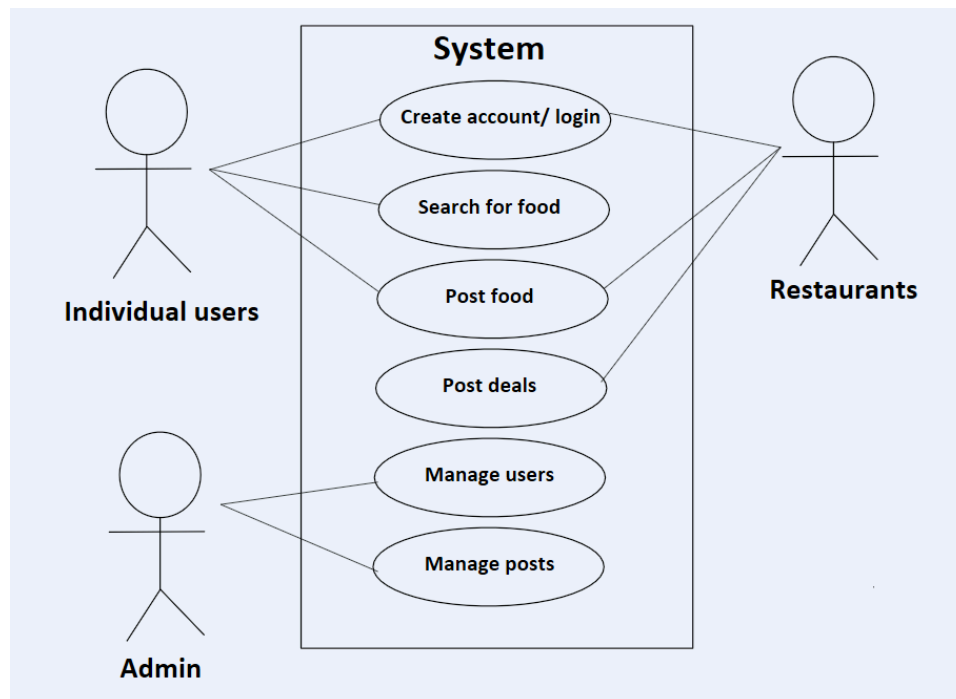


Figure 3.6: EcoEats' proposal UML use case diagram

Iteration of Prototypes

Mid-Fidelity Prototype

First Iteration

The first iteration of our prototype involved the creation of a simple "To Do List" mobile application(Figure 3.7). This version incorporated the SQLite database library and basic navigation across different pages and lays a solid foundation for the more complex features required in our final application. This initial prototype allowed us to establish the technical groundwork necessary for building the application.

Second Iteration

The development is focused on the "Share" page, a critical component of our application. This stage involved implementing a bottom navigation bar for easy access to different sections of the app, a search bar capable of filtering posts by name, and a toggle button that allowed users to display either food posts or object posts. Additionally, we set up a new SQLite database to support these functionalities, with the database schema and relationships(Figure x.x).

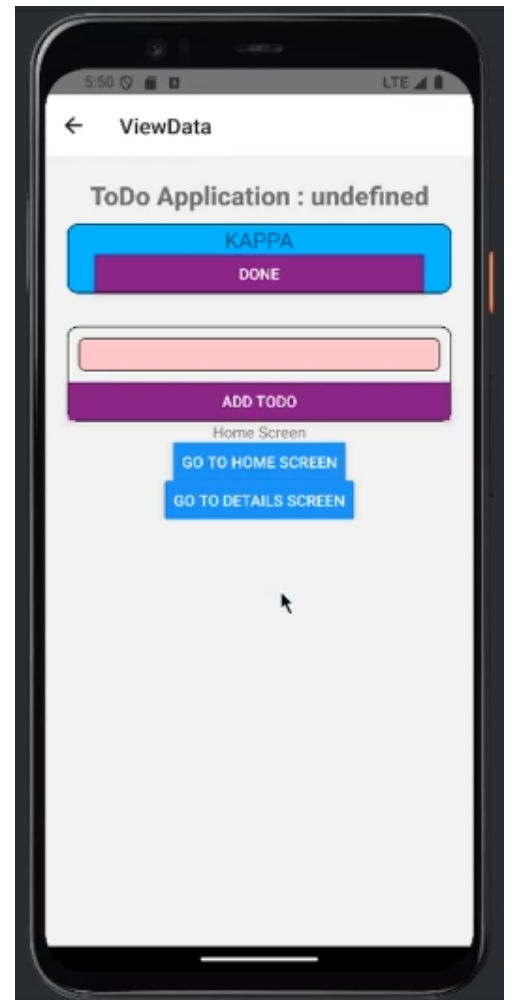


Figure 3.7: To-Do list app

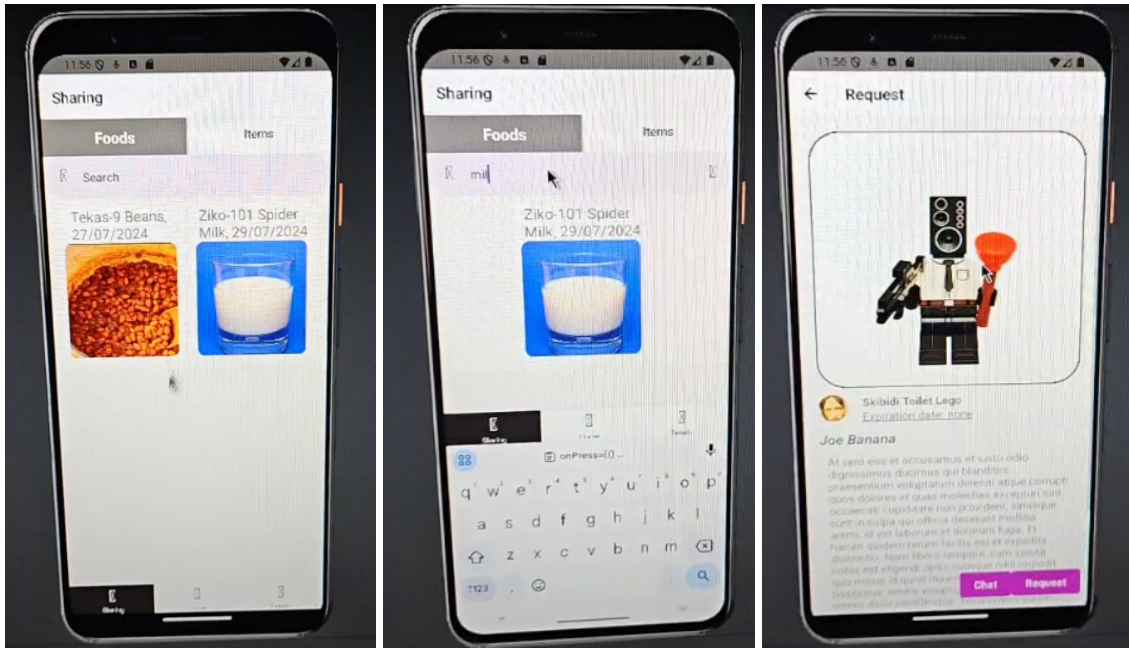


Figure 3.8, 3.9, and 3.10 : Early prototype on the “Share” page

Despite the minimality, this provided us with insights into the application's future functionalities and our work scope. Working through this iteration revealed that certain features we initially planned are overly ambitious given our time and resource limitations.

Overall, this has been crucial in refining our vision for the application and allowed us to prioritise on delivering the core functionalities.

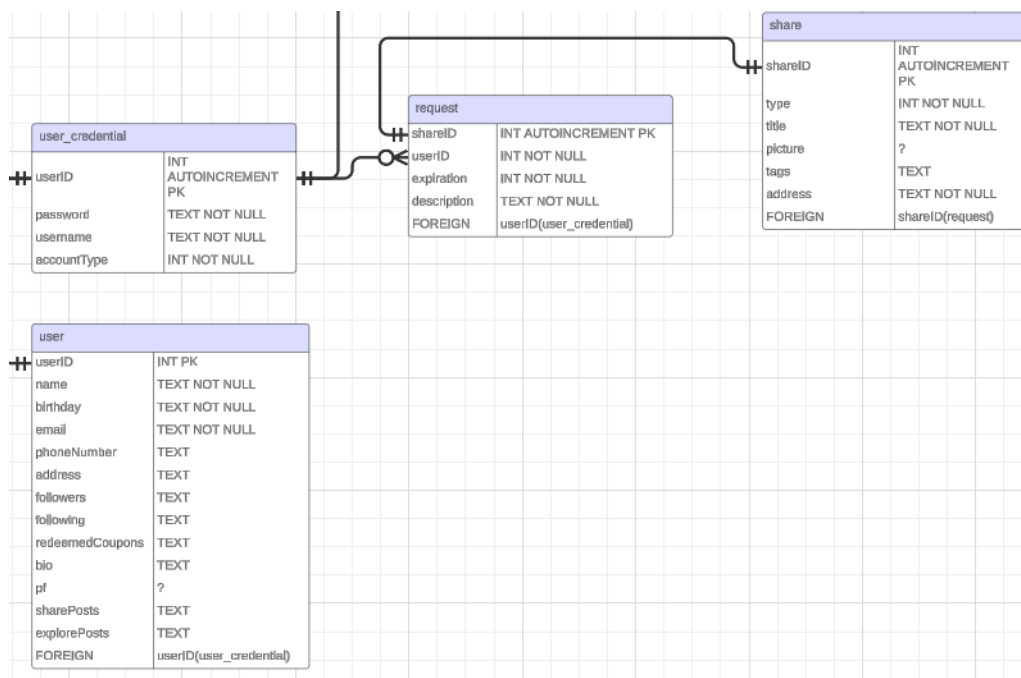


Figure 3.11 : Data tables relating to the “Share” page

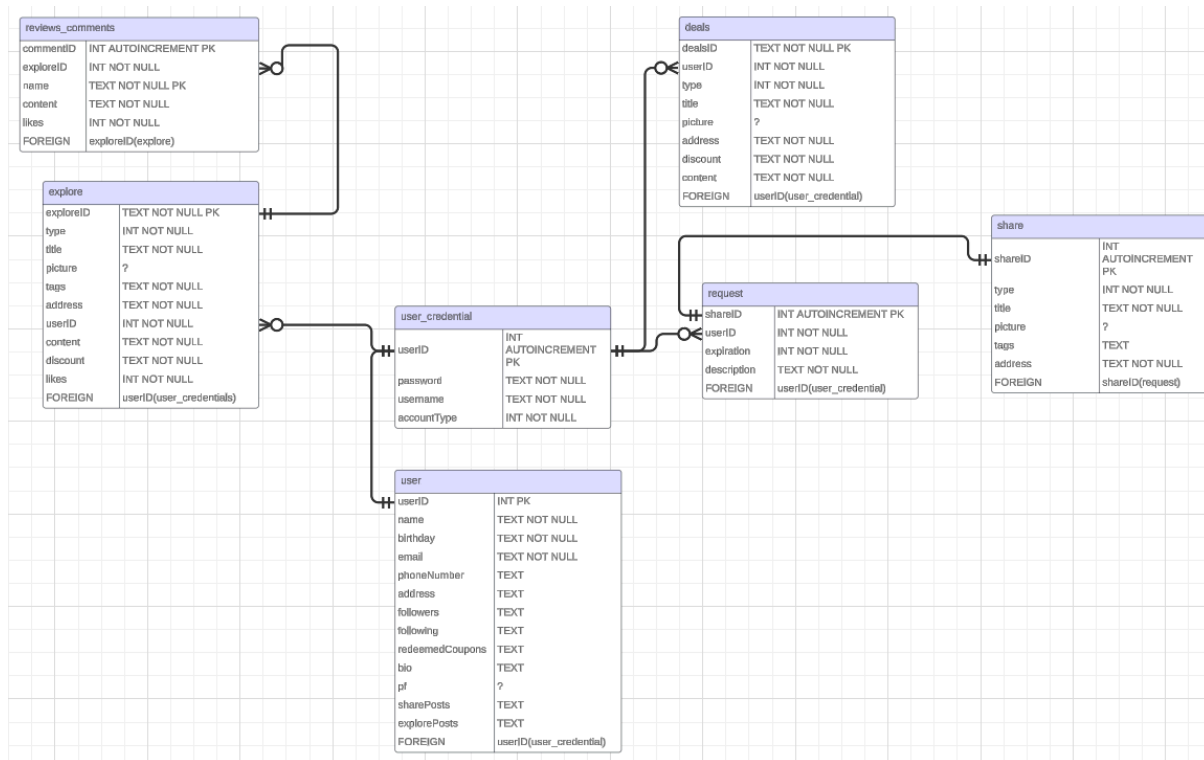


Figure 3.12 : The entire database relation

High-Fidelity Prototype

With improved vision on the project, our primary objective during this iteration is to ensure that all essential features (Explore Page, Deals Page, and User Page) are fully operational.

Focusing on functionality first allows us to detect and fix any issues with the user experience or technical implementation early in the process, and ensure that all components are working smoothly before we turn our attention to refining the interface or enhancing the page's interactions.

First Iteration

The core functionalities of the "Share" page was used as the base functions for the other pages. Having already established and tested essential features in the previous iteration let us efficiently implement the essential features efficiently. This foundation also ensured consistency across the entire application.

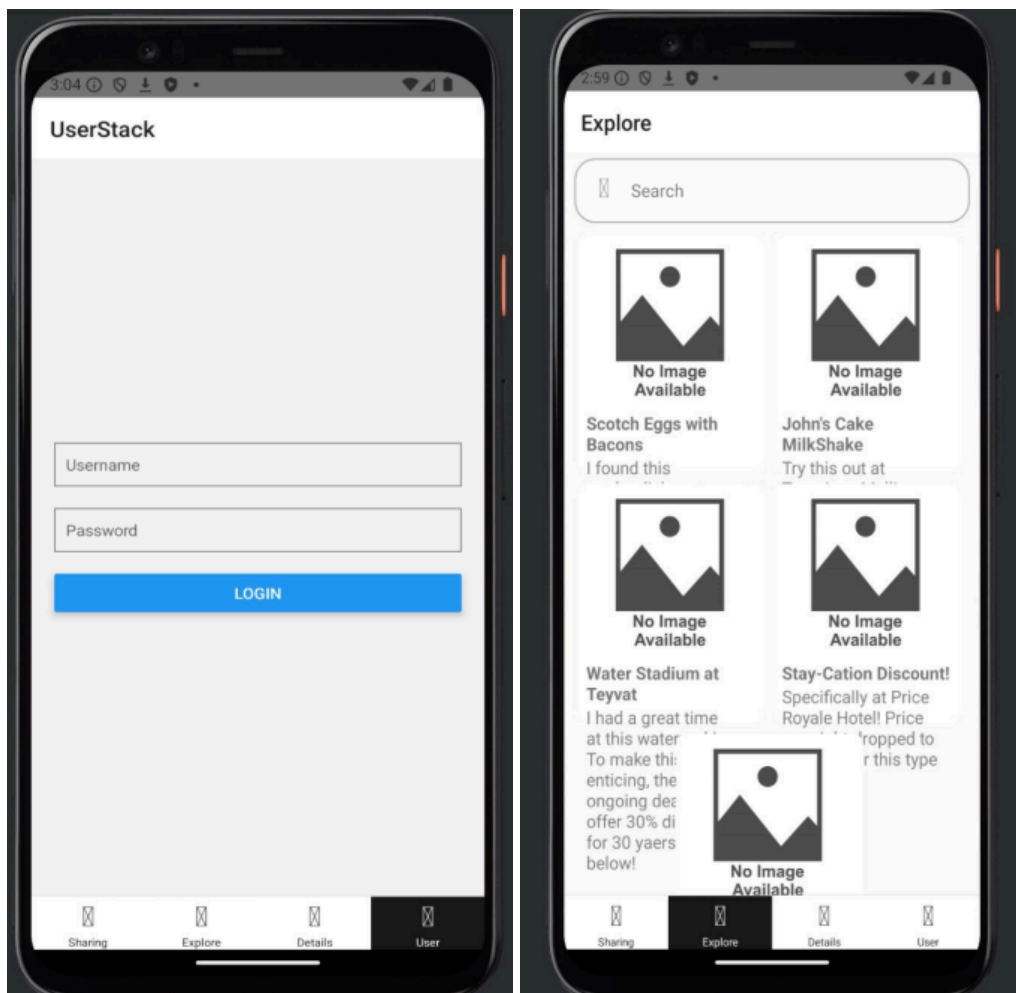


Figure 3.13 and 3.14 : current iteration's "Login" page and "Explore" page

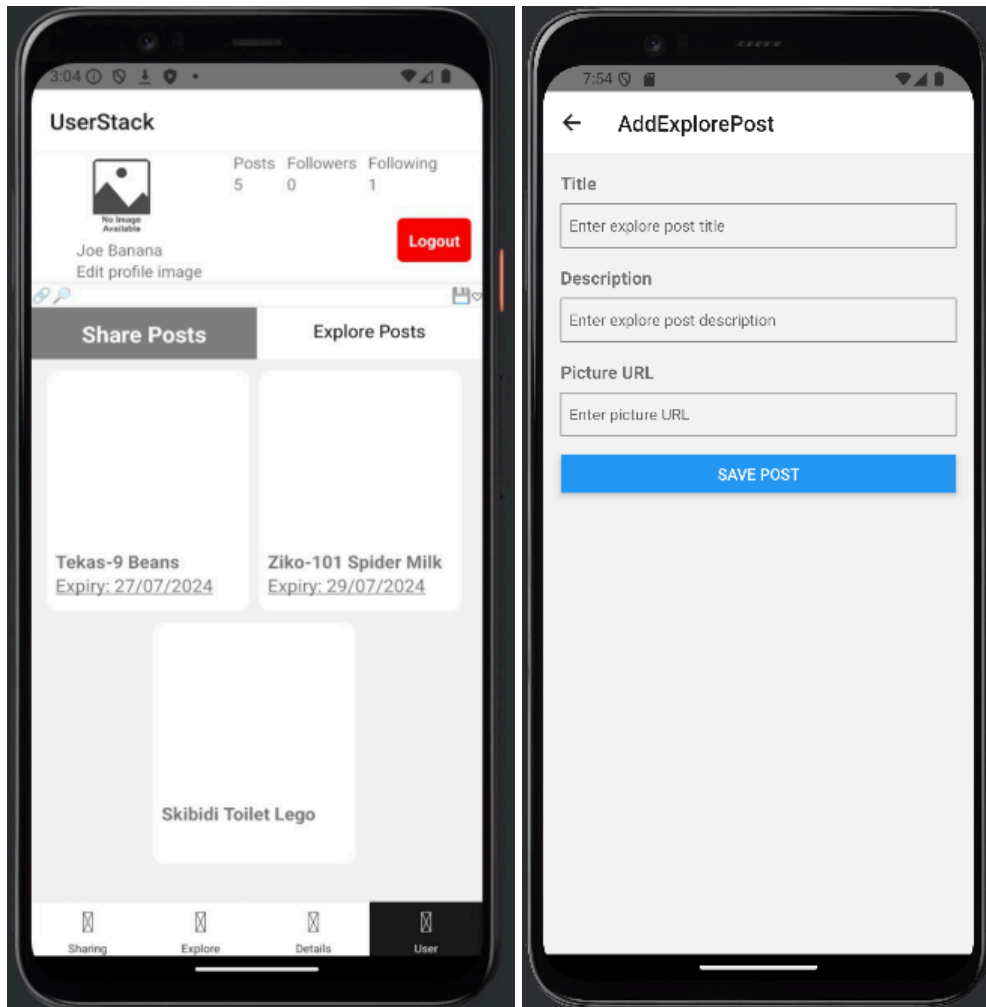


Figure 3.15 and 3.16 : current iteration's "User" page and "AddExplore" page

In this iteration, to enhance user navigation, we implemented a feature that redirects users who are not logged in to the "Login" page when they tap the "User" button on the bottom navigation bar. For logged-in users, tapping the "User" button always redirects them to their own profile page. This allows users to easily access their profile at any time.

Initially, we encountered implementation challenges due to the `initialParams={{ userID: userId }}` in the `bottomTabNavigator`. This parameter only applies when the user navigates to their profile page for the first time. Consequently, after a logged-in user views another user's profile, which will change the `'userId'` variable, pressing the "User" tab on the bottom navigation bar would not redirect them back to their own profile. Rather, it will redirect the previously viewed profile. We resolved this issue by overriding the default behaviour of the "User" button, modifying its options (Figure 3.17).

```

<Tab.Screen
  name="User"
  component={UserScreen}
  initialParams={{ userID: userId }}
  options={{
    tabBarButton: (props) =>
      <TouchableOpacity
        {...props}
        onPress={() => {
          // Navigate to the User screen with the loggedUser parameter
          navigation.navigate('MainTabs', { screen: 'User', params: { userID: userId } });
          // // If you want to include the default behavior after your custom action:
          // props.onPress(); // This will still trigger the default navigation behavior
        }}
      />,
    tabBarIcon: ({ color, size }) => (
      <Text style={{ fontSize: size, color: color }}>👤</Text>
    ),
  }}
/>
</Tab.Navigator>

```

Figure 3.17: “User” tab of the Bottom Navigation Bar

```

const UserScreen: React.FC<Props> = ({ route, navigation }) => {
  const { userID } = route.params;
  const { userId } = useContext(UserContext);
  const {setUserId} = useContext(UserContext);

```

Figure 3.18: Global variable “userId”

Second Iteration

We have integrated additional features to enhance the application's usability, Such as the ability for users to select and upload images from their own devices. For example, when creating a new post on the "Share" page, users may select the button to "Pick An Image". Users then can access their device storage and select an image to be displayed in their post. The selected image is initially stored as a Uniform Resource Identifier (URI), which references the image's local directory.

To ensure that the image can be viewed by other users across different devices, the application includes functionality to upload the selected image to an image hosting service (which is Imgur, for our demonstration). This upload process allows the image to be stored remotely, making it accessible to all users regardless of the device they are using. Additionally, compared to the previous iteration where the user has to type or paste in the image's link, this change allows users to select their desired image easier.

After implementing the image selection and upload functionality on the "Share" page, we applied the same code to other pages with similar requirements. Once again, this allowed consistency and hastened the development process.

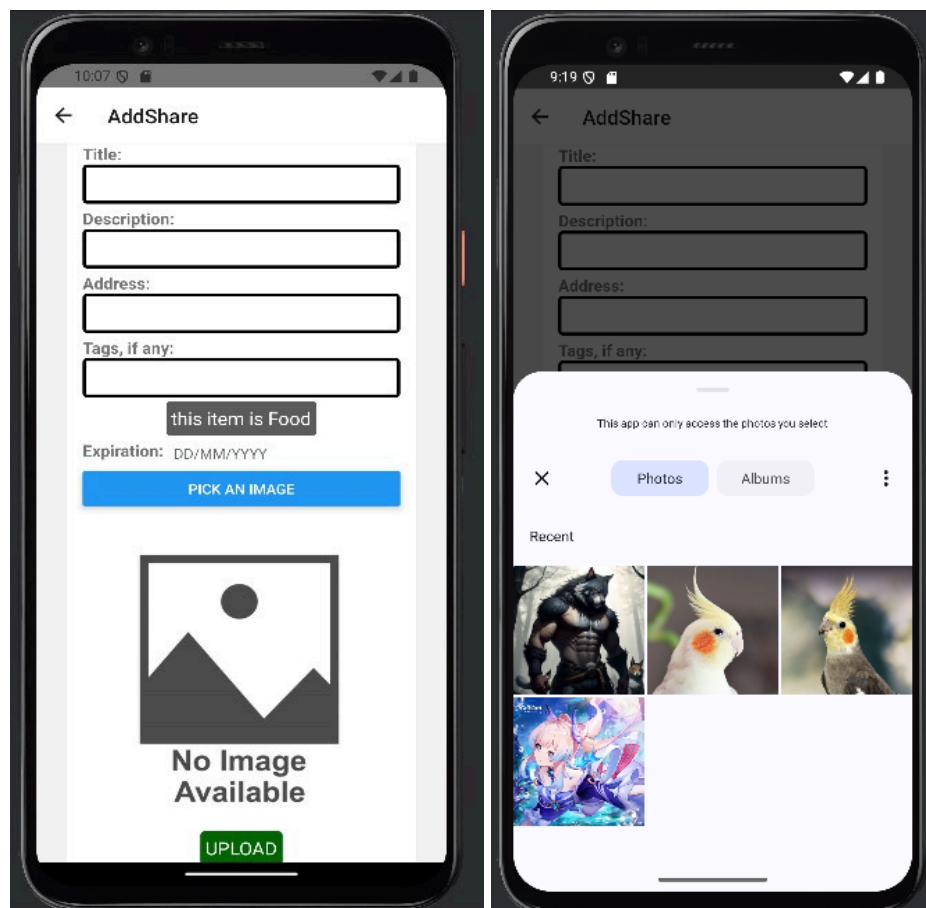


Figure 3.19 and 3.20: current iteration's pages that demonstrate image selection.

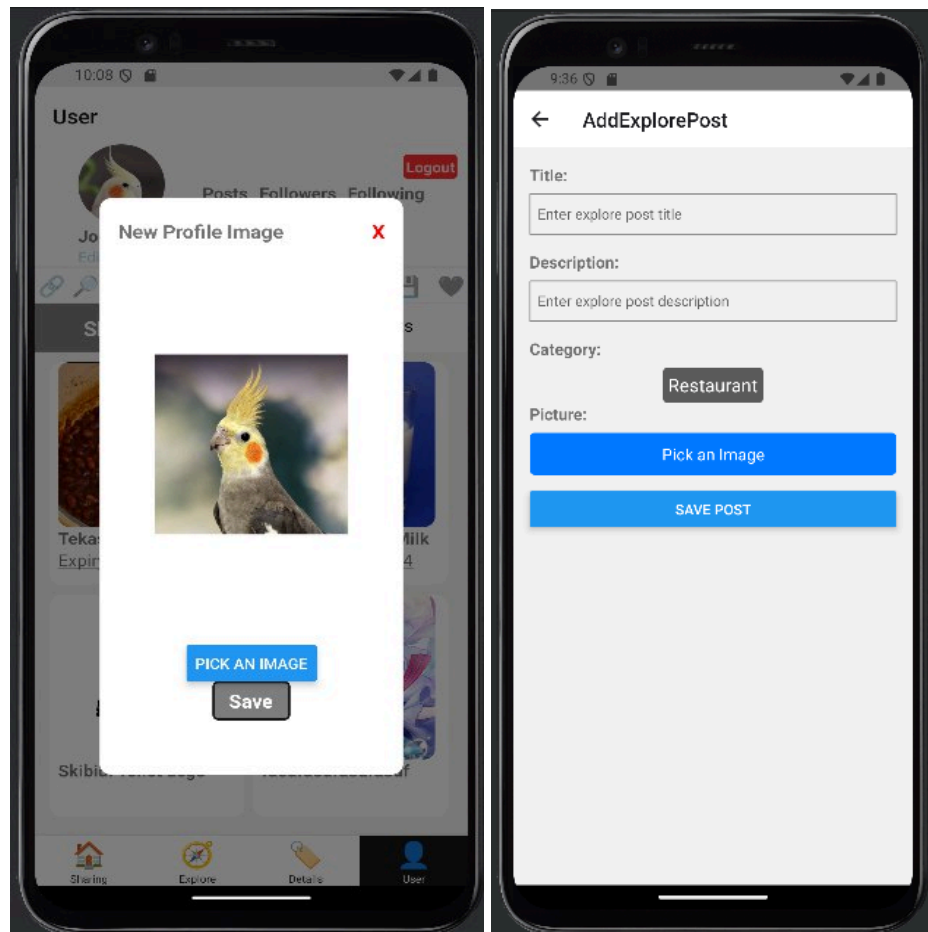


Figure 3.21 and 3.22: current iteration's pages that demonstrate image selection.

As we move into the third iteration, most of the core functionalities have been successfully implemented. Our focus will shift to concentrate on refining the app's styling and establishing a consistent theme throughout the interface to enhance the user experience. Our theme will incorporate green elements to align with and emphasise our goal of promoting eco-friendly practices. Furthermore, we will optimise the interaction between pages to allow users who are not logged in to freely browse certain services. This enhancement aims to encourage more engagement with the application's features and adhering to one of our deliverables from the proposal.

Third Iteration

Our survey results indicated that the app design consistency and clarity is not sufficient. In response, we decided to adopt a more minimalistic theme with a limited colour palette of mainly moss green, light blue, and white to make the interface easier on the eyes. The colours are now used exclusively to highlight key information such as which parts of the application are interactive. This approach aims to create a more cohesive and user-friendly design.

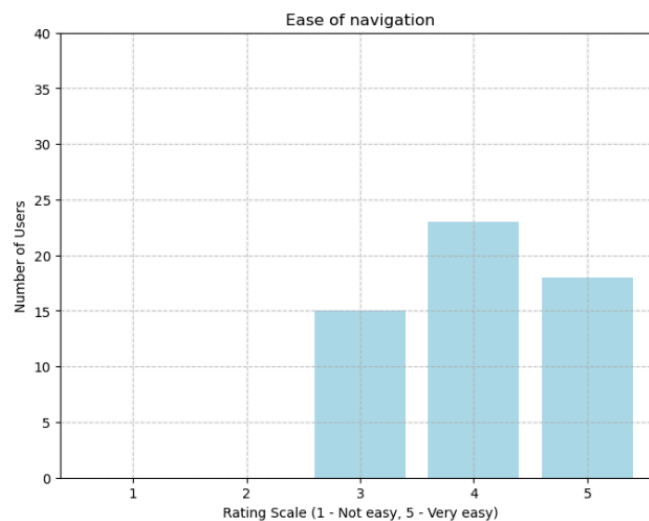


Figure 3.23: a result from our survey.

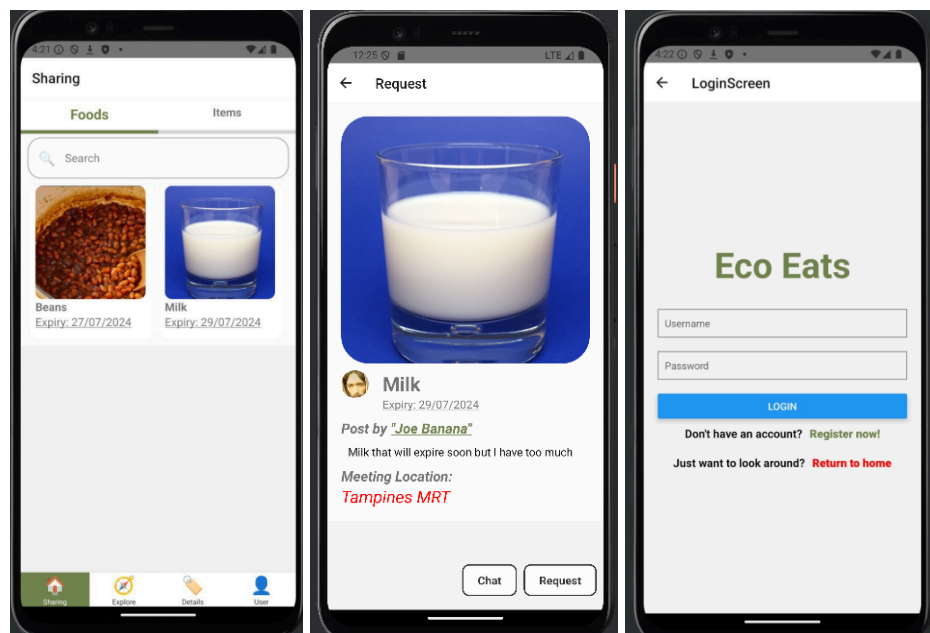


Figure 3.23, 3.24, and 3.25: Demonstrate the page redirection for users who are not logged in

For users who are not logged in, they are able to freely browse the sharing page. However, if they attempt to request a listing by clicking the request button, they will be redirected to the login page, where they can either log in or register for an account (Figure). This was implemented by utilising the 'useContext' hook across many pages to check the global variable 'userId', which stores the identification number of the logged-in account. If this variable is undefined or null, it indicates that the user is not logged in. This allows the application to determine the user's login status and adjust the features accordingly.

```
{userId ? (  
  <TouchableOpacity style={styles.endButton} onPress={() => setModalVisible(true)}>  
    <Text style={styles.endButtonText}>Request</Text>  
  </TouchableOpacity>  
) : (  
  <TouchableOpacity style={styles.endButton} onPress={() => navigation.navigate('LoginScreen')}>  
    <Text style={styles.endButtonText}>Request</Text>  
  </TouchableOpacity>  
)}
```

Figure 3.26: Button's functionality depending on if the user is logged-in

Once users have logged in, they now gain access to various features that were blocked off before such as adding new share listings or commenting on explore listings. This approach ensures that only registered users can interact with the listings.

Here are the additional functionalities that we find worth mentioning:

- For the 'Request' page in Figure 3.24, tapping on the green name text will redirect the user to the poster's profile.
- For the "DealDetails" page, shown in Figure 3.27, we implemented a functional slider with back-end support to modify the relevant data tables. This ensures that users can only claim each deal once. Additionally, only accounts classified as "Business" can add new deals, reinforcing account-specific functionalities.
- For the "User" page, shown in Figure 3.32, logged-in users viewing their own profile will have access to the "Edit Profile Picture" functionality and a "Logout" button. These options are only available when the user is viewing their own profile, ensuring that personal account management features are accessible. Additionally, in the same pop up to change
- For the "Explore" page, shown in Figure 3.35, logged-in users can comment and like posts on the "Explore" page, encouraging interaction and engagement among users.
- User credentials are hashed, as shown in Figure 3.36, to enhance the security of the application. This ensures that sensitive information is protected from unauthorised attackers.

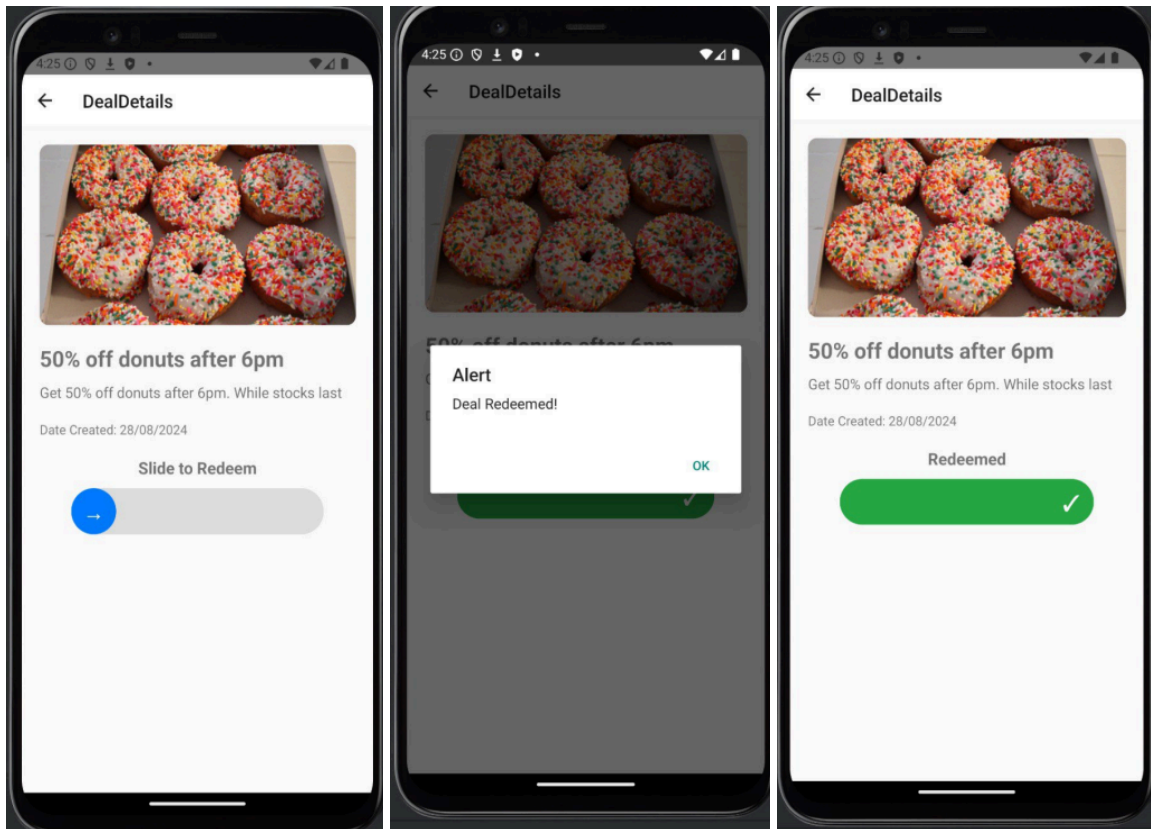


Figure 3.27, 3.28, and 3.29: Demonstrate the state of the slider in the “Redeem” page

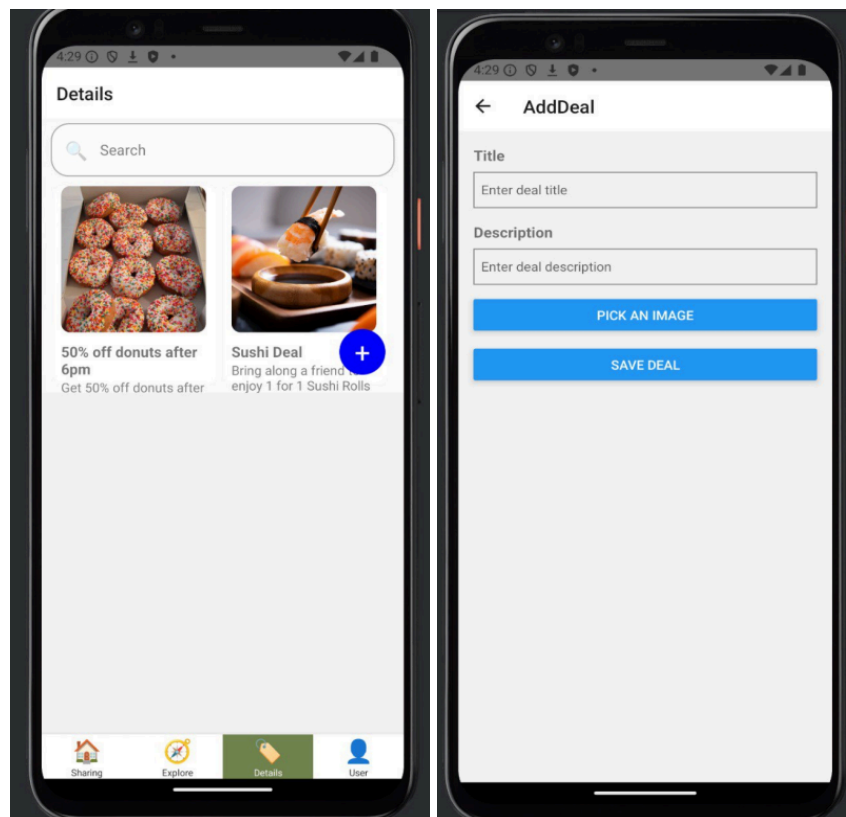


Figure 3.30 and 3.31: The current iteration’s “Deals” page and “AddDeals” page

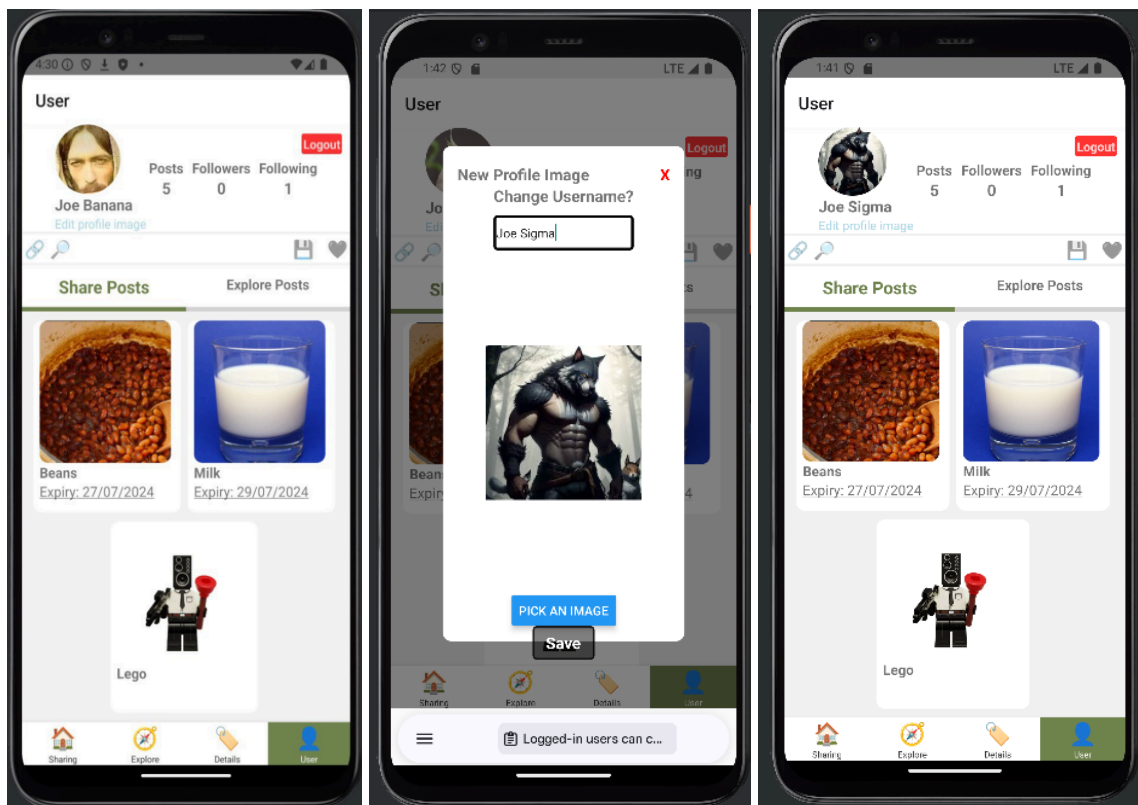


Figure 3.32, 3.33, and 3.34: Demonstrate how a user can change their name and picture

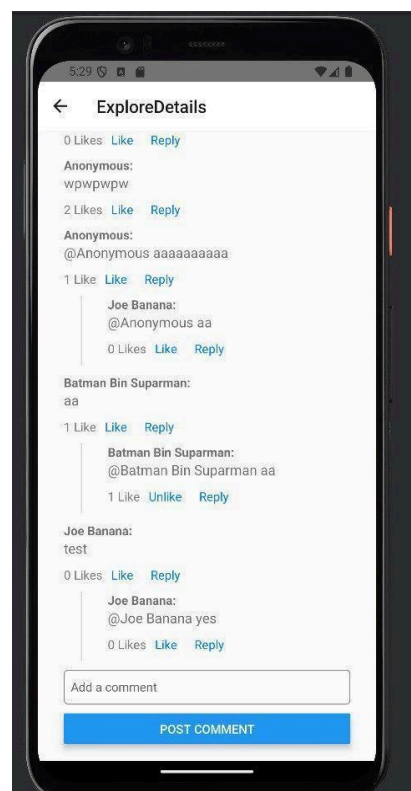


Figure 3.35: The comment section under a post from the “Explore” page

```

export const hashPassword = async (password: string): Promise<string> => {
  // Convert the password string to an ArrayBuffer
  const passwordBuffer = SimpleCrypto.utils.convertUtf8ToArrayBuffer(password);

  // Hash the ArrayBuffer using SHA-256
  const hash = await SimpleCrypto.SHA.sha256(passwordBuffer);

  // Convert the hash (ArrayBuffer) to a hexadecimal string
  return SimpleCrypto.utils.convertArrayBufferToHex(hash);
};

export const checkLoginDetails = async (db: SQLiteDatabase, username: string, loginHashedPassword: string) => {
  try {
    const profileQuery = `SELECT * FROM User_credential WHERE username = ?`;
    const [results] = await db.executeSql(profileQuery, [username]);

    if (results.rows.length > 0) {
      const storedHashedPassword = results.rows.item(0).password;

      // Compare the hashed passwords
      if (storedHashedPassword === loginHashedPassword) {
        // Return the entire user data object
        return results.rows.item(0);
      } else {
        throw new Error('Invalid password');
      }
    } else {
      throw new Error('User not found');
    }
  } catch (error) {
    console.error("Error in checkLoginDetails:", error);
    throw new Error('Failed to get user credentials');
  }
};

```

Figure 3.36: Codes that hash and compare the user's password

Factors To Consider

User testing

The empirical evaluation of our application was conducted in two distinct phases. We conducted phase one after our first iteration, focusing on basic functionalities and layout. We conducted phase two after our second and third iteration, concentrating on specific features and overall user satisfaction.

In Phase 1, feedback was slightly positive with several key areas of improvement, including the need for better design consistency, clearer navigation, better working functions, aesthetics and minor bug fixes. These areas were addressed in Phase 2, leading to significantly higher ratings which indicated that the improvements made between the two prototypes were successful in addressing user concerns and enhancing the overall user experience.

The project progressed from an initial prototype with some design and functionality issues to a near-ready product for beta deployment. The feedback from Phase 2 confirmed that the new features and design enhancements were well-received.

Phase one: Basic functionality and layout

The goal of this evaluation is to gather feedback on the core usability and design aspects through a survey, involving 56 participants who provided feedback on the intuitiveness of the layout, ease of navigation, core functionality, and other key aspects of the application.

Response: Participants mostly found the layout intuitive and the navigation straightforward, though some noted areas for improvement, particularly in core functionality and design consistency across different screens.

The feedback indicates that while users generally felt that the app met their expectations, improvements are needed for the core functionalities and overall layout.

Below is an overview of the questions posed to participants and the responses:

1. Intuitiveness of the Application Layout

Participants were asked to rate how intuitive they found the overall layout of our application.

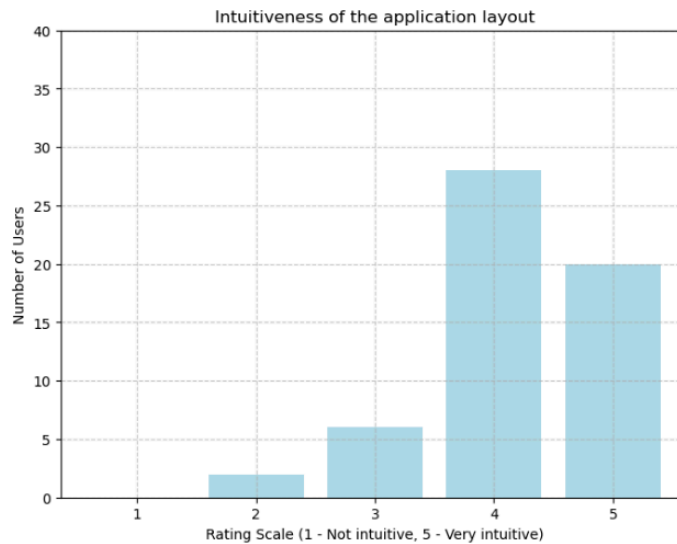


Figure 3.37

This indicates that the majority found the application's layout highly intuitive.

2. Ease of Navigation

This question assessed how easily participants were able to navigate between the login, share and profile pages.

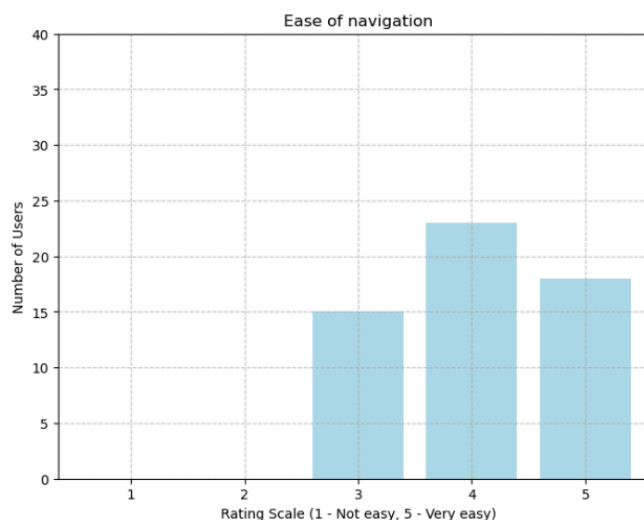


Figure 3.38

We received some feedback that overall it is easy to move between pages but the navigation and design is a little inconsistent. The 'back' button is too small, making it hard to navigate on smaller phones .

3. Satisfaction with Core Functionality

Participants were asked to rate their satisfaction with the core functionalities such as log in, sharing post and profile access function.

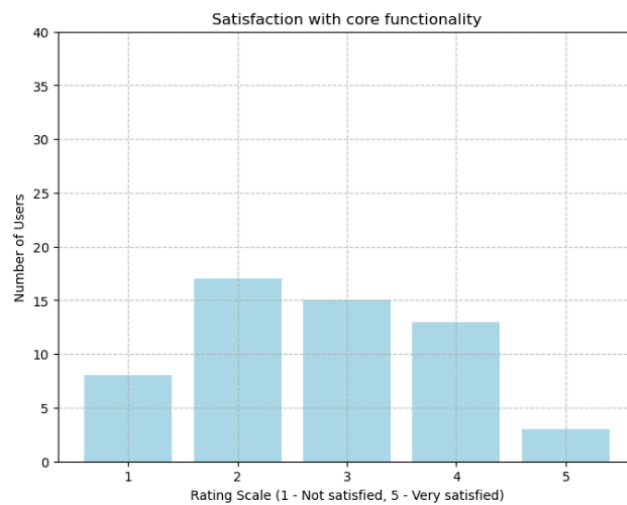


Figure 3.39

We received some feedback that while the sharing, explore, deals and profile page are well laid out, some functions are missing or do not work very well and are not very user friendly, such as the upload image function where users have to upload a link of the images.

Encountered Bugs or Issues

This question measured the extent to which participants encountered bugs or issues during their use of the prototype.

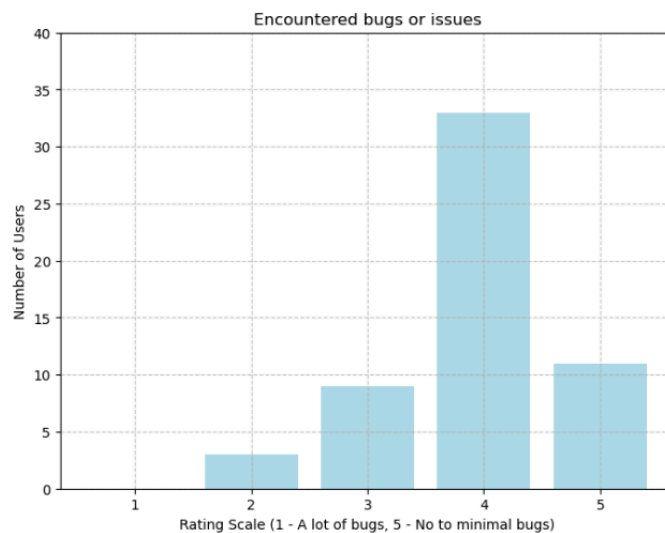


Figure 3.40

The feedback we received stated that although there are some minimal bugs, those bugs did not affect the user experience much. The main bug that most users faced was that the application does not update the deals properly sometimes and that some mild buttons such as the back buttons did not work.

4. Application Loading Speed

Participants were asked to evaluate the loading speed of the application.

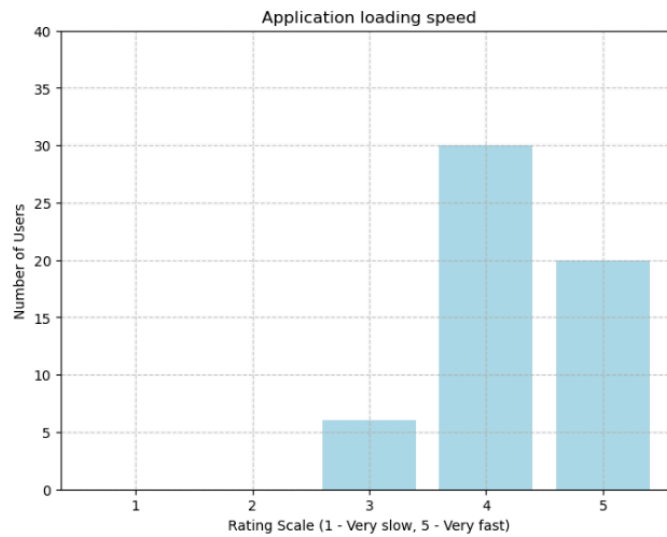


Figure 3.41

This indicates that most users found the loading speed to be fairly fast, suggesting a generally positive perception of the app's performance.

5. Consistency Across Screens

This question assessed the consistency of the design across different screens within our application.

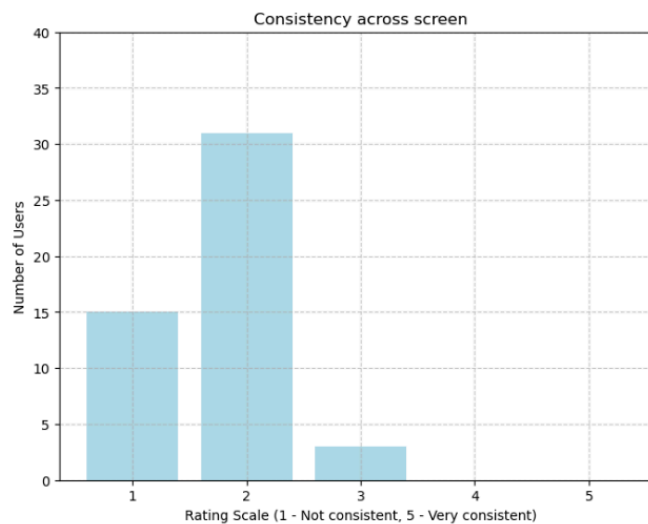


Figure 3.42

The feedback received was that there is no common theme between the pages and the design of the pages are very bare and minimal. The overall aesthetic, colour scheme and font size can be improved.

6. Difficulty in Using Features

Participants were asked whether they found any features difficult to use.

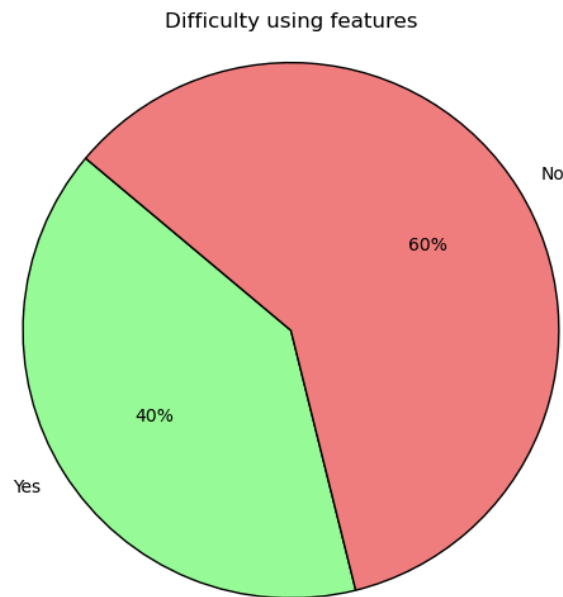


Figure 3.43

The feedback received was that while most of the main available features are straightforward and usable with minimal bugs, the image upload function could be improved as uploading images using a link is inconvenient. Users are also unable to freely browse if the user is not signed in.

Phase two: Specific features and overall satisfaction

This test focused on evaluating the specific features added in the second iteration of our application. It aims to gauge overall user satisfaction as the application. This test involved the previous 56 participants, who provided feedback on the usability of new features, responsiveness, design quality, and improvements compared to the first prototype, to ensure the reliability of the results.

The feedback from this phase was largely positive. Participants found the newly added features highly usable and were satisfied with the application's responsiveness and performance. The design met expectations, and users noted significant improvements compared to the first prototype. Most participants indicated that they would recommend the application and believed it was ready for beta deployment.

Below are the questions and the responses:

1. Usability of Newly Added Features

Participants to rate the usability of the newly added features in our application, specifically the image upload function and login-based features restriction.

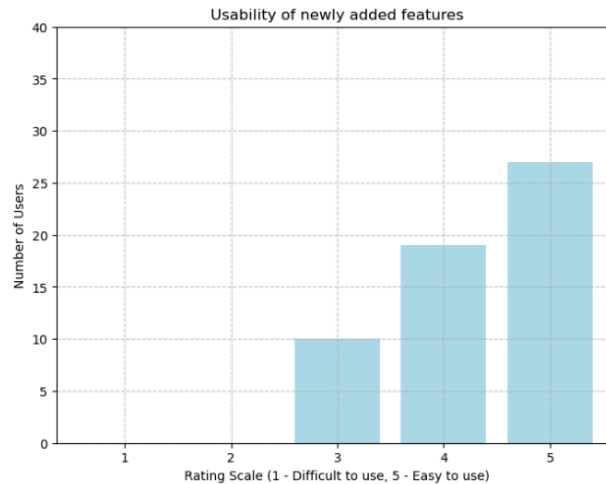


Figure 3.44

Most users find the features easy to use, and many feedback that the new upload function that allows users to upload images from their phone gallery was very helpful. They also mentioned that the redeem slider was interactive and easy to use with just one swipe, and the comments and like comment functionalities were simple, easy to use and made the app engaging and interactive.

2. User experience with new thematic design

Aim to determine if the improvement in visual designs made the application more appealing to users.

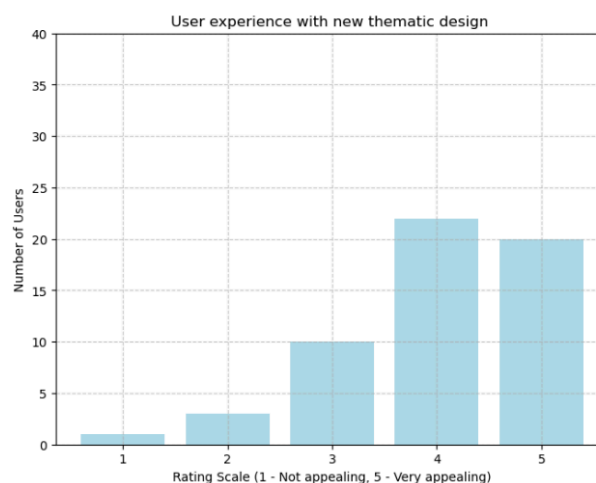


Figure 3.45

Users feedback that the designs and theme was more appealing and more aesthetically pleasing.

3. Overall Improvement Compared to First Prototype

This question aimed to gauge the perceived improvement in the application compared to the first prototype.

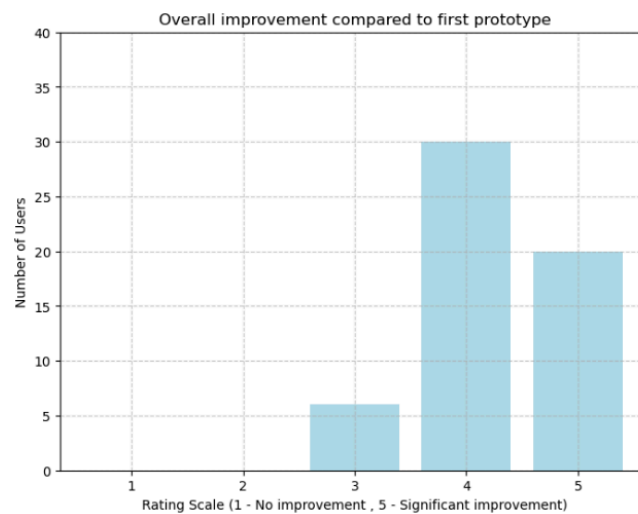


Figure 3.46

Most users stated that there was a significant improvement compared to the first prototype and that they were more satisfied with the applications in terms of usability.

4. Difficulty with the updated features:

Aim to identify specific difficulties users faced when using our new features.

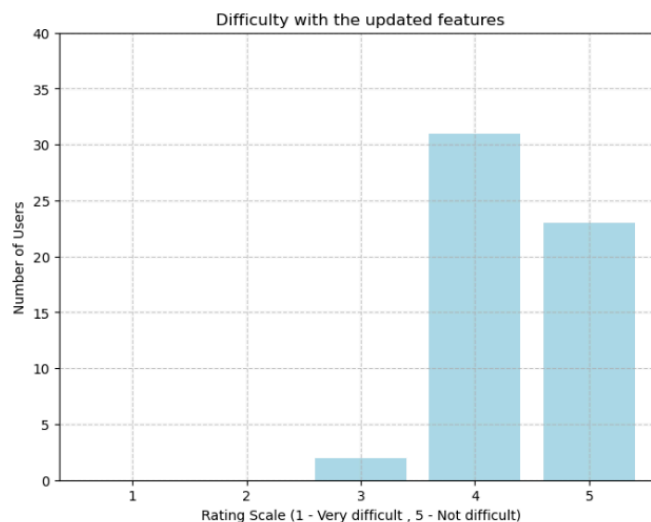


Figure 3.47

Most users stated that they did not find the new features difficult to use and that the features were very straightforward with the necessary guiding instructions to tell the user what to do. The new features include the register page, reply comments, like comments, redeem slider, change profile picture and display name and upload image from gallery.

5. Meeting Expectations for a Functional Prototype

Participants were asked whether the application met their expectations for a functional prototype.

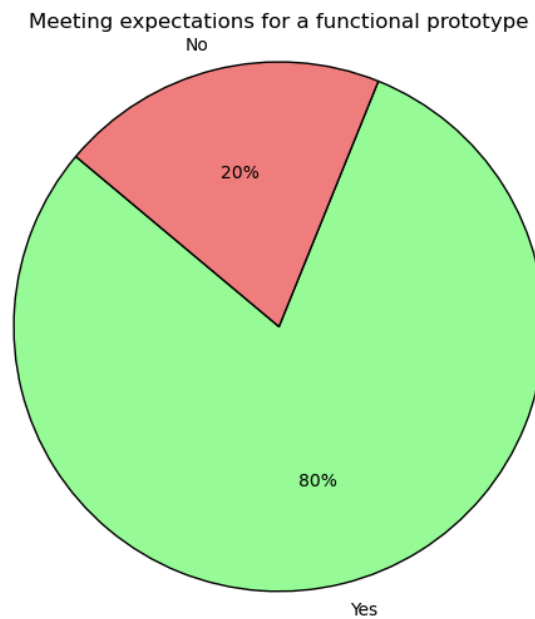


Figure 3.48

Majority were satisfied with the prototype's functionality, while a small portion saw room for improvement.

6. Is the Application Ready for Beta Deployment?

Participants were asked if they believed the application was ready for beta deployment.

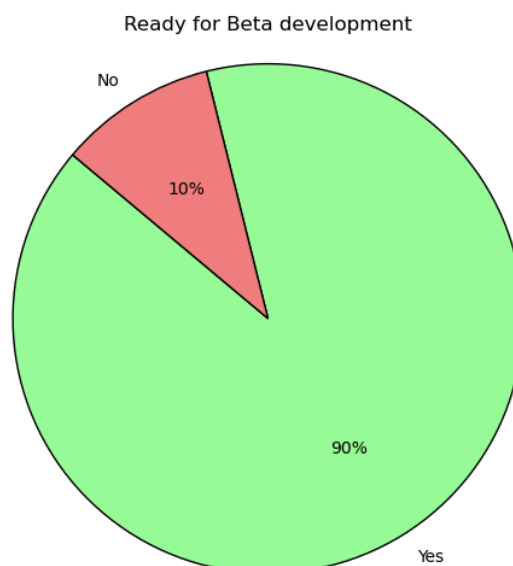


Figure 3.49

Most users stated that the application was ready for beta development

7. Would You Recommend This Application?

Participants were asked whether they would recommend the application to others based on their experience.

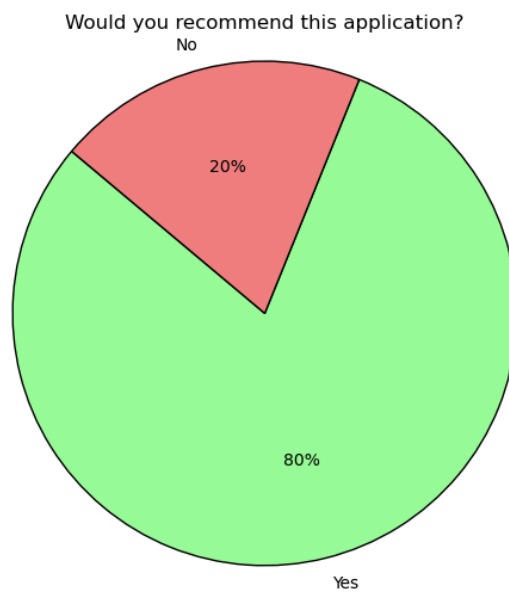


Figure 3.50

Most participants stated that they would recommend the application to others based on their experience.

DESIGN

Front-end design

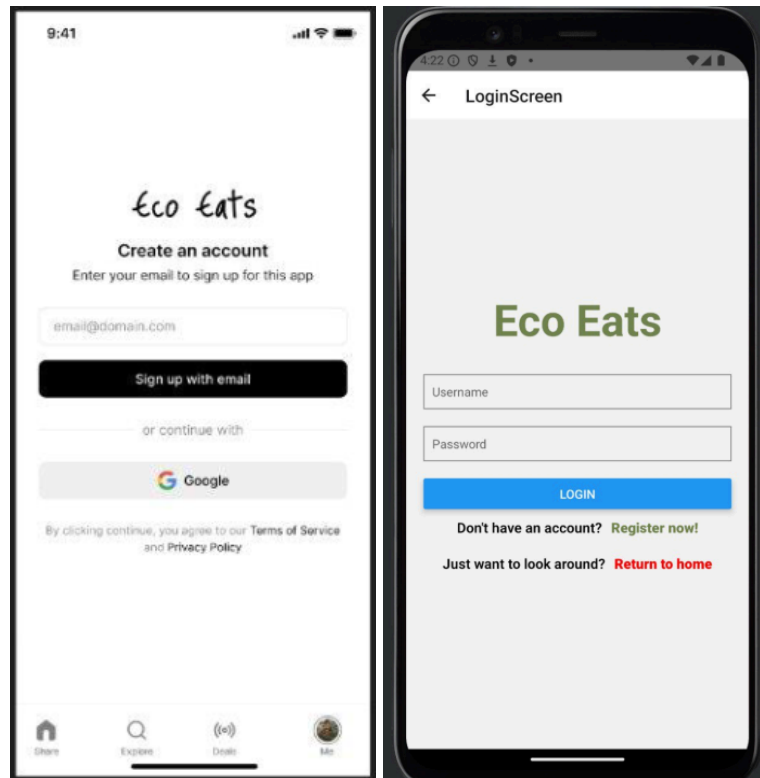


Figure 4.1 and 4.2: Comparison between proposal's and final's "Login" page

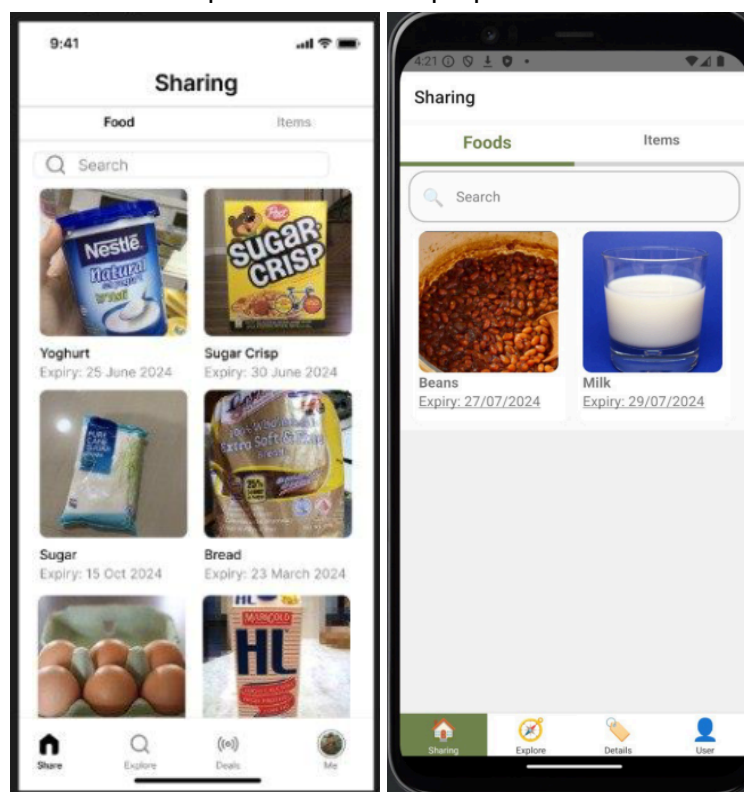


Figure 4.3 and 4.4: Comparison between proposal's and final's "Share" page

In comparison to the initial prototype, our final design features an expanded colour palette. This broader palette not only enhances the visual appeal, but also helps in differentiating between various UI elements and guiding user interactions.

Aside from the updated colour scheme, most of the UI design has remained consistent with the original prototype. However, there are a few notable changes such as the buttons used to display different types of “Share” posts have been revamped. An underline now appears to indicate the type of post the user is currently viewing. This subtle change provides clearer visual cues. Additionally, the login page's title has been simplified and is less stylised than in the Figma prototype.

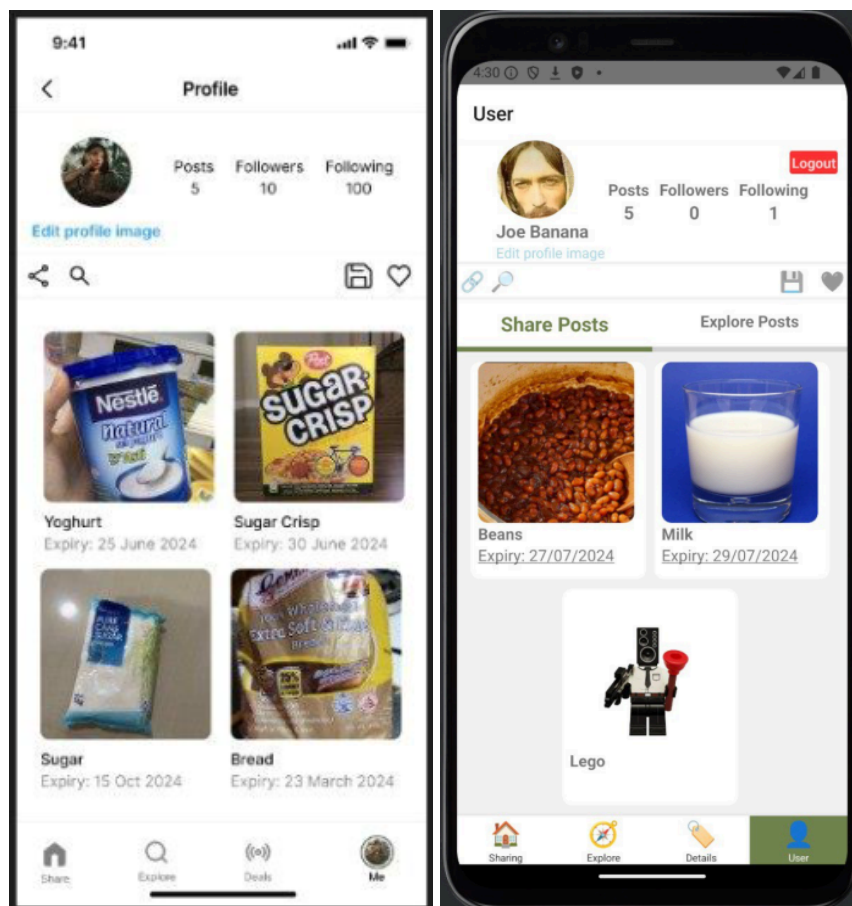


Figure 4.5 and 4.6: Comparison between proposal's and final's “User” page:

Back-end design

The most significant changes in our project is the transition from a mobile-friendly website to a mobile application. This shift led to a change in the language we used. Initially, the project was to be built using HTML, JavaScript, and CSS. However, moving to mobile app development requires us to use TypeScript, the primary language for React Native development. Additionally, since the project is now a mobile application, there is no longer a need to set up Node.js.

We opted to use local SQLite databases instead of an online database such as MongoDB. While this decision limits the application's current functionality to being a demonstration, it simplifies the development process. Fortunately, the robust planning of the application's architecture allows the transitioning to a proper online database in the future can be accomplished with ease.

We also decided to remove the proposed geolocation service as we realised that setting up a database with the exact coordinates of all restaurants in Singapore would be extremely time-consuming to complete within the project's timeframe.

```
const App = () => {
  return (
    <SafeAreaView style={{flex: 1}}>
      <View style={styles.container}>
        <MapView
          style={styles.mapStyle}
          initialRegion={{
            latitude: 37.78825,
            longitude: -122.4324,
            latitudeDelta: 0.0922,
            longitudeDelta: 0.0421,
          }}
          customMapStyle={mapStyle}>
          <Marker
            draggable
            coordinate={{
              latitude: 37.78825,
              longitude: -122.4324,
            }}
            onDragEnd={
              (e) => alert(JSON.stringify(e.nativeEvent.coordinate))
            }
            title={'Test Marker'}
            description={'This is a description of the marker'}
          />
        </MapView>
      </View>
    </SafeAreaView>
  );
};
```

Figure 4.7: A snippet of code that required to use GoogleMap API in React-Native.

The differences between initial and final database designs are apparent. The proposal's Entity-Relationship Diagram (ERD) included several unnecessary tables, such as separate tables for different account types. In contrast, the final database design only utilises two tables: "User" and "User_credentials". Account types in the final design are differentiated by a value in the "type" column within the "User_credentials" table. This simplification aims to reduce complexity while efficiently fulfilling the application's requirements. Overall, a more simple concise design can help the development process in the long term.

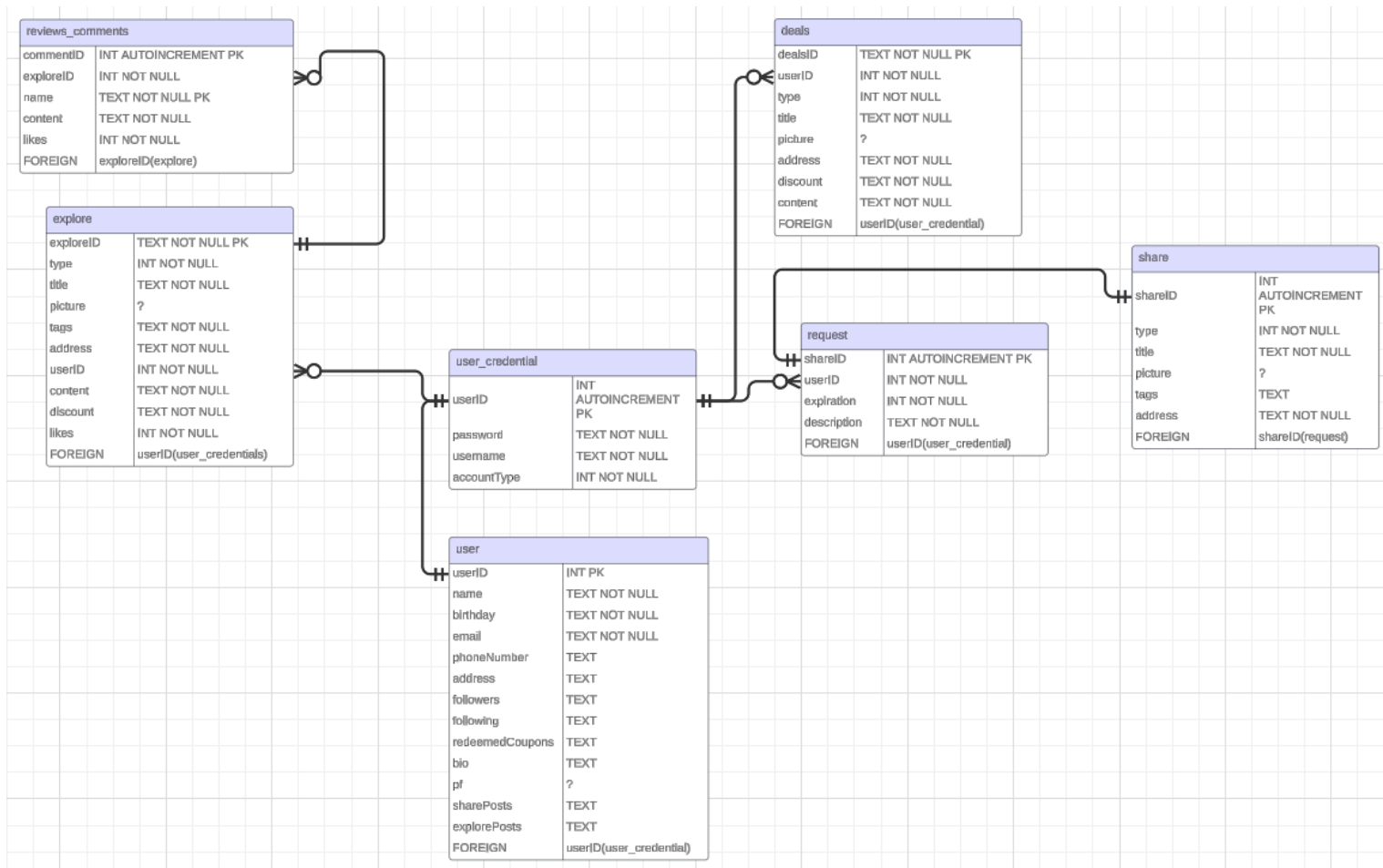


Figure 4.8: The ERD of EcoEats' final database.

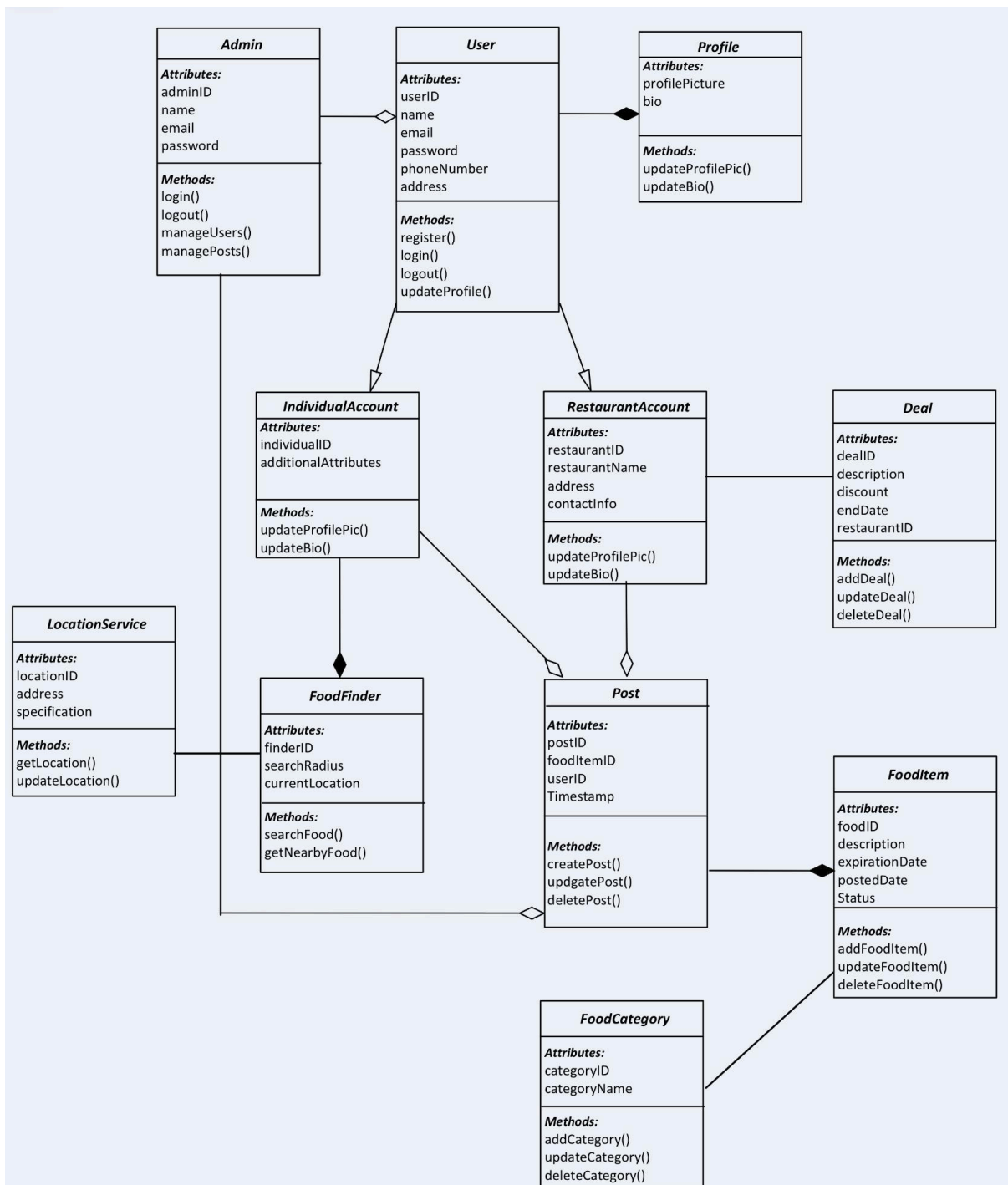


Figure 4.9: The ERD of EcoEats' proposal database.

SYSTEM DEVELOPMENT

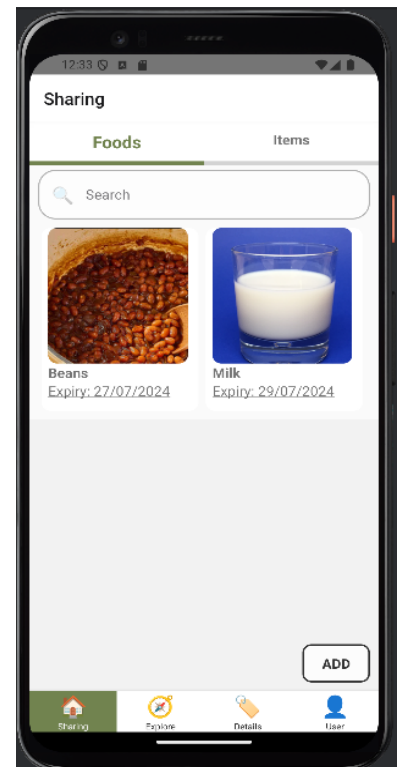
Product's core components

The prototype in the proposal reports consists of the following pages: Share, Request, Explore, Deals, and User, with each serving distinct functions.

Share Page

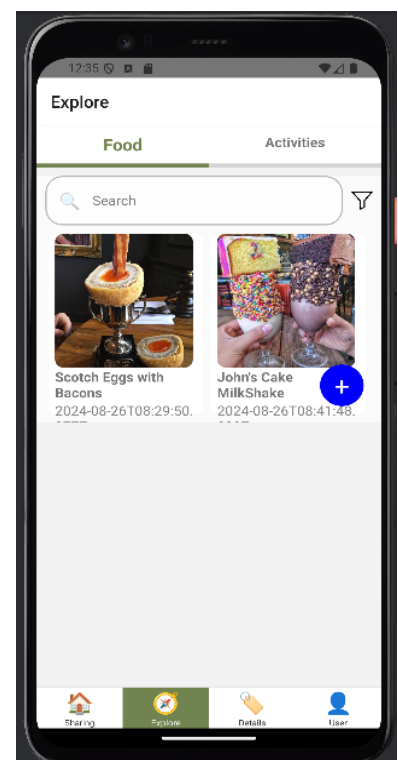
The Share page functions as the home page, enabling users to view food or objects others are willing to share or donate. All users, regardless of their login status, can browse these posts and tap on them to access the Request page for detailed descriptions and to make a meeting request.

Logged-in users have the additional capability to create new sharing posts. The page also features a search bar and a filter system to allow users to filter the displayed posts by category.



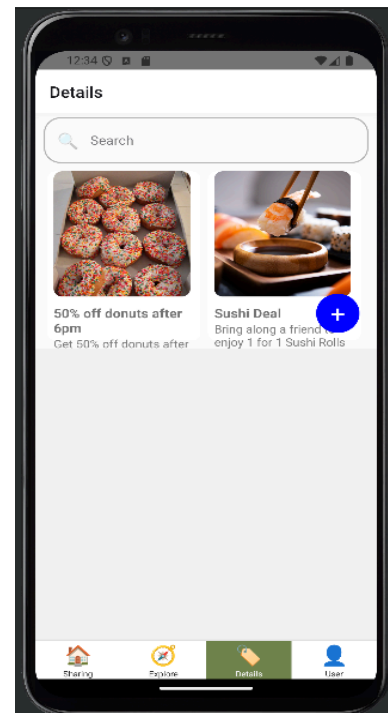
Explore Page

The Explore page offers similar functionalities to the Share page but focuses on user recommendations of restaurants and activities across various price ranges. Posts on this page include detailed reviews, and users can view the number of likes and comments associated with each post. Only logged-in users can like or comment on these posts.



Deals Page

This page displays discounts and offers from eateries and services that collaborate with the platform. Logged-in users can claim these offers. Like the Share and Explore pages, the Deals page includes a search bar and a filter system to narrow down the types of restaurants or activities shown.

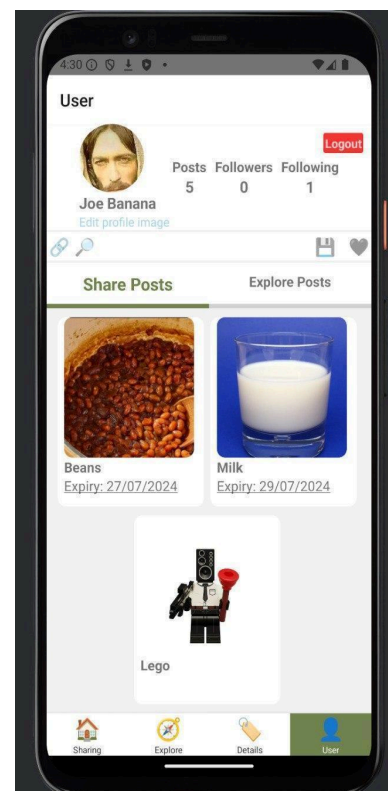


User Page

Two types of user accounts, normal users and business accounts, are differentiated with the business account having the ability to post new deals.

The User page displays all posts made by the user across both the Share and Explore pages. Logged-in users can follow other profiles, share profiles via links, and view posts they have liked.

Otherwise, not logged-in users will be redirected to the login page if they attempt to view any profiles. A registration page is also available for new users.



Approach taken with Test-Driven Development

Throughout the project, we employed a Test-Driven Development (TDD) approach, which was facilitated by our use of GitHub for version control and Trello as our Kanban board.

Trello allowed us to outline the tasks required for each feature, with clear expectations for the output. With each task being further divided into smaller, manageable parts, we utilised GitHub to run the according tests safely.

Each feature was developed by first writing tests to define the expected behaviour before proceeding with the actual implementation. This ensures that the code meets the minimum requirements and allows us to catch potential bugs early on. After passing all tests, the feature was merged into the main branch.

As seen in the figure below, multiple commits were made to one of our GitHub branches, demonstrating the incremental progress. By committing regularly, we ensured that each step of development was tested and functional before moving on to the next, minimising risks of larger issues arising later in the process.

Furthermore, adhering to TDD allowed us to discover a minor error that by setting 'picture' as "any" type could cause interaction SQLite data retrieval issues and eventually unable to display a post's image properly.

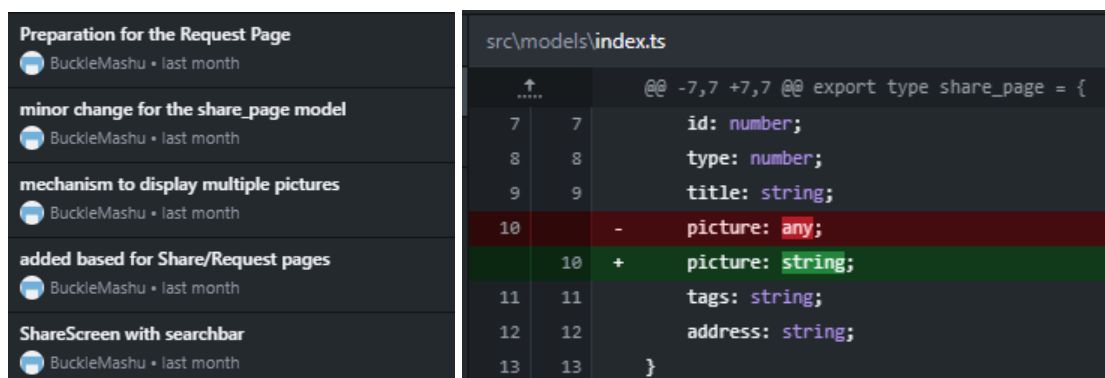


Figure 5.1: Incremental progress of this project to demonstrate TDD

Experience highlights

React Native Setup

Setting up the React Native environment posed significant challenges, especially for team members unfamiliar with emulators and virtualisation. The multiple methods for initialising a React Native project further added to the confusion, making the setup process particularly daunting for those new to the platform.

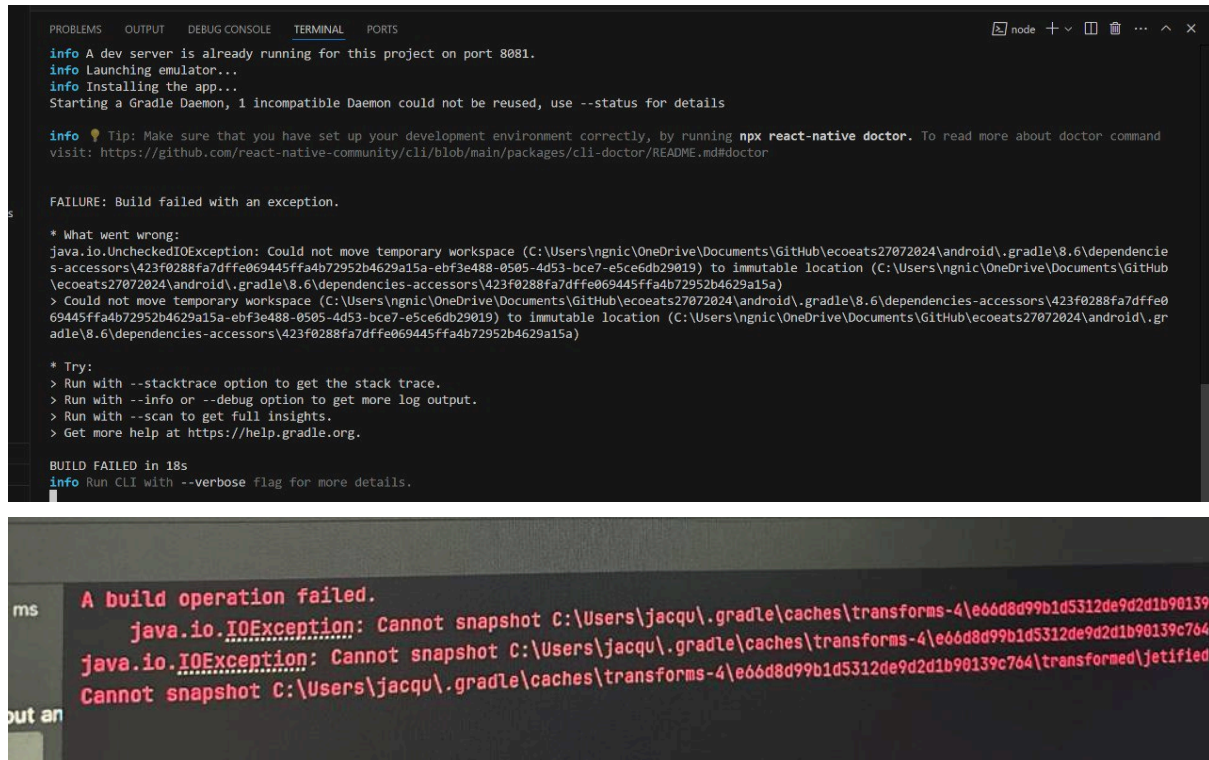


Figure 5.5 and 5.6: Images of Errors we faced in React-Native

Learning Curve with React Native

Although our team had experience in web development, transitioning to React Native was not without its difficulties. The differences in formatting, particularly in styling and component usage, required a period of adjustment. For example, while React Native's styling method is similar to CSS, it has enough differences to cause confusion and slow down development.

Frontend Development Challenges

Implementing image upload features on the Deals, Explore, and Sharing pages was a significant challenge. Initially, users had to input image URLs manually, which was not user-friendly. Transitioning to a system where users could select images from their gallery improved usability but required managing permissions, handling different image formats, and ensuring proper display across devices.

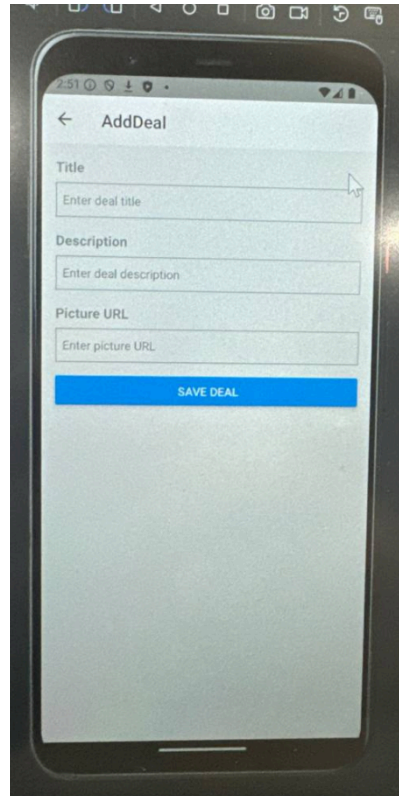


Figure 5.7: Image where user had to input image url

Creating the Redeem Slider on the Deals page involved ensuring it functions correctly across devices and persisted its state (redeemed or not) even when users navigated away from and back to the page. This required careful state management and backend integration to maintain the redemption status. Adding the comment and like functionality also required meticulous backend management to ensure that the like counts and comment threads were accurately maintained across different user sessions.

Maintaining a consistent visual theme across all pages was challenging. As new features were added, visual inconsistencies emerged, requiring continuous refinement of styles and components to ensure uniformity.

React Native's limitations with dynamically stored images necessitated the development of a workaround where images were mapped to URLs at app launch, making them static and displayable. Later, we integrated Imgur to manage image uploads more efficiently, allowing users to upload and store images seamlessly.

```

<Image style={styles.mainImage} source={{uri:post.picture || 'https://i.imgur.com/50exbMa.png'}}/>
<View style={styles.middleDetails}>
  <Image style={styles.pf} source={{uri:userPicture || 'https://i.imgur.com/50exbMa.png'}}/>
  <View style={styles.titleNexpiration}>
    <Text style={styles.title}>{post.title}</Text>
    {post.expiration != 'none' && (

```

Figure 5.8: Code using Imgur

Backend Development Challenges

Initially, we attempted to save the database in the Android assets folder, but this method didn't support adding new rows during app usage. Implementing an SQLite database was necessary, but it has an unsolved bug. The database had to be manually copied to the correct directory on the device, a process that involved replacing empty database files generated.

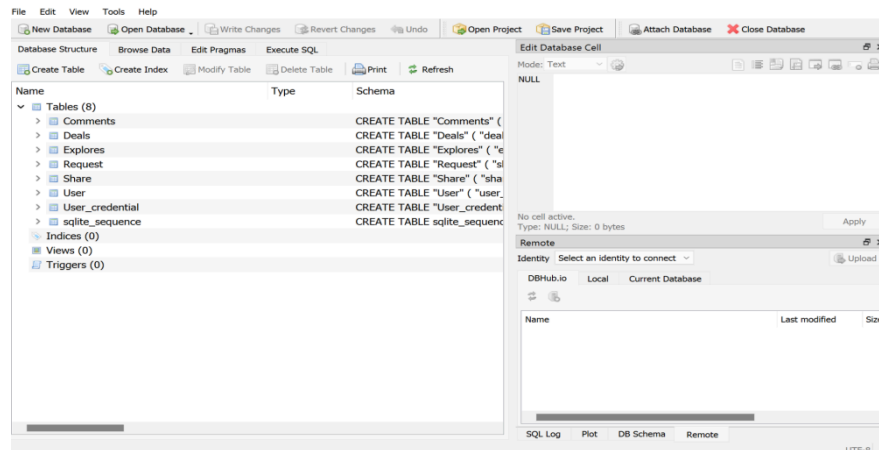


Figure 5.9: Image of EcoEats Database

As the app evolved, managing database migrations and ensuring data integrity became crucial. Adding new fields to tables, such as the `isRedeemed` field in the `Deals` table and the `like_count` field in the `Comments` table, required careful migration to avoid disrupting existing data. Optimising database queries to handle increasing amounts of data also became necessary to maintain app performance.

Configuring the development environment and setting up React Native with the necessary libraries and dependencies was a significant challenge. We also faced issues with third-party components for image handling and swipe gestures.

Overall, the development process involved overcoming a variety of challenges across different aspects of the project.

Code challenges

Throughout the development of our application, we identified several areas where the backend and frontend could have been improved to create a more streamlined and effective user experience.

Frontend Product Concept Issues:

One key area for improvement on the frontend was the initial design and implementation of the user interface. Early on, the absence of a unified design system led to inconsistencies in the user interface, with variations in button styles, sliders, and input fields. Establishing a comprehensive design system from the start, with consistent styles and components, could have significantly enhanced visual coherence and usability.

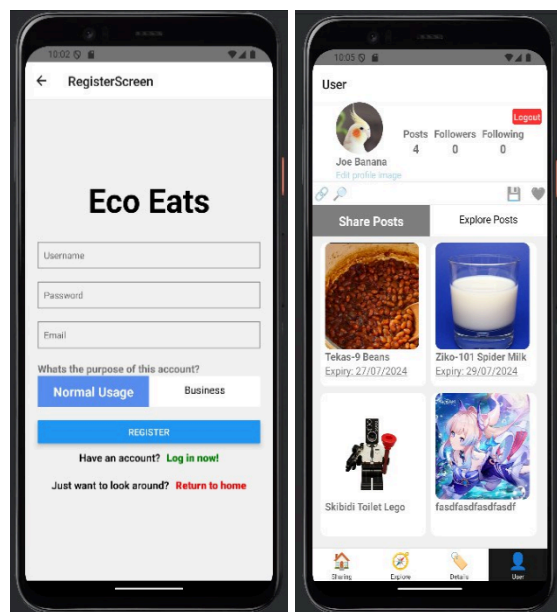


Figure 5.10 and 5.11: Images of Old App Design with no common Green EcoEats Theme

The app's responsiveness across various devices also required more attention. While it was designed to work on different screen sizes, components like image galleries and the Redeem Slider needed further adjustments to be more dynamic.

A more thorough approach to responsive design, using flexible layouts and media queries, could have ensured a consistent experience across devices.

Error handling and user feedback on the frontend could also have been more robust. For example, when users uploaded images or redeemed deals, the feedback was sometimes unclear. Implementing more effective error messages and user notifications could have improved the overall user experience by reducing confusion and frustration.

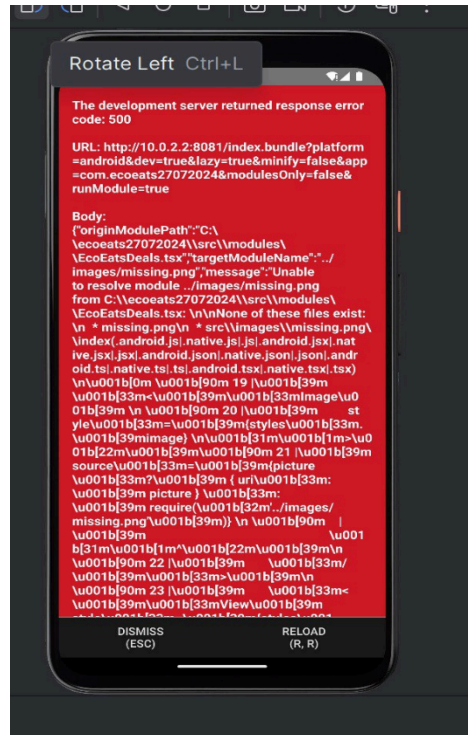


Figure 5.12: Image of unclear error

Backend Product Concept Issues

On the backend, the initial database design was a significant area for improvement. As the project evolved, frequent schema changes were necessary. A more forward-thinking approach to database design, anticipating future features and scalability, could have reduced the need for such changes.

Managing asynchronous operations and state synchronisation was another challenge. Initially, there were issues in ensuring that user actions, like redeeming a deal or uploading an image, were consistently reflected across the app and stored correctly. Implementing more robust state management could have improved data consistency and user experience.

Additionally, the backend could have benefited from better logging and monitoring tools. Troubleshooting issues like database errors was hindered by a lack of detailed logs. Implementing comprehensive logging and monitoring from the beginning could have facilitated quicker resolutions.

The setup and configuration of the development environment across different devices also posed challenges. Better documentation and automation, such as using Docker for consistent development environments, could have reduced setup time and minimise errors.

ANALYSIS

Code Solutions to Aims and Objectives

The primary aim of EcoEats was to create a mobile platform that facilitated the sharing and discovery of surplus food, making food more accessible while reducing waste.

User Management and Authentication

One of the core objectives was to ensure secure user management, which was achieved through the implementation of secure login and registration processes. The use of SHA-256 password hashing ensured that user credentials were safely stored, adhering to best practices in data security. In doing so, the project addressed the issue of secure data handling, which is critical for any platform that involves personal information. However, as noted in literature (e.g., Anderson, 2020), multi-factor authentication could further enhance security. This represents an opportunity for future improvement.

Social Interaction and Community Engagement

EcoEats included features such as likes, comments, and followers, which facilitated interaction and engagement between users. According to social media engagement theories (Boyd & Ellison, 2010), these features are critical for building an active and involved user community. The successful implementation of a nested comment system, where users can reply to comments, added depth to the interaction model.

Redeem Slider

The redeem slider was a novel solution implemented on the Deals page, allowing users to claim deals in an intuitive and visually engaging way. The slider design aimed to simplify the interaction, reducing the cognitive load on users. According to Nielsen's usability heuristics (Nielsen, 1994), this aligns with the principle of "minimising user effort".

Methodologies and Their Impact

Agile Development

Our team adopted an Agile methodology, which facilitated iterative development and rapid response to challenges. According to Schwaber and Sutherland (2017), Agile is particularly effective in projects where requirements evolve over time. This was the case with EcoEats, where the initial project scope was refined based on user feedback and the discovery of technical constraints. The flexibility of Agile allowed the team to shift priorities—such as dropping the website version in favour of a mobile app—and to focus on core functionalities like user authentication and posting mechanisms.

However, the scope creep observed during the addition of non-core features (e.g., commenting system) hindered the timely completion of some foundational elements, such as database optimization. This reflects a common pitfall of Agile, where flexibility can lead to prioritisation issues if not carefully managed (Poppendieck & Poppendieck, 2003). More structured sprint planning could have mitigated this.

User-Centred Design (UCD)

The project also employed user-centred design principles, which align with the literature on effective UX design (Norman, 2013). By conducting two rounds of user testing—one for basic functionality and one for feature refinement—the team was able to gather valuable insights that shaped the final product. For example, user feedback on the comment and like system led to improvements in the way nested comments were displayed.

However, one area where UCD could have been more rigorously applied is in the onboarding process. Incorporating more onboarding tutorials or contextual help features could improve the overall user experience, as suggested in studies by Vredenburg et al. (2002).

EVALUATION

The development of the EcoEats mobile application was a dynamic and iterative process, involving both technical and collaborative challenges. The project's success is evident in its ability to provide a functional platform for food sharing and surplus management, but there are several areas where improvements could be made. This evaluation will critically assess the development processes, team collaboration, and the final product, while highlighting key areas for future work.

Development Processes

Throughout the project, we employed a combination of Agile methodologies and user-centred design principles. These processes allowed us to break down the project into manageable sprints, focusing on iterative development, user feedback, and continuous testing. However, while the Agile approach facilitated adaptability and rapid response to challenges, we encountered issues related to time management and prioritisation.

Agile Methodology

The use of Trello as a Kanban board helped in tracking the progress of various tasks and keeping the team aligned. However, the division of responsibilities, particularly when faced with technical challenges, sometimes slowed the process. Some tasks took longer than initially planned, such as integrating image upload functionalities and implementing password hashing. This reflects the need for a more rigid time allocation strategy in future projects to ensure that critical milestones are met.

Collaboration and Communication

Team communication was primarily handled via Telegram and Discord, which worked well for real-time problem-solving and maintaining the workflow. However, coordination between team members with different skill levels presented occasional challenges. For instance, onboarding new members into React Native development took longer than expected, affecting the speed of implementation. This could have been mitigated by scheduling more frequent pair programming sessions or collaborative coding reviews to address knowledge gaps more efficiently.

Technical Challenges

The decision to implement the project as a mobile application using React Native introduced several technical challenges, particularly in terms of setting up a consistent development environment and managing device compatibility issues. While the flexibility of React Native allowed for cross-platform development, the learning curve slowed initial progress. Additionally, the SQLite database integration required more effort than anticipated due to limitations in managing data insertion during runtime.

Testing and Iteration

User testing was a significant part of the process, with empirical evaluations providing insights into user experience and functionality. While the iterative process allowed us to refine key features like the comment system and redeem slider, the scope of testing could have been broadened. For example, testing with a more diverse user group would have provided more comprehensive feedback, especially in identifying bugs or usability issues that might not be immediately apparent to our core testers.

Team and Methodological Reflection

Overall, the team's performance was strong in areas of problem-solving and collaboration, but there were moments where a more structured approach could have yielded better results. Pair programming and code reviews were underutilised, which could have helped address some technical issues earlier in the development process. In future projects, implementing more rigorous continuous integration (CI) practices would ensure that the codebase remains stable as features are added, reducing the number of bugs and regressions.

Final Product Evaluation

The EcoEats platform successfully delivers on its core objective of creating a space for food sharing and community engagement, yet there are several areas for enhancement in both its functionality and design.

User Interface (UI) and Experience (UX)

While the UI is generally user-friendly, there are some inconsistencies across pages that could benefit from refinement. Each page has distinct layouts, but the overall aesthetic needs more cohesion, particularly in terms of colour schemes and button styles. Future work should focus on establishing a more unified design system that provides a seamless user experience across all sections of the app. Additionally, more advanced UI testing could help identify friction points in user navigation, particularly for less tech-savvy users.

Backend and Database Management

The current implementation of the SQLite database works well for a demo version of the app, but it presents limitations in scalability. As the user base grows, there may be performance issues with data retrieval and management. Future versions of the app would benefit from transitioning to a more robust database solution, such as MySQL or a cloud-based database like Amazon RDS.

Security and Data Handling

The implementation of password hashing using the SHA-256 algorithm significantly improved the security of user credentials, but other security measures could be enhanced. For example, integrating two-factor authentication (2FA) would provide an additional layer of security, especially as the platform scales and handles more sensitive user data.

Future Work and Improvements

Several key areas for future development can enhance both the user experience and the scalability of the platform:

Scalability and Cloud Integration

Transitioning the backend to a cloud-based solution would allow the app to handle a growing number of users more efficiently. This would also enable features like real-time notifications and geolocation-based search functionalities.

User Feedback and Personalization

Future iterations of EcoEats could introduce more personalised features, such as recommendation systems that suggest food deals based on a user's previous interactions. These enhancements would help increase user retention and engagement.

Design Cohesion

Establishing a more uniform design system, incorporating feedback from diverse user groups, and refining the UI would create a more polished and professional look for the platform.

Extended Security Measures

Introducing 2FA, improving error handling, and integrating secure payment gateways would bolster the app's overall security and make it more viable for public use.

CONCLUSION / SUMMARY

The EcoEats project successfully achieved its core aim of creating a mobile application that promotes both eco-friendly and economically friendly practices by reducing food waste within local communities. We provided users with a convenient way to share and claim surplus food, fostering a sense of community while contributing to sustainability efforts. The mobile app allows individuals and businesses to post food deals, share surplus items, and engage in interactions.

One of the most significant achievements of this project is the creation of a functional, user-friendly platform that operates seamlessly on mobile devices, allowing for on-the-go accessibility. The decision to prioritise a mobile app over a website has proven effective in meeting the needs of our target audience. The development process also resulted in the successful implementation of key features, such as posting food and items, password hashing for secure user authentication, a redeem slider for deal claiming, and a comment and like system.

The impact of EcoEats extends beyond the technical achievements. By providing a platform that helps reduce food waste, EcoEats has the potential to influence environmental sustainability efforts in local communities. Users are empowered to participate by sharing food that would otherwise go to waste, lowering the community's overall carbon footprint and promoting more responsible consumption practices. Additionally, the platform contributes to economic accessibility by enabling users to access affordable food options, addressing food insecurity while benefiting those who wish to avoid wasting excess food.

Despite these accomplishments, the project also highlighted several areas for potential improvement and future work such as adding the extra functionalities we originally planned on implementing. Scalability remains a key consideration, as the current infrastructure would need to be expanded to handle larger user bases.

APPENDIX

Work history

Cuong

- Familiarise myself with React-Native by creating a 'To do list' application.
 - Identified the libraries needed for our application to function.
 - Implemented the 'Bottom navigation bar', 'Temporary SQLite database for data retrieval and insertion', and 'Page navigation'.
- Set up the GitHub repository and invited all teammates as collaborators.
- Helped teammates to set up their React-Native development environment and Android Studio.
- Designed the relationships between SQLite data tables for the application.
- Convert the said 'To do list' application into a base template for the group to work on their respective pages.
- Initially implemented the "react-native-image-picker" library to allow users to select an image from their device to be used in the application.
 - Used in "Share" page, "User" page, "Deals" page, and "Explore" page.
- Developed the entire 'Share' page.
 - Code the backend of the 'Share' page
 - Display the posts.
 - Filter posts based on their type.
 - Implemented a working search bar that filter post based on their titles.
 - Tapping on a post will redirect the users to a page that will display the post's details and if the users is logged in, allows them to make a request. This page is called the 'Request' page.
 - If the user is logged in, they can make a post to the share page through the "addShareItem" page.
 - Style the front end of the 'Share' page
- Developed the entire 'addShareItem' page
 - Code the backend of the 'addShareItem' page
 - Tapping the 'Add' button in the 'Share' page will redirect the user to this page.
 - Takes in inputs from the textboxes, image selected, and post's type button to insert a new row into the 'Share' page's data table as a new post.
 - Redirect the user to the 'Share' page after posting an item.
 - Style the front end of the 'addShareItem' page
- Developed the entire 'Request' page.
 - Code the backend of the 'Request' page
 - Display the post's content.
 - Tapping the poster's name will redirect the user to the poster's profile.
 - Implemented popup screen (with Modal) and dropdown boxes (with Picker) into 'Request' page.
 - If the user is logged in, the user can request a meet up and select their desired meeting date and time with the poster.
 - Style the front end of the 'Request' page
- Developed the entire 'User' page.

- Code the backend of the 'User' page
 - Display the profile's content.
 - Display the number of posts made by the user, the number of followers the user has, and the number of people the user is following.
 - Display all of the 'Share' posts made by the displayed user.
 - Display all of the 'Explore' posts made by the displayed user.
 - Filter posts based on their type.
 - If the user is logged in, the user can change their profile picture, profile name, and logout of the application.
 - If the user is not logged in, view the 'User' page will automatically redirect the user to the 'Login' page.
- Style the front end of the 'User' page
- Implemented Imgur API into the application as an image hosting database service.
 - Done by implementing the 'axios' library.
- Reimplemented the SQLite database and have it stored locally in the emulated device rather than as a 'read-only' database from the project's android assets folder.
- Developed the entire authentication system to check if the user is logged into the application.
 - Done by using react's createContext.
- Heavily improved the backend of the 'Login' page.
 - Improves the redirection to the "Login" page if the user is not logged in.
 - Initially implemented the comparison of password and username to check if the login credentials are correct.
 - Allows users to be redirected to the 'Register' page if they want to make an account.
 - Allows the users to be redirected to the 'Share' page if they just want to look through what the application offers.
- Developed the 'Register' page.
 - Code most of the backend of the 'Register' page
 - Initially takes the inputs from the credential textboxes and account type button to insert a new row into the 'User_credential' data table as a new user.
 - Set up a default profile details for newly created account and insert them into the 'User' data table as a new row.
 - Allows users to be redirected to the 'Login' page if they already have an account.
 - Allows the users to be redirected to the 'Share' page if they just want to look through what the application offers.
 - Partially style the front end of the 'Register' page
 - Used Xiang En's 'Login' page styling as a base.
- Resolves most of the bugs or errors the team face during the development process.
- Overlooked the merging of branches and debug any issues arised from the merging.
- Wrote the entire 'Planning and Research' section of the report.
- Wrote the entire 'Design' section of the report.
- Improve the 'Prototype and Iteration' section of the report.
- Wrote the 'Product's core components' and 'Approach taken with Test-Driven Development' of the 'System Development' section of the report.

Nicole

- Familiarise myself with React-Native and created a To Do list to keep track of pending tasks
- Learnt how React-Native works and the libraries needed
- Set up GitHub and joined the repository
- Resolved path and file permission issues to ensure the development environment was correctly configured
- Set up React-Native development environment and Android Studio.
- Developed the 'Explore' page.
 - Code the backend of the 'Explore' page
 - Created Explores and Comments Table in Database to store items in Explore Page
 - Integrated Database to display the correct items and comments.
 - Implemented navigation between pages, when a user clicks on a post they will be redirected to a page displaying all the information
 - Implemented conditions such that if the user is logged in, they will be able to see a plus button that when clicked, redirects them to the add explore post page to allow the user to create a post.
 - Implemented conditions to allow the user to post comments, reply comments and like comments only when they are logged in.
 - Implemented styling to show nested comments.
 - Allowed each user to only be able to like each comment once and toggle between like and unlike.
 - Users will still be able to view comments and like count if they are not logged in.
 - Successfully navigated back to the Explore page to display all posts after adding a new one.
 - Style the front end of the 'Explore' page
- Developed the 'Deals Page'.
 - Code the backend of the 'Deals' page
 - Created Deals Table in Database to store deals in Deals Page
 - Integrated Database to display the correct items and correct state of the slider.
 - Filter items based on their type.
 - Implement a working search bar that filters posts based on their titles.
 - Implemented navigation between pages, when a user clicks on a post they will be redirected to a page displaying all the information and a slider
 - Implement redeem slider to allow users to swipe to redeem the deal
 - Programmed the slider to turn green and display 'Redeemed' upon a successful slide, ensuring the redeemed state persists in the database across sessions.
 - Used alert to prompt the user when the deal has been redeemed
 - Implemented conditions such that if the user is logged in only as a business account, they will be able to see a plus button that when clicked, redirects them to the add deal page to allow the user to create a deal.

- Style the front end of the 'Deals' page
- Implemented Password Hashing in Register and Login Page
 - Implemented password hashing using SimpleCrypto SHA-256 algorithm to allow passwords to be hashed before being stored instead of storing plain passwords to ensure security and enhance security for user credentials.
 - Resolved the password hashing issues by successfully integrating the hashing function into the React Native app.
 - Stored the hashed password in the original password field of the User_credentials table.
- Used Imgur API to process and save images into the database.
- Reimplemented the SQLite database and have it stored locally in the emulated device rather than as a 'read-only' database from the project's android assets folder.
- Constantly committed and merged changes and resolved any conflicts
- Helped teammates with errors they faced during the setup of react-native and merging of files in GitHub
- Troubleshoot and resolve issues related to Git configuration errors, ensuring a smooth workflow for version control.
- Dealt with path and file permission issues, ensuring that the development environment was correctly set up.
- Refactored existing code to improve readability, maintainability, and performance across various components of the project.
- Ensured that code follows best practices and adheres to the project's coding standards, contributing to a cleaner and more efficient codebase.
- Conducted thorough testing of new features, including the password hashing mechanism and Explore post functionality, to identify and resolve any bugs or issues before deployment.
- Developed and executed test cases for both frontend and backend components, ensuring robust and error-free performance.
- Enhanced the overall user experience by making UI elements intuitive and user-friendly, ensuring that features like the RedeemSlider are accessible and easy to use.
- Gathered and incorporated user feedback to refine and improve the Explore page and Reviews page, aligning the final product with user expectations.
- Collaborated effectively with team members, sharing insights and solutions to overcome challenges related to database management, UI design, and Git configuration.
- Regularly communicated progress and updates, ensuring that the project remained on track and any potential issues were addressed promptly.
- Wrote the 'Empirical' section of the report.
- Modified and improved report based on comments and suggestions.
- Created a cover page for the report

Xiang En

Familiarised with React-Native:

- Created a 'To Do List' app to track pending tasks, helping to solidify my understanding of React-Native.

- Learned how React-Native functions and identified the essential libraries required for our application.

Set Up Development Environment:

- Assisted teammates in setting up their React-Native development environment and Android Studio.
- Set up GitHub, joined the project repository, and familiarised myself with Git commands like commit, uncommit, and effective use of shortcuts.
- Resolved path and file permission issues to ensure the development environment was correctly configured for all team members.
- Constantly committed and merged changes and resolved any conflicts
- Troubleshoot and resolve issues related to Git configuration errors, ensuring a smooth workflow for version control.

Developed the 'Login' Page:

- Coded the frontend and part of the backend for the 'Login' page.
 - Implemented the initial default username and password functionality for initial access.
 - Developed the initial page navigation to redirect users upon successful login.
 - Enabled the initial logout functionality, which redirects users back to the login page when logged out.
 - Learnt how libraries needed for authentication
 - After the Code Lead, Cuong views the coding, he suggested to apply a different way for the backend
 - Initially wanted to use Auth context from React but Cuong decided to use React's createContext.

Contributed to the Project Report:

- Took charge of the overall formatting and styling of the project report, ensuring it was visually appealing and well-organised.
- Assist teammates on their parts for the Final Report.
- Scan through the entire report to check for grammatical errors and sentence structure.
- Authored the 'Background' section, including the introduction, deliverables, features, and an overview of the project.
- Consulted with the professor to identify crucial elements for the report and ensured timely submission by delegating tasks effectively. I communicated which parts of the report were most important and highlighted key areas to focus on.

Thien Tran

- Familiarised myself with React-Native by creating a basic 'Snooker game' using Matter.js to understand game mechanics and React-Native integration.
- Assisted in the implementation of the 'Explore' page functionality:
 - Added features to display and filter posts by category (Food and Activities) using the database's type field.
 - Integrated an image picker functionality to allow users to upload images from their device gallery.
- Contributed to writing the 'Empirical Evaluation' and 'User Testing' sections of the report by drafting survey questions and summarising feedback.

- Using matplotlib to compile and illustrate the survey result.
- Compare and draw conclusions for user test one and two.

Jacqueline

- Wrote the “Initial Concept Design” section of the report.

GitHub page

<https://github.com/BuckleMashu/ecoeats27072024>

Project references

Libraries installed

- **@expo/vector-icons:** <https://www.npmjs.com/package/%40expo/vector-icons>
- **@react-native-picker/picker:**
<https://www.npmjs.com/package/@react-native-picker/picker>
- **@react-navigation/bottom-tabs:**
<https://www.npmjs.com/package/@react-navigation/bottom-tabs>
- **@react-navigation/native:**
<https://www.npmjs.com/package/@react-navigation/native>
- **@react-navigation/stack:** <https://www.npmjs.com/package/@react-navigation/stack>
- **@types/react-native-sqlite-storage:**
<https://www.npmjs.com/package/@types/react-native-sqlite-storage>
- **@types/react-native-vector-icons:**
<https://www.npmjs.com/package/@types/react-native-vector-icons>
- **axios:** <https://www.npmjs.com/package/axios>
- **bcrypt:** <https://www.npmjs.com/package/bcrypt>
- **bcryptjs:** <https://www.npmjs.com/package/bcryptjs>
- **mysql:** <https://www.npmjs.com/package/mysql>
- **react:** <https://www.npmjs.com/package/react>
- **react-native:** <https://www.npmjs.com/package/react-native>
- **react-native-crypto-js:** <https://www.npmjs.com/package/react-native-crypto-js>
- **react-native-fs:** <https://www.npmjs.com/package/react-native-fs>
- **react-native-gesture-handler:**
<https://www.npmjs.com/package/react-native-gesture-handler>
- **react-native-image-picker:**
<https://www.npmjs.com/package/react-native-image-picker>
- **react-native-paper:** <https://www.npmjs.com/package/react-native-paper>
- **react-native-safe-area-context:**
<https://www.npmjs.com/package/react-native-safe-area-context>
- **react-native-screens:** <https://www.npmjs.com/package/react-native-screens>

- **react-native-simple-crypto:**
<https://www.npmjs.com/package/react-native-simple-crypto>
- **react-native-sqlite-storage:**
<https://www.npmjs.com/package/react-native-sqlite-storage>
- **react-native-vector-icons:**
<https://www.npmjs.com/package/react-native-vector-icons>
- **sqlite3:** <https://www.npmjs.com/package/sqlite3>

Website References

Iqbal, S. (2023, February 17). *Secure Login System by Sajid Iqbal, Saeed Ullah Jan, Asad Ullah*. Search eLibrary :: SSRN. Retrieved September 4, 2024, from

https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4359398

Michael. (2024, January 2). *Mobile App Vs. Responsive Website: Which Is The Better?* WEZOM. Retrieved September 4, 2024, from

<https://wezom.com/blog/mobile-app-vs-responsive-website-which-is-the-better>

MoldStud. (2024, January 16). *The Role of Push Notifications in Mobile App Engagement*. MoldStud. Retrieved September 4, 2024, from

<https://moldstud.com/articles/p-the-role-of-push-notifications-in-mobile-app-engagement>

Rehman, S., & Khazin, B. (2021, October 13). *Why mobile app developers need to prioritize user data privacy and security — and what they can do to ensure it*.

Security Magazine. Retrieved September 4, 2024, from

<https://www.securitymagazine.com/articles/96296-why-mobile-app-developers-need-to-prioritize-user-data-privacy-and-security-and-what-they-can-do-to-ensure-it>

SG101. (2022, December 28). *#ChallengeAccepted Food Wastage*. SG101.

Retrieved September 4, 2024, from

<https://www.sg101.gov.sg/resources/connexionsg/foodwaste/>

GitHub Desktop (Version Control)

GitHub Desktop is employed to manage version control, allowing us to keep track of one another's progress. It provides a clear overview of code contributions by displaying each team member's commits and pushes. This ensures that all changes are documented and easily accessible, facilitating coordinated development efforts.

Therefore, referencing our proposal, we decided to not use "Visual Studio Code - Live Share Extension" as it clashes with the purpose of GitHub.

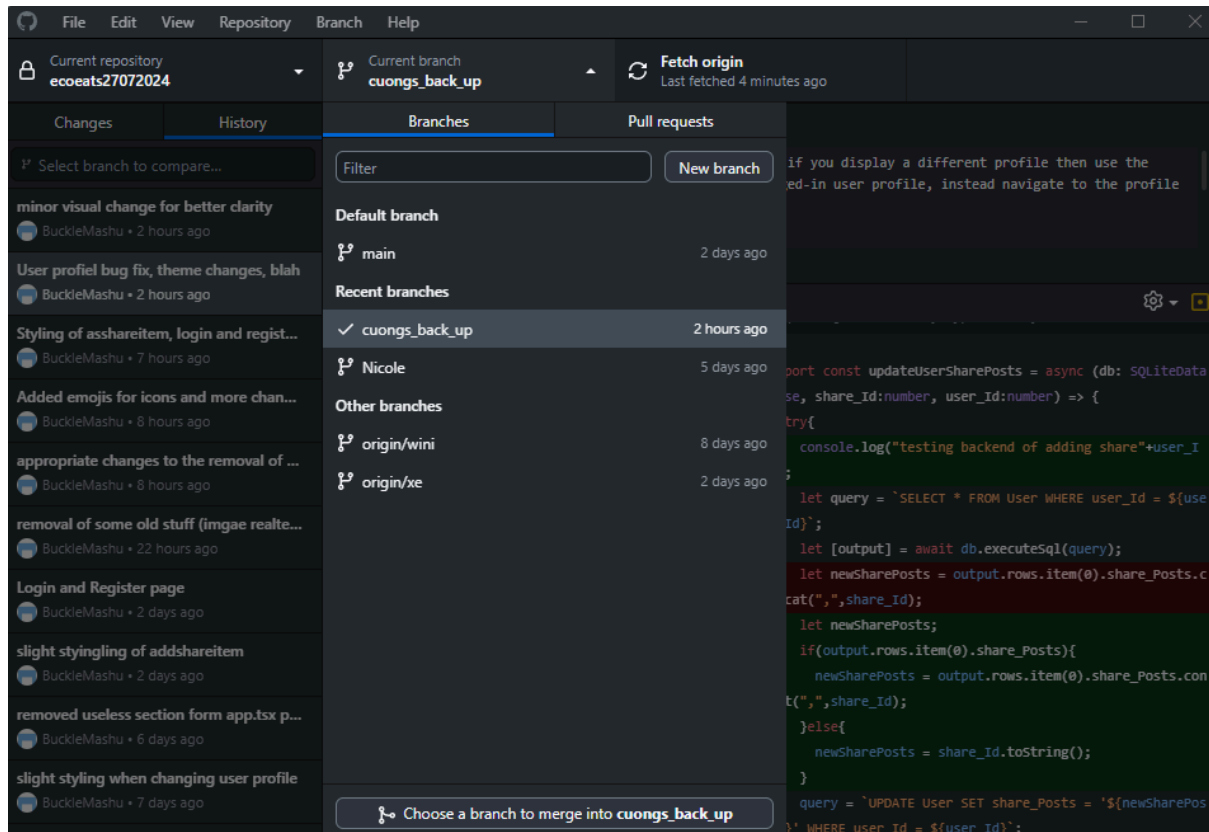
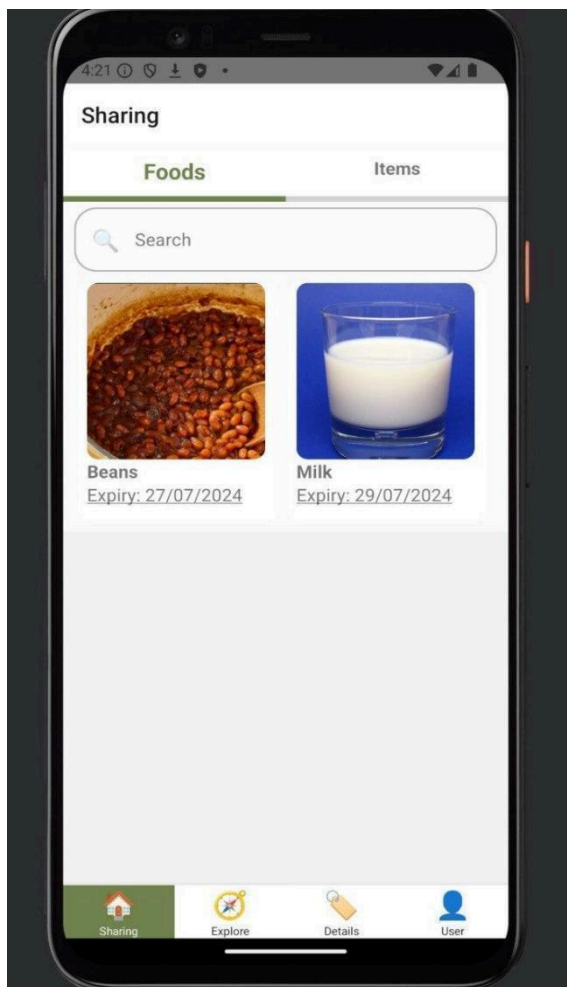


Figure 2.13: A screenshot of our group's GitHub progress as of 19/8/2024

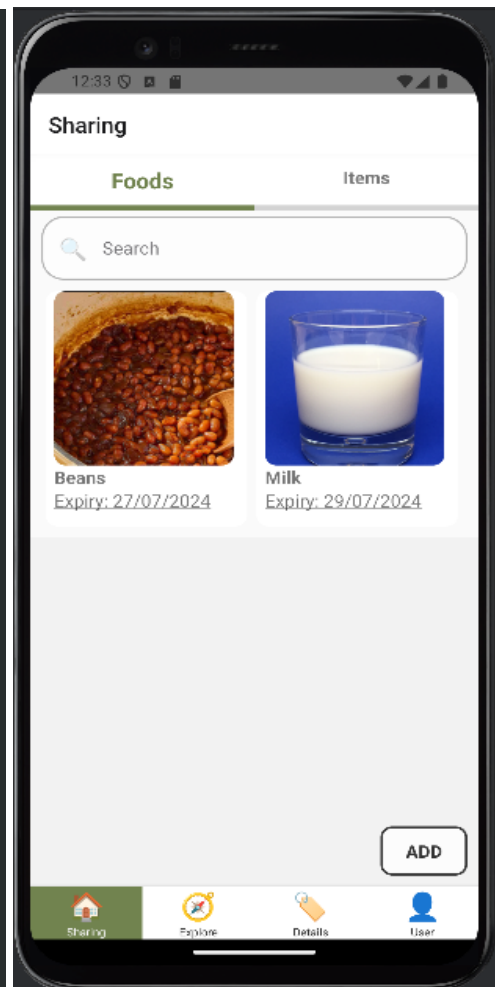
Additionally, the capability to create new branches ensures the main branch is not constantly updated to minimise code "clashing" issues continuously and allows each of us to have a base copy to fall back to if a change or a merge ruins a section of the application.

Design Details

- Share page (Home page)

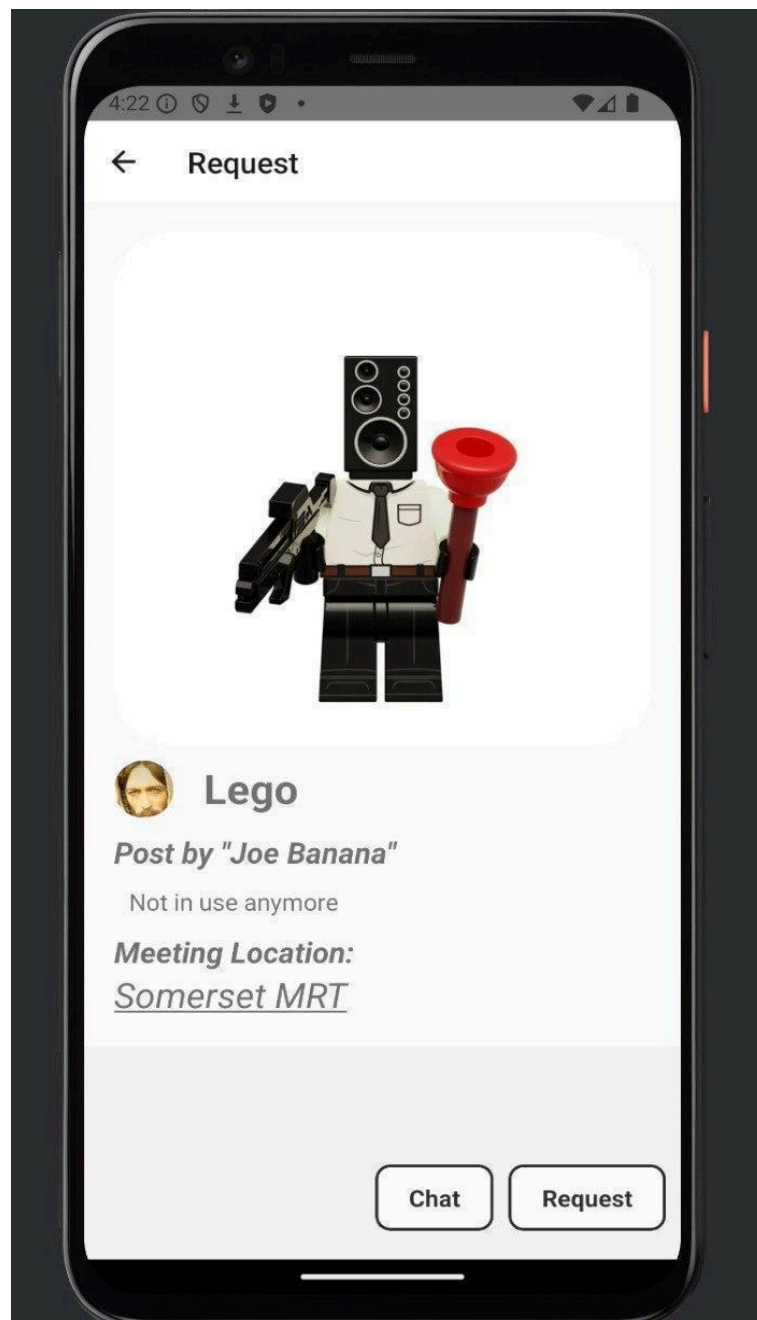


“Not logged in”

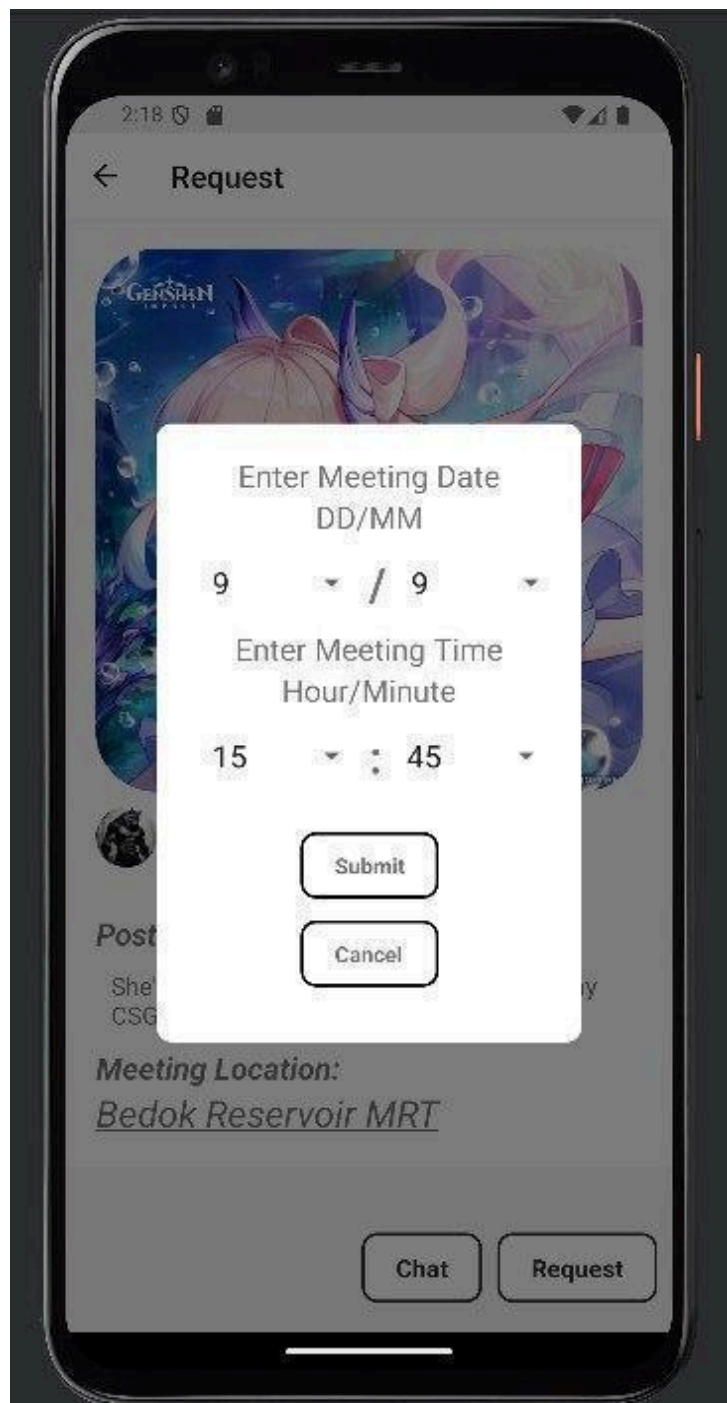


“Logged in”

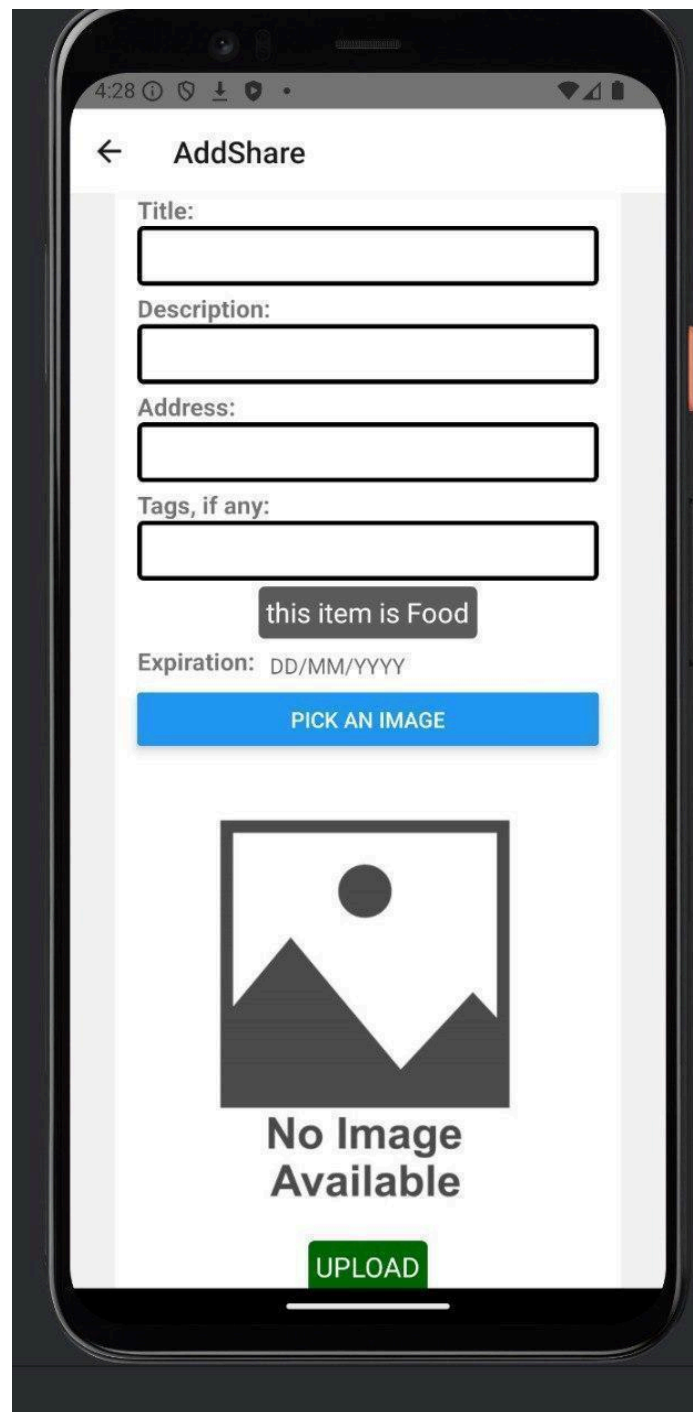
- Share page – Post



- Share page - Post's request popups



- Share page - Add post



The screenshot shows a mobile application interface for adding a post. At the top, there is a status bar with the time 4:28 and various icons. Below the status bar is a header with a back arrow and the text "AddShare". The main form consists of several input fields: "Title:", "Description:", "Address:", and "Tags, if any:". Below these fields is a text input containing "this item is Food". Underneath is an "Expiration:" label followed by a date input field showing "DD/MM/YYYY". A blue button labeled "PICK AN IMAGE" is positioned below the expiration field. Below the button is a placeholder image showing a landscape with mountains and a sun. Below the placeholder image, the text "No Image Available" is displayed. At the bottom of the form is a green button labeled "UPLOAD".

4:28

← AddShare

Title:

Description:

Address:

Tags, if any:

this item is Food

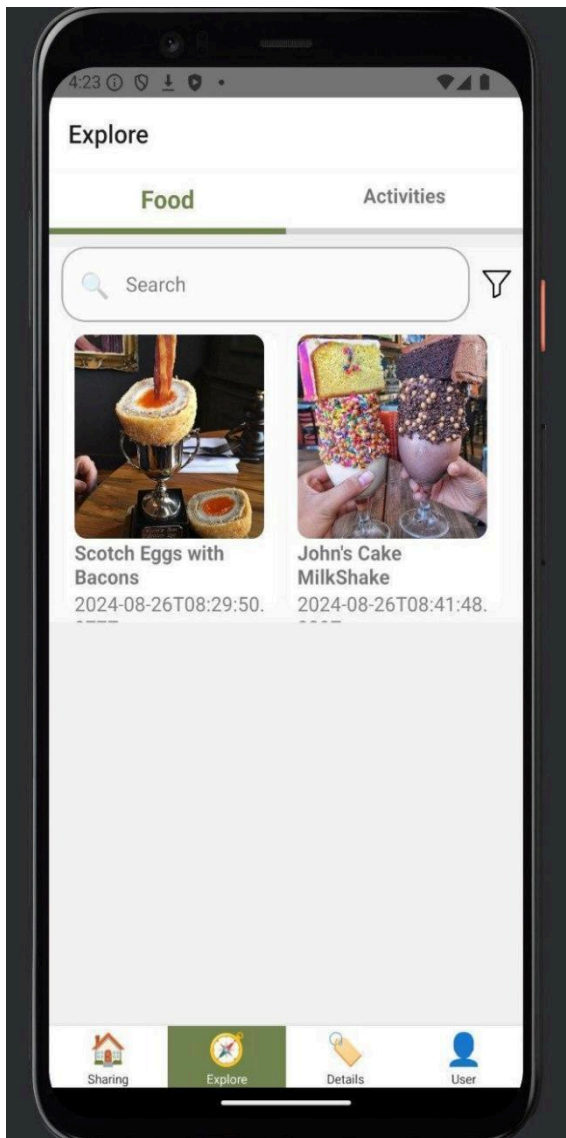
Expiration: DD/MM/YYYY

PICK AN IMAGE

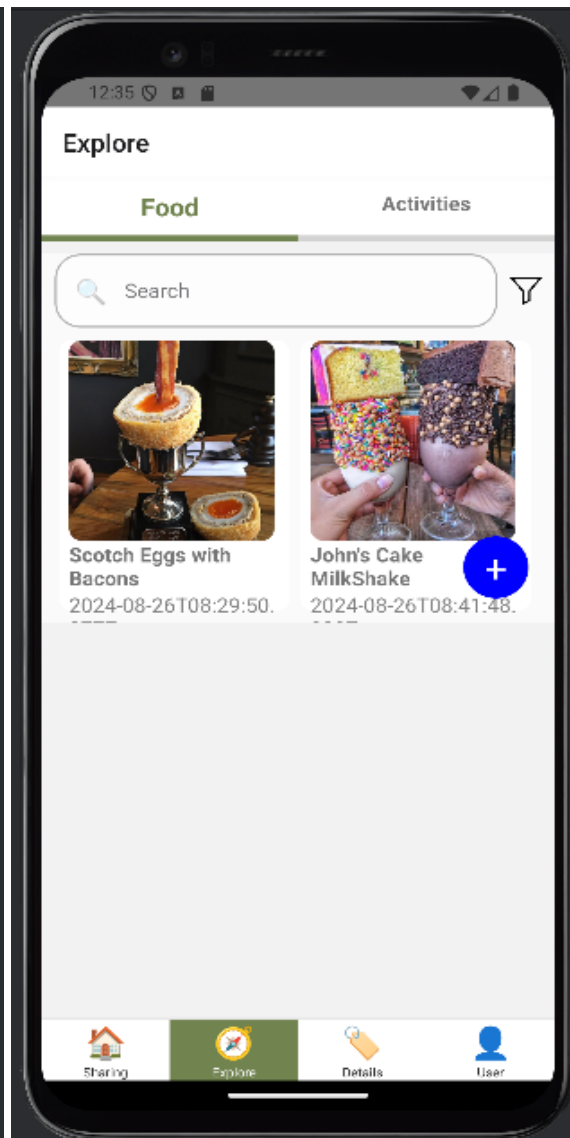
No Image Available

UPLOAD

- Explore page

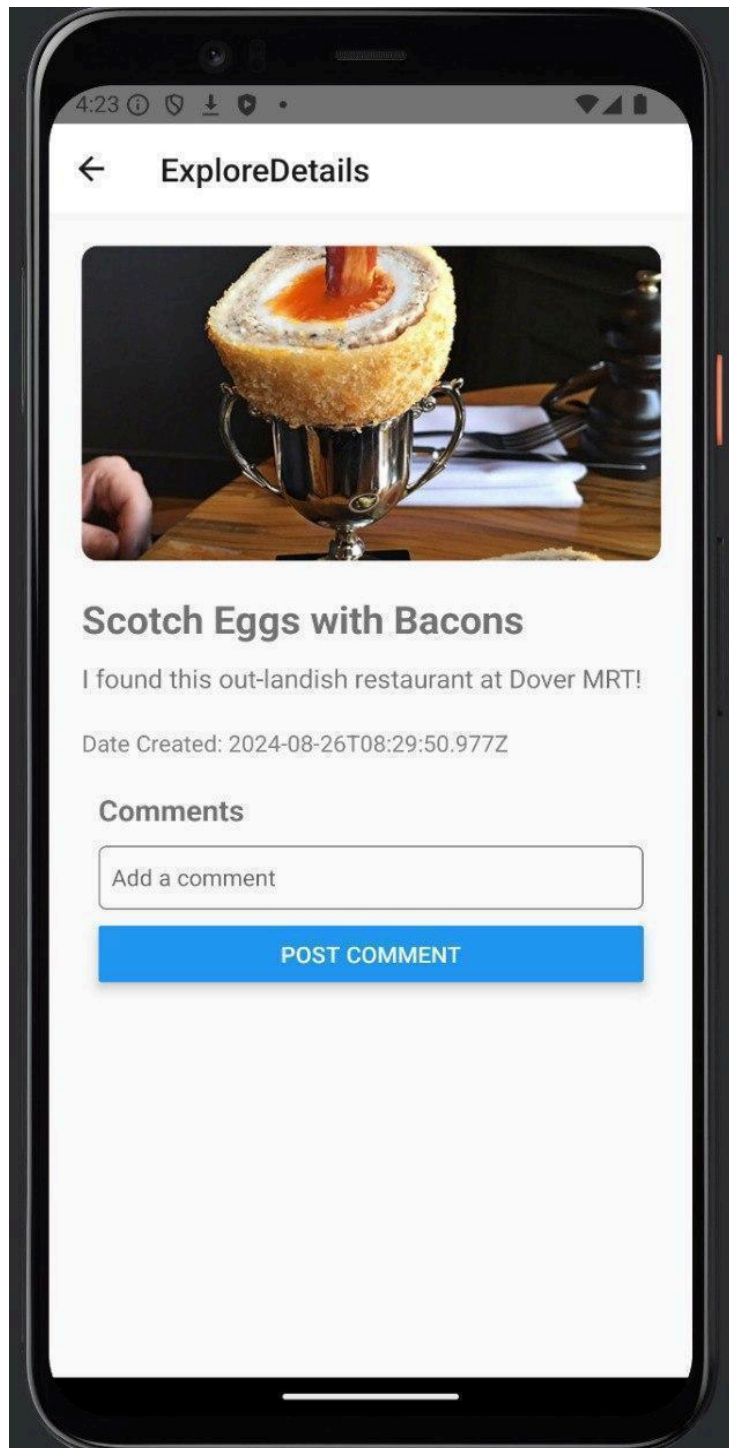


“Not logged in”



“Logged in”

- Explore page – Post



- Explore page - Add post

4:28

← AddExplorePost

Title:

Enter explore post title

Description:

Enter explore post description

Category:

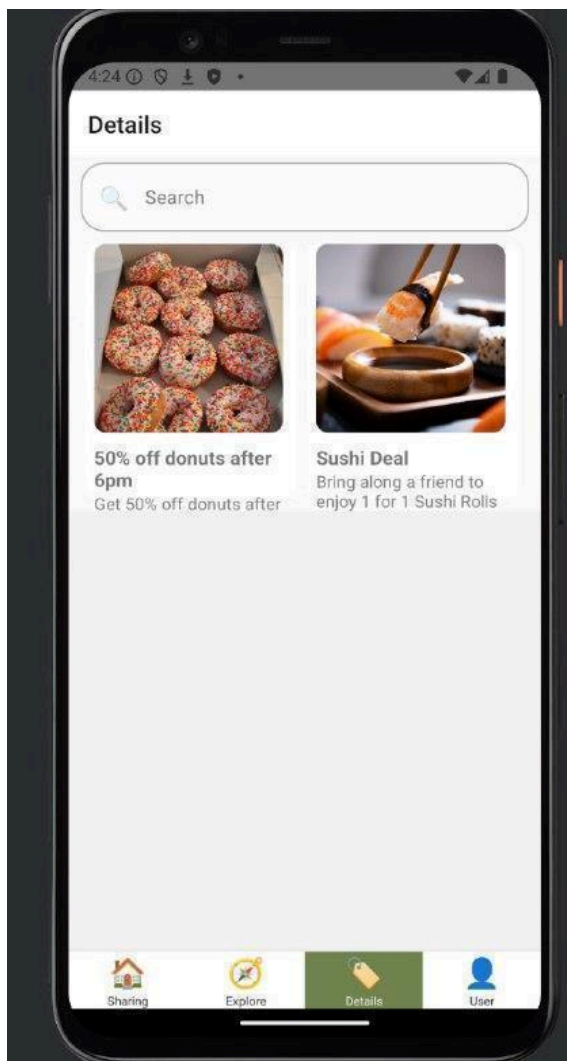
Restaurant

Picture:

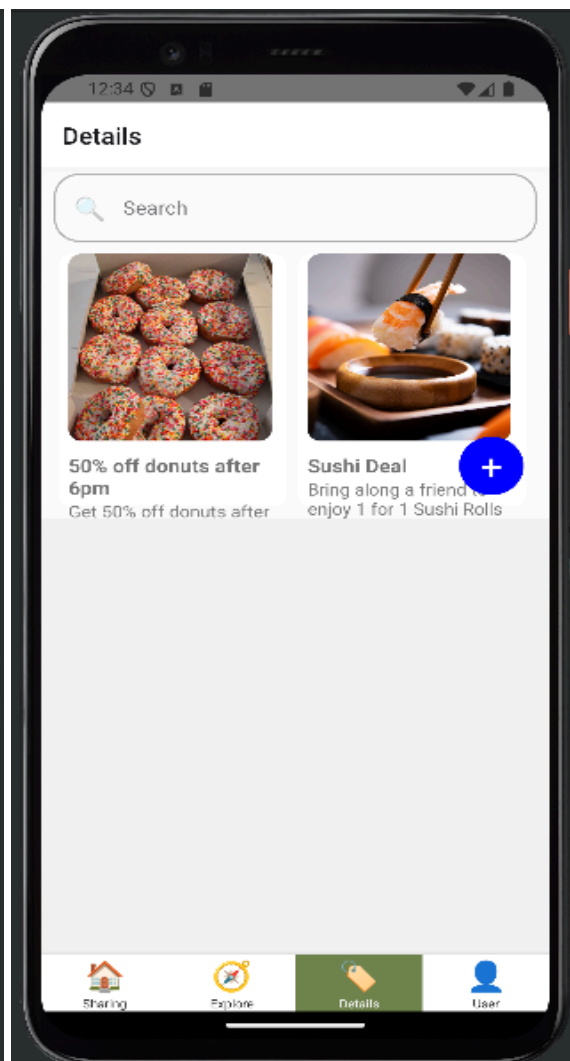
Pick an Image

SAVE POST

- Deals page

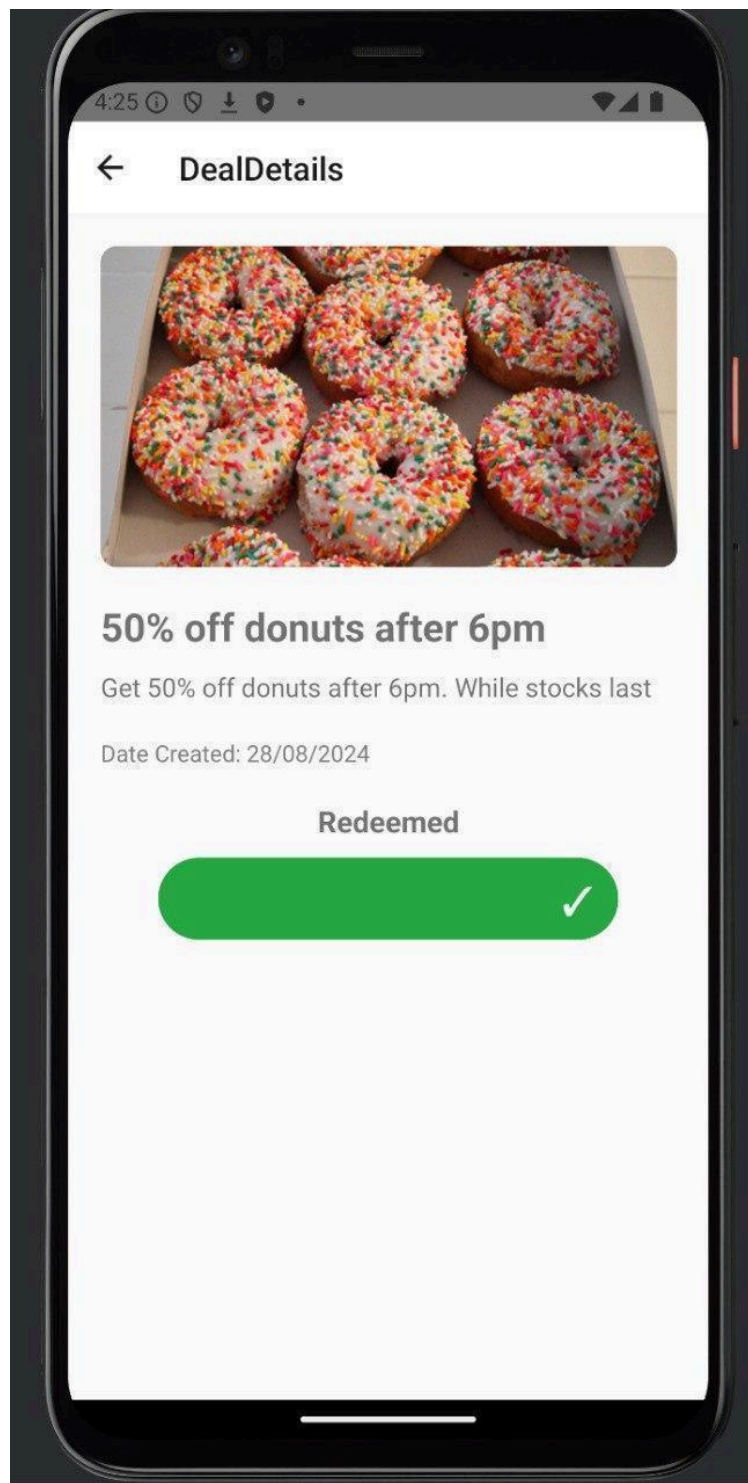


“Not logged in”

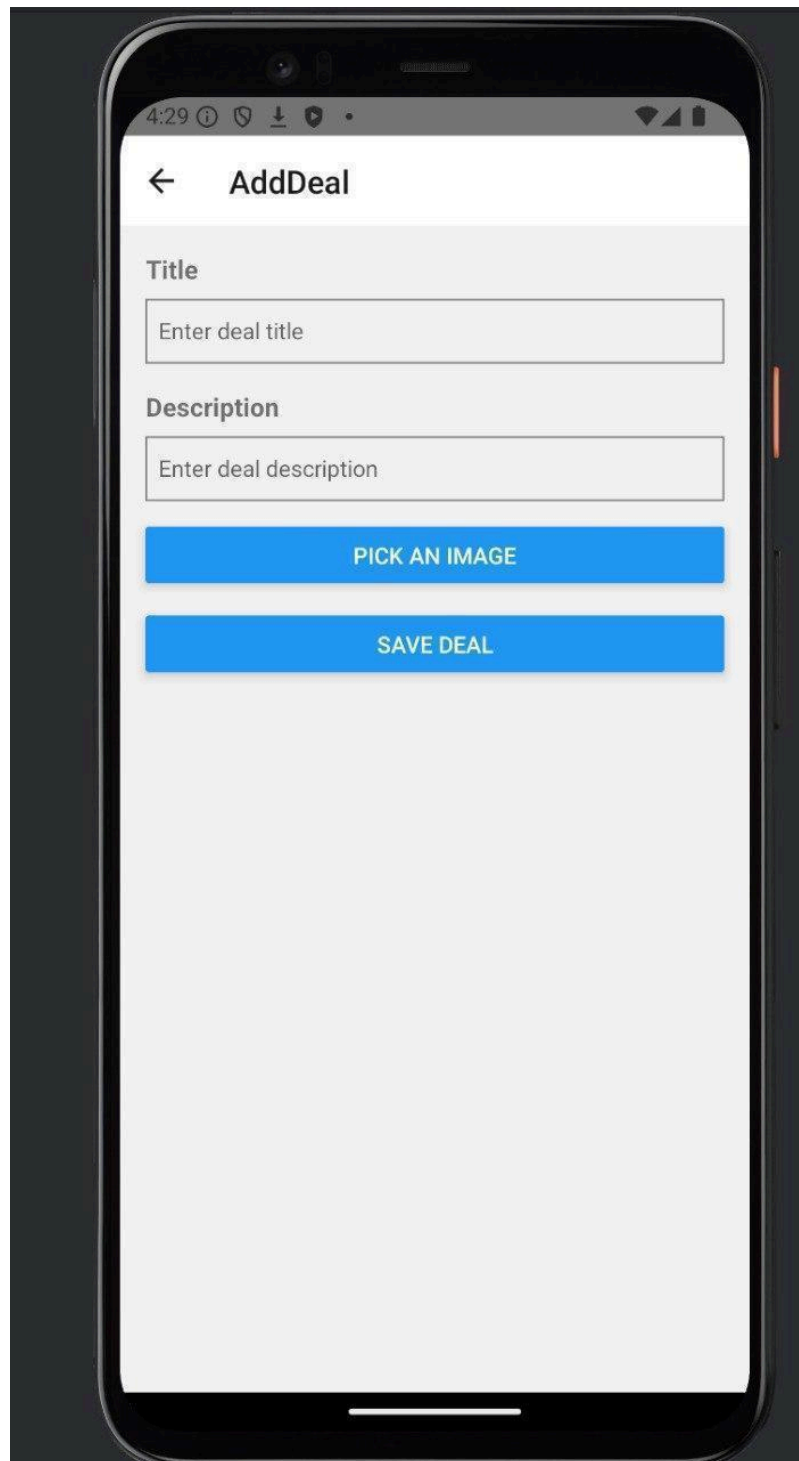


“Logged in”

- Deals page – Post



- Deals page - Add deal

A mobile application interface for adding a deal. The screen is titled 'AddDeal' with a back arrow on the left. It features two text input fields: 'Title' with the placeholder 'Enter deal title' and 'Description' with the placeholder 'Enter deal description'. Below these fields are two blue buttons: 'PICK AN IMAGE' and 'SAVE DEAL'. The status bar at the top shows the time 4:29 and various icons. The phone's home indicator bar is visible at the bottom.

4:29 ⓘ 🔒 ⬇️ 🔒 •

← AddDeal

Title

Enter deal title

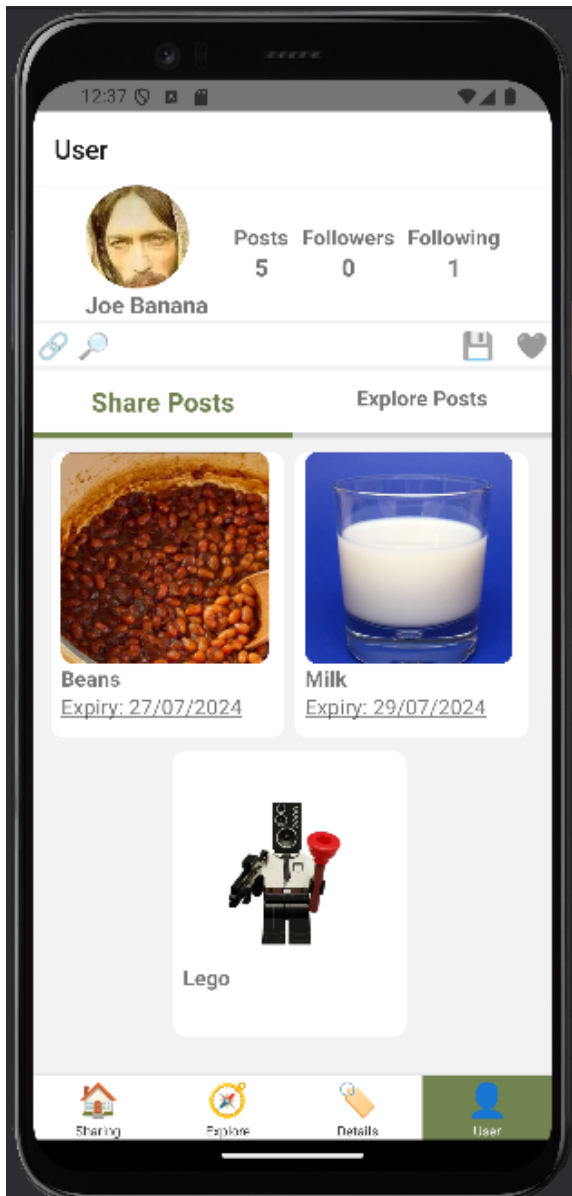
Description

Enter deal description

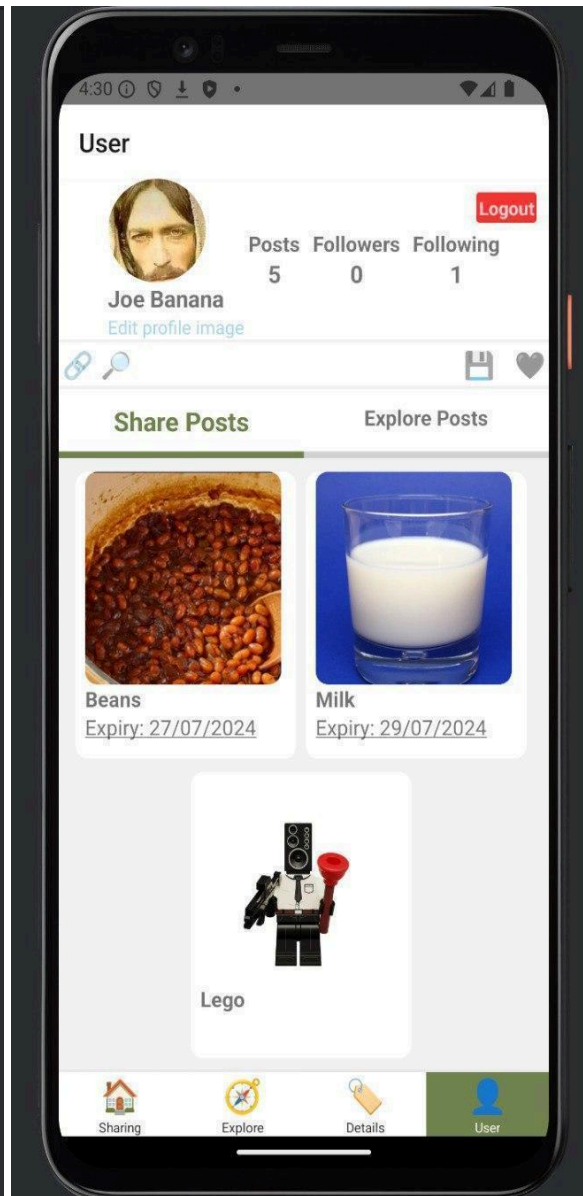
PICK AN IMAGE

SAVE DEAL

- User page

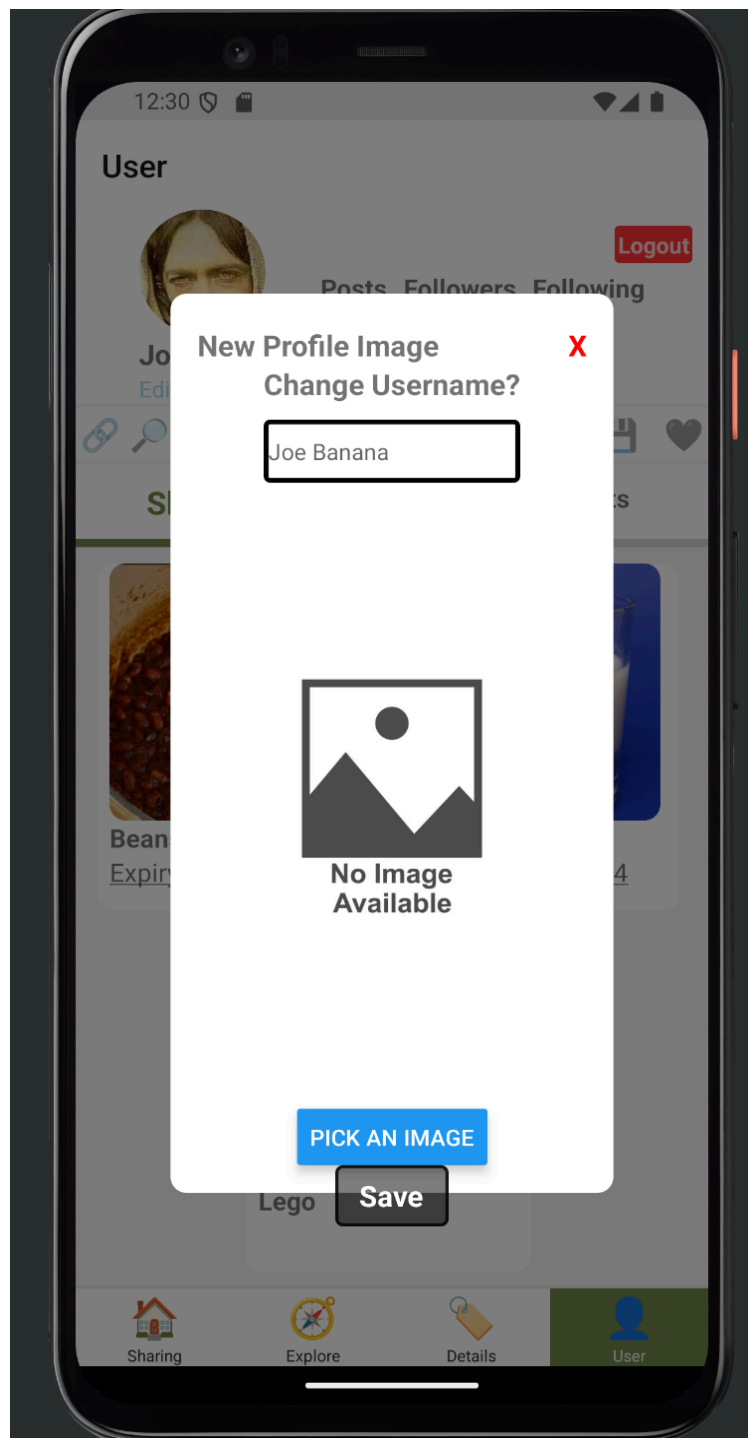


“Other user’s profile”

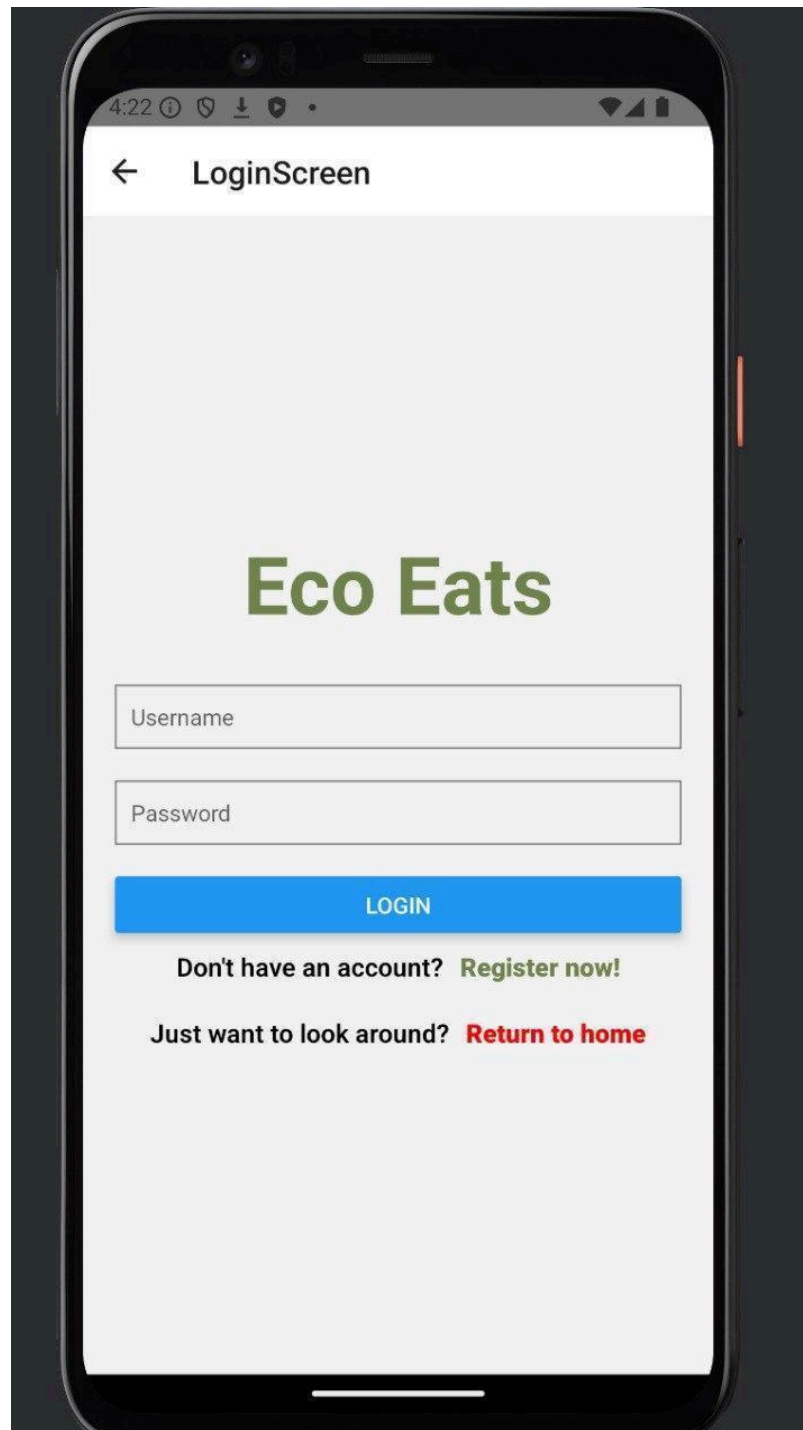


“Logged user’s profile”

- User page - change profile picture and/or name popups



- Login page



- Register page

The image shows a mobile application interface for a registration page. At the top, there is a status bar with the time 4:26 and various icons. Below the status bar is a header with a back arrow and the text 'RegisterScreen'. The main content area has a light gray background. In the center, the app's name 'Eco Eats' is displayed in a large, bold, green font. Below the name are three input fields for 'Username', 'Password', and 'Email'. Under these fields is a question 'Whats the purpose of this account?' followed by two buttons: 'Normal Usage' (highlighted in blue) and 'Business' (white with a gray border). Below the buttons is a large blue button labeled 'REGISTER'. At the bottom, there are two lines of text: 'Have an account? Log in now!' and 'Just want to look around? Return to home'.

4:26 ⓘ 🔒 ⚡ 🔋

← RegisterScreen

Eco Eats

Username

Password

Email

Whats the purpose of this account?

Normal Usage Business

REGISTER

Have an account? **Log in now!**

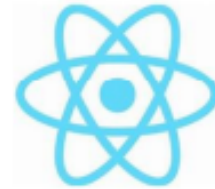
Just want to look around? **Return to home**

Development Process Details

Required modules

React Native

React Native is a popular framework developed by Facebook that enables developers to build mobile applications using JavaScript and React. It allows for the development of cross-platform applications with a single codebase, which can run on both iOS and Android devices.



React Native served as the foundation of our app development, enabling us to build a cross-platform application with a single codebase. This significantly streamlined the development process. However, setting up the development environment proved challenging, especially for those unfamiliar with emulators and virtual machines.

To overcome this, we standardised our project initialization process and ensured all team members were adept at using the necessary tools. We employed several libraries to enhance our React Native project. These included React Native (react-native), which provided the core framework; React Native File System (react-native-fs) for handling file operations and React Native Image Picker (react-native-image-picker) for handling image selection from the device. These libraries provided essential functionality but also introduced their own set of challenges, which we addressed through careful implementation and problem-solving.

Each of these libraries brought specific benefits but also required us to tackle various challenges, such as navigating between pages, managing file storage, and ensuring consistent UI across devices.

SQLite



SQLite is a lightweight, self-contained SQL database engine that is widely used for local data storage in mobile applications. It is known for its simplicity and ease of integration, making it a popular choice for mobile developers.

For local data storage, we selected SQLite and utilised the react-native-sqlite-storage library to integrate it into our app. The initial setup presented significant hurdles. Our first attempt to store the database in the Android assets folder failed because it did not support row insertion during runtime. Additionally, installing the app resulted in the creation of an empty database file, complicating our data management. To address these issues, we implemented a workaround by manually copying a pre-populated database file into the app's data directory. This solution, while effective during development, had limitations for production deployment. Despite these challenges, this approach allowed us to create a functional database capable of handling data operations effectively.

Axios



Axios is a promise-based HTTP client for JavaScript that simplifies the process of making network requests. It is widely used for interacting with APIs and handling asynchronous operations.

Axios was chosen to handle HTTP requests, particularly for interacting with APIs like Imgur. As a promise-based library, Axios streamlined our process for making network requests and managing asynchronous operations. Integrating Axios into the app was relatively straightforward, but ensuring flexible handling of network requests, especially for image uploads, was crucial.

Imgur API

Imgur API is a web service provided by Imgur, a popular image-sharing platform, which allows developers to upload, retrieve, and manage images. The Imgur API was integrated to facilitate image uploads and storage, which was a key feature of our app.



Initially, we struggled with displaying dynamic images stored as local directory URLs in variables, due to React Native's requirement for static image handling. To resolve this, we implemented a script to map all images to their corresponding local directory URLs. This resulted in the issue that we were unable to add in new images dynamically, since the project's root folder can't be access during the application runtime.

Subsequently, we moved to using Imgur to store/host our application images. Integrating the Imgur API allowed users to upload images, which allows us to receives the image's Imgur URL and then store it in the database. This process allows us to display all images of all posts across different devices without issue. This solution not only addressed the limitations of static image handling but also provided a scalable and flexible method for managing user-generated content.

Application Structure

The /models folder, which contains index.ts, defines the data structures we use across the application. This ensured that our data remains consistent and reliable across different pages, which is essential for smooth integration with other parts of the app.

The /modules folder contains modules that play a vital role in handling repeated sections in the application such as displaying posts.

These modules are designed to keep the functionality modular and manageable.

The /screens folder is where we've put all the different pages to display and for users to interact with.

Finally, in our root folder, we have key files such as App.tsx and db-service.ts which is to set up the navigation system of the application and to store all back-end SQLite commands for all of the pages respectively.

As for how everything works together, it begins with the App.tsx. When a user wants to navigate to the UserScreen.tsx, the App.tsx's BottomTabNavigator helps users switch to the desired page. The App.tsx will then provide the parameter value (userId) which is then used to retrieve the profile details (the queries are stored in the db-service.ts). With all of the necessary information, the profile details are then displayed on the "User" page accordingly.

```

✓ ECOEATS27072024
  > __tests__
  > .bundle
  > android
  > ios
  > node_modules
  ✓ src
    > images
    ✓ models
      TS index.ts
    ✓ modules
      EcoEatsDeals.tsx
      EcoEatsExplore.tsx
      EcoEatsShare.tsx
      RedeemSlider.tsx
    ✓ screens
      EcoEatsDealsScreen.tsx
      EcoEatsExplorePostScreen.tsx
      EcoEatsShareItem.tsx
      DealDetailsScreen.tsx
      DetailsScreen.tsx
      ExploreDetailsScreen.tsx
      ExploreScreen.tsx
      LoginScreen.tsx
      RegisterScreen.tsx
      RequestScreen.tsx
      SharingScreen.tsx
      UserScreen.tsx
      ViewDataScreen.tsx
    readmeForSRCfile.md
    JS requestDateTimePicker.js
    uploadToImgur.tsx
    .eslintrc.js
    .gitattributes
    .gitignore
    JS .prettierrc.js
    {} .watchmanconfig
    {} app.json
    App.tsx
    Babel.config.js
    TS db-service.ts
```

Code Details

Resources Used

This section provides an overview of several essential libraries used in React Native development. These libraries each serve a specific purpose of page navigation, enhancing the functionality, user experience, and data management capabilities of our mobile React Native 'EcoEats' application.

Front-end Resources

- [@react-navigation](#)

'@react-navigation' is a robust library for managing navigation within EcoEats. It provides a flexible framework for us to manage different types of navigation patterns, such as stack navigation and tab navigation. Therefore, it is essential to build a seamless navigation experience for all different pages.

Additionally, to further aid with the navigation, we implemented a bottom navigation bar through the library '@react-navigation/bottom-tabs'. It is to provide a flexible user-friendly navigation interface and also helps the user identify the main pages and services of our application.

```
// Main Tab Navigator
Tabnine: Test | Fix | Explain | Document | Ask
function MainTabNavigator() {
  return (
    <Tab.Navigator
      initialRouteName="Sharing"
      screenOptions={{
        tabBarActiveTintColor: '#f5f5f5',
        tabBarInactiveTintColor: '#2e2e2e',
        tabBarActiveBackgroundColor: '#1a1919',
      }}
    >
      <Tab.Screen
        name="Sharing"
        component={SharingScreen}
        options={{
          tabBarLabel: 'Sharing',
          tabBarIcon: ({ color, size }) => (
            <Icon name="info-circle" color={color} size={size} />
          ),
        }}
      />
    </Tab.Navigator>
  )
}
```

Figure 2.1 : An example of '@react-navigation/bottom-tabs' in App.tsx

```

// Main App Component
const App = () => {
  useEffect(() => {
    copyDatabase();
  });

  // copyImage();
  // }, []); // The empty dependency array ensures this runs only once
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen
          name="MainTabs"
          component={MainTabNavigator}
          options={{ headerShown: false }}
        />
        <Stack.Screen name="Explore" component={ExploreScreen} />
        <Stack.Screen name="Details" component={DetailsScreen} />
        <Stack.Screen name="Sharing" component={SharingScreen} />
        <Stack.Screen name="Request" component={RequestScreen} />
        <Stack.Screen name="AddShare" component={AddShareScreen} />
        <Stack.Screen name="User" component={UserScreen} />
        <Stack.Screen name="AddDeal" component={AddDealScreen} />
        <Stack.Screen name="DealDetails" component={DealDetailsScreen} />
        <Stack.Screen name="AddExplorePost" component={AddExplorePostScreen} />
        <Stack.Screen name="ExploreDetails" component={ExploreDetailsScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
};

```

Figure 2.2: An example of '@react-navigation' in App.tsx

```

console.log(await saveNewShareItem(db,itemType,title,tags,address,p
await updateUserSharePosts(db,shareIdnew[0]["insertId"],user_Id);

navigation.navigate('MainTabs', {screen: 'Sharing'});
} catch(error){
  console.error(error);
}

```

Figure 2.3: An example of '@react-navigation' in AddShareItem.tsx

Overall, this library has allowed us to quickly create the navigation system easily and quickly for EcoEats.

Back-end Resources

- React-native-image-picker

The react-native-image-picker library facilitates image and video selection from the device's library or directly captures new media using the device's camera.

```
const pickImage = () => {
  launchImageLibrary({ mediaType: 'photo' }, async (response) => {
    if (response.didCancel) {
      console.log('User cancelled image picker');
    } else if (response.errorCode) {
      console.log('ImagePicker Error:', response.errorMessage);
    } else {
      if (response.assets && response.assets[0].uri) {
        const uri = response.assets[0].uri;
        setSelectedImageUri(uri);
      }
    }
  });
};
//-----
```

Figure 2.4: pickImage in 'addShareItem.tsx'

For functionalities that allow the user to select an image from their device, this library will help by returning the chosen image's Uniform Resource Identifier (URI). The images' URI will be used as input for other functions to perform their tasks. This can range from simply displaying the image to uploading the image to Imgur.

```
<Button title="Pick an Image" onPress={pickImage} />
{imageUrl &&
  <Image source={{ uri: selectedImageUri || 'https://i.imgur.com/50exbMa.png' }} style={styles.image}/>
}
```

Figure 2.5: Implementation of the pickImage function

- React-native-fs

The 'react-native-fs' library provides access to the device's file system, enabling React Native applications to perform various file operations, including reading, writing, updating, and deleting files.

```
// Function to copy the database from assets to internal storage
export const copyDatabase = async () => {
  try {
    const dbExists = await RNFS.exists(internalDbPath);
    console.log('Internal DB Path:', internalDbPath); // Log the internal DB path
    console.log('Exists:', dbExists);
    if (dbExists) {
      await RNFS.unlink(internalDbPath); // Delete the existing database
      console.log('Old database deleted from internal storage');
    }

    await RNFS.copyFileAssets(`www/${dbName}`, internalDbPath);
    console.log('Database copied to internal storage');
    console.log(internalDbPath);
  } catch (error) {
    console.error('Error copying database: ', error);
    throw error;
  }
};

// Function to get the database connection
export const getEcoEatsDBConnection = async (): Promise<SQLiteDatabase> => {
  const dbPath = `${RNFS.DocumentDirectoryPath}/${dbName}`;
  console.log('Connecting to database at:', dbPath);
  return openDatabase({
    name: dbName,
    // createFromLocation: '~www/ecoeats.db',
    location: 'default', // Ensure it uses the correct location
  });
};
```

Figure 2.6: To copy the database into the emulated device's storage

This library allows us to initiate making a copy of our SQLite database, 'ecoeats.db' stored in the android's asset folder, and paste it into the emulated android device's storage space. Having the db in the Android device will allow us to freely demonstrate the functionality of the application. Additionally, this sets the groundwork for the implementation of a MySQL-based service like Amazon RDS into our application if we are to expand our application and push it to the public market.

The 'react-native-fs' library is also a core component of the 'uploadimageToImgur' function.

- Axios and Imgur API

'Axios' is a promise-based HTTP client used for making HTTP requests from within React Native applications. It simplifies the process of interacting with external APIs, handling requests, and responses efficiently. In turn, 'Axios' helps us to utilise The Imgur API.

Integrating Imgur's image hosting and sharing capabilities into our applications, enabling seamless interaction with image display. For us, it allows us to simulate an image hosting service with the downside of having the images stored being public due to Imgur policy. Once again, this sets up the groundwork for us to utilise image hosting services such as Amazon S3 (Simple Storage Service).

```
import axios from 'axios';
import RNFS from 'react-native-fs';

const uploadImageToImgur = async (imageUri:any) => {
  const clientId = 'XXXXXXXXXX';

  try {
    // Read the image file as a base64 string
    const base64Image = await RNFS.readFile(imageUri, 'base64');

    // Upload the image to Imgur
    const response = await axios.post(
      'https://api.imgur.com/3/image',
      {
        image: base64Image,
        type: 'base64',
      },
      {
        headers: {
          Authorization: `Client-ID ${clientId}`,
        },
      },
    );

    if (response.data.success) {
      const imageId = response.data.data.id;
      const imageUrl = response.data.data.link;
      console.log('Image uploaded to Imgur:', imageUrl);
      return [imageUrl, imageId];
    }
  } catch (error) {
    console.error('Error uploading image to Imgur:', error);
  }
};

export default uploadImageToImgur;
```

Figure 2.7: To upload image to Imgur

- React-native-sqlite-storage

The react-native-sqlite-storage library provides an SQLite database interface for React Native applications, enabling us to perform SQL queries on 'ecoeats.db' for data retrieval and storage.

```
export const getSharePage = async (
  db: SQLiteDatabase,
  type: number | undefined,
  keyword: string,
  id: string | undefined
): Promise<share_page[]> => {
  try {
    const sharePageItems: share_page[] = [];
    let query;
    if (keyword === '') {
      query = `SELECT * FROM Share WHERE type=${type}`;
    } else {
      query = `SELECT * FROM Share WHERE type=${type} AND title LIKE '%${keyword}%'`;
    }
    console.log(id);
    console.log(type);
    console.log(keyword);
    if (id && type === undefined && keyword === '') {
      query = `SELECT * FROM Share WHERE share_id IN (${id})`;
    } else if (id === null && type === undefined && keyword === '') {
      return [];
    }

    const results = await db.executeSql(query);
    results.forEach(result => {
      for (let index = 0; index < result.rows.length; index++) {
        sharePageItems.push(result.rows.item(index));
      }
    });
    return sharePageItems;
  } catch (error) {
    console.error(error);
    throw Error('Failed to get share pages items');
  }
};
```

Figure 2.8: To retrieve share posts according to the input filter.